

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN ĐIỆN TỬ
BỘ MÔN KỸ THUẬT ĐIỆN TỬ - THÔNG TIN



ĐỒ ÁN TỐT NGHIỆP

**XÂY DỰNG HỆ THỐNG TAY KẸP ROBOT
THÍCH NGHI TÍCH HỢP CẢM BIẾN
XÚC GIÁC VÀ TRÍ TUỆ NHÂN TẠO**

NGÀNH CÔNG NGHỆ KỸ THUẬT MÁY TÍNH

Sinh viên: **NGUYỄN ANH MINH**

MSSV: 22119103

TP. HỒ CHÍ MINH – 6/2026

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN ĐIỆN TỬ
BỘ MÔN KỸ THUẬT ĐIỆN TỬ - THÔNG TIN



ĐỒ ÁN TỐT NGHIỆP

**XÂY DỰNG HỆ THỐNG TAY KẸP ROBOT
THÍCH NGHI TÍCH HỢP CẢM BIẾN
XÚC GIÁC VÀ TRÍ TUỆ NHÂN TẠO**

NGÀNH CÔNG NGHỆ KỸ THUẬT MÁY TÍNH

Sinh viên: **NGUYỄN ANH MINH**

MSSV: 22119103

Hướng dẫn: **TS. Vũ Chí Cường**

TP. HỒ CHÍ MINH – 6/2026

THÔNG TIN ĐỒ ÁN TỐT NGHIỆP

1. THÔNG TIN SINH VIÊN

- Họ và tên sinh viên: Nguyễn Anh Minh
- Mã số sinh viên: 22119103
- Họ và tên sinh viên (nếu có):
- Mã số sinh viên (nếu có):
- Lớp: 22119CL3A
- Ngành: Công nghệ Kỹ thuật Máy tính
- Khoa: Điện-Điện tử
- Email: 22119103@student.hcmute.edu.vn
- Số điện thoại: 0877511576

2. THÔNG TIN ĐỀ TÀI

- Tên đề tài (Tiếng Việt): **Xây dựng hệ thống tay kẹp robot thích nghi tích hợp cảm biến xúc giác và trí tuệ nhân tạo**

- Tên đề tài (Tiếng Anh): **Design of an Adaptive Robotic Gripper System Using Tactile Sensors and Artificial Intelligence**

- Loại hình:

- ☐ Nghiên cứu
- ☒ Thiết kế
- ☒ Chế tạo
- ☐ Mô phỏng
- ☐ Khác:

- Thời gian thực hiện:

- Ngày giao đề tài: 01/02/2026
- Ngày hoàn thành: 11/06/2026

3. GIẢNG VIÊN HƯỚNG DẪN

- Họ và tên: Vũ Chí Cường
- Học hàm/Học vị: Tiến sĩ
- Đơn vị công tác: Trường Đại học Công nghệ Kỹ thuật TP.HCM
- Email: cuongvc@hcmute.edu.vn

4. TÓM TẮT NỘI DUNG ĐỒ ÁN

Đề tài thực hiện thiết kế và xây dựng hệ thống tay kẹp robot thích nghi có tích hợp cảm biến xúc giác và trí tuệ nhân tạo. Hệ thống sử dụng camera kết hợp mô hình YOLO để nhận diện vật thể, đồng thời sử dụng mô hình mạng Perceptron nhiều lớp MLP (Multilayer Perceptron) để dự đoán lực kẹp ban đầu và đánh giá trạng thái kẹp dựa trên dữ liệu cảm biến. Từ kết quả nhận diện và phản hồi xúc giác, tay kẹp có thể tự điều chỉnh lực kẹp nhằm hạn chế hiện tượng trượt vật và tránh tác động lực quá mức lên vật thể.

5. KẾT QUẢ ĐẠT ĐƯỢC

- ■ Báo cáo đồ án tốt nghiệp.
- ■ Mô hình / Thiết bị / Phần mềm hoàn chỉnh.
- ■ Mã nguồn chương trình.
- ■ Kết quả mô phỏng và thực nghiệm.
- □ Bài báo khoa học (nếu có).
- □ Sản phẩm khác:

6. KIỂM TRA ĐẠO VĂN

- Phần mềm kiểm tra: Turnitin
- Ngày kiểm tra: 21/06/2026
- Tỷ lệ tương đồng (Similarity Index): 6 %
- Kết luận:
 - Đạt yêu cầu theo quy định của Nhà trường.
 - Không đạt yêu cầu theo quy định của Nhà trường.

7. CAM KẾT CỦA SINH VIÊN

Tôi cam kết rằng đồ án tốt nghiệp này là kết quả nghiên cứu và thực hiện của riêng tôi dưới sự hướng dẫn của giảng viên hướng dẫn. Các tài liệu tham khảo, số liệu, hình ảnh và kết quả nghiên cứu của người khác đều được trích dẫn đầy đủ theo đúng quy định. Tôi hoàn toàn chịu trách nhiệm trước Nhà trường về tính trung thực và tính hợp pháp của nội dung đồ án.

Sinh viên thực hiện

(Ký và ghi rõ họ tên)



Nguyễn Anh Minh

BẢN NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

Họ và tên sinh viên/MSSV: Nguyễn Anh Minh - 22119103

Họ và tên sinh viên/MSSV:

Ngành: Công nghệ Kỹ thuật Máy tính

Tên đề tài: Xây dựng hệ thống Robot tay gắp thích nghi sử dụng cảm biến xúc giác ứng dụng AI

Giảng viên hướng dẫn: TS. Vũ Chí Cường

1. Nhận xét chung về tinh thần, thái độ làm việc, nội dung và kết quả thực hiện đề tài

Hệ thống đạt yêu cầu về tinh thần và thái độ làm việc đạt yêu cầu

3. Đánh giá chung: Mức độ hoàn thành các mục tiêu đề ra:

☐ Xuất sắc ☐ Tốt ☒ Khá ☐ Đạt yêu cầu ☐ Chưa đạt yêu cầu

4. Kết luận

☒ Đồng ý cho sinh viên bảo vệ Đồ án Tốt nghiệp trước Hội đồng đánh giá.

☐ Không đồng ý cho sinh viên bảo vệ Đồ án Tốt nghiệp trước Hội đồng đánh giá.

Lý do (nếu không đồng ý):

Đạt yêu cầu của đồ án tốt nghiệp

TP. Hồ Chí Minh, ngày 26 tháng 6 năm 2026

GIẢNG VIÊN HƯỚNG DẪN

(Ký và ghi rõ họ tên)

Vũ Chí Cường

Tp. HCM, ngày 11 tháng 07 năm 2026

BẢN GIẢI TRÌNH CHỈNH SỬA ĐỒ ÁN TỐT NGHIỆP
SAU PHẢN BIỆN
NGÀNH: CNKT MÁY TÍNH (HỆ ĐT CLC)

1. Tên đề tài: Xây dựng hệ thống tay kẹp robot thích nghi tích hợp cảm biến xúc giác và trí tuệ nhân tạo
2. Tên sinh viên: Nguyễn Anh Minh MSSV: 22119103
3. GVHD: TS. Vũ Chí Cường
4. Hội đồng bảo vệ HD 2, phòng A105, ngày 06 tháng 07 năm 2026
5. Giải trình chỉnh sửa báo cáo đồ án tốt nghiệp theo ý kiến của thầy/cô Phản biện:

TT	Nội dung góp ý của thầy/cô Phản biện	Kết quả chỉnh sửa, bổ sung	Ghi chú
1	Điều chỉnh tên đề tài cho phù hợp.	Đã chỉnh tên thành “Xây dựng hệ thống tay kẹp robot thích nghi tích hợp cảm biến xúc giác và trí tuệ nhân tạo”.	
2	Cảm biến xúc giác Velostat tự chế có nhược điểm về tính phi tuyến và hiện tượng trễ; cần bổ sung bước tiền xử lý hoặc bù nhiễu trước khi đưa dữ liệu vào huấn luyện MLP.	Đã bổ sung mục tiền xử lý dữ liệu cảm biến xúc giác trong phần 3.3.5, trang 58.	
3	Mô hình MLPRegressor có chỉ số R^2 âm; cần đánh giá lại bộ đặc trưng đầu vào vì riêng bounding-box từ YOLO chưa đủ để nội suy chính xác lực kẹp.	Đã bổ sung phần đánh giá sau Bảng 4.1 trong phần 4.1.3, trang 74.	
4	Tài liệu tham khảo cần được trích dẫn theo thứ tự tăng dần trong luận văn. Không sử dụng nguồn từ các trang mạng.	Đã bổ sung thêm trích dẫn vào nội dung các chương đồng thời xóa các tài liệu trích dẫn từ các trang mạng.	

Xác nhận của Trưởng ngành

PGS.TS. Trương Ngọc Sơn

Nhóm thực hiện báo cáo
(Ký họ và tên)

Nguyễn Anh Minh

LỜI CẢM ƠN

Trước hết, em xin bày tỏ lời cảm ơn sâu sắc đến Thầy Vũ Chí Cường, người đã trực tiếp hướng dẫn, tận tình chỉ bảo và định hướng cho em trong suốt quá trình thực hiện đồ án. Những ý kiến đóng góp quý báu, sự hỗ trợ và tinh thần trách nhiệm của Thầy đã giúp em hoàn thành đề tài một cách nghiêm túc và hiệu quả.

Em xin chân thành cảm ơn quý Thầy, Cô Trường Đại học Công nghệ Kỹ thuật TP. Hồ Chí Minh, đặc biệt là quý Thầy, Cô Khoa Điện Điện Tử và Ngành Công nghệ Kỹ thuật Máy tính đã truyền đạt cho em những kiến thức nền tảng và chuyên môn quý báu trong suốt thời gian học tập tại trường. Đây là hành trang quan trọng giúp em có đủ kiến thức và kỹ năng để hoàn thành đồ án tốt nghiệp.

Em cũng xin gửi lời cảm ơn đến gia đình và bạn bè đã luôn động viên, hỗ trợ và tạo điều kiện thuận lợi cả về tinh thần lẫn vật chất trong suốt quá trình học tập và thực hiện đồ án.

Xin kính chúc quý Thầy, Cô cùng gia đình và bạn bè luôn dồi dào sức khỏe, hạnh phúc và thành công.

Em xin chân thành cảm ơn!

TÓM TẮT

Sự phát triển mạnh mẽ của lĩnh vực robot và trí tuệ nhân tạo trong những năm gần đây đã mở ra nhiều hướng nghiên cứu mới nhằm nâng cao khả năng tương tác và thích nghi của robot với môi trường thực tế. Trong đó, việc trang bị cho robot khả năng cảm nhận thông tin từ quá trình tiếp xúc với vật thể đóng vai trò quan trọng, giúp cải thiện độ chính xác, tính linh hoạt và độ an toàn trong các thao tác. Bên cạnh đó, các kỹ thuật học máy và học sâu ngày càng được ứng dụng rộng rãi để hỗ trợ robot xử lý dữ liệu cảm biến và đưa ra quyết định một cách thông minh hơn.

Xuất phát từ nhu cầu đó, đề tài “Xây dựng hệ thống tay kẹp robot thích nghi tích hợp cảm biến xúc giác và trí tuệ nhân tạo” được thực hiện nhằm nghiên cứu các giải pháp kết hợp giữa cảm biến xúc giác, hệ thống điều khiển robot và các phương pháp trí tuệ nhân tạo. Đề tài tập trung tìm hiểu các nguyên lý cảm nhận lực tiếp xúc, khả năng thu thập và xử lý dữ liệu cảm biến, đồng thời khảo sát những phương pháp có thể hỗ trợ robot thích nghi với các điều kiện làm việc và đặc tính vật thể khác nhau.

Thông qua quá trình nghiên cứu và triển khai, đề tài góp phần cung cấp cái nhìn tổng quan về việc ứng dụng cảm biến xúc giác và trí tuệ nhân tạo trong lĩnh vực robot. Kết quả đạt được là cơ sở để tiếp tục phát triển các hệ thống robot có khả năng tương tác linh hoạt hơn với môi trường, hướng đến những ứng dụng thực tiễn trong công nghiệp, dịch vụ và đời sống trong tương lai.

Mục lục

DANH MỤC HÌNH ẢNH

DANH MỤC BẢNG

DANH MỤC TỪ VIẾT TẮT

1	GIỚI THIỆU	1
1.1	Giới thiệu	1
1.2	Mục tiêu đề tài	2
1.3	Giới hạn đề tài	2
1.4	Phương pháp nghiên cứu	3
1.5	Đối tượng và phạm vi nghiên cứu	4
1.5.1	Đối tượng nghiên cứu	4
1.5.2	Phạm vi nghiên cứu	4
1.6	Bố cục quyển báo cáo	4
2	CƠ SỞ LÝ THUYẾT	6
2.1	Cảm biến xúc giác	6
2.1.1	Khái quát về xúc giác	6
2.1.2	Công nghệ cảm biến xúc giác	6
2.2	Tổng quan về thị giác máy tính (Computer Vision)	9
2.2.1	Định nghĩa	9
2.2.2	Bài toán phát hiện vật thể (Object detection)	10
2.3	Mô hình YOLO	10
2.3.1	Giới thiệu về YOLO	10

2.3.2	Thuật toán phát hiện vật thể của YOLO	11
2.3.3	Tổng quan về mô hình YOLOv8	14
2.4	Tổng quan về học máy (Machine Learning)	15
2.4.1	Khái niệm	15
2.4.2	Phân loại học máy	16
2.5	Tổng quan về học sâu (Deep Learning)	17
2.6	Mạng nơ-ron nhân tạo (Artificial Neural Network)	18
2.6.1	Nơ-ron nhân tạo	18
2.6.2	Mạng nơ-ron nhân tạo (Artificial Neural Network - ANN)	19
2.7	Mô hình Multilayer Perceptron (MLP)	20
2.7.1	Khái niệm	20
2.7.2	Cấu trúc	20
2.7.3	Hàm kích hoạt	22
2.7.4	Thuật toán lan truyền ngược (Backpropagation)	23
2.7.5	Hàm mất mát MSE (Loss Function)	24
2.7.6	Bộ tối ưu Adam (Adam Optimizer)	25
3	THIẾT KẾ HỆ THỐNG	27
3.1	Mô hình hệ thống	27
3.2	Thiết kế phần cứng	28
3.2.1	Chức năng của phần cứng	28
3.2.2	Sơ đồ khối phần cứng	29
3.2.3	Thiết kế từng khối	33
3.3	Thiết kế phần mềm	48
3.3.1	Chức năng hoạt động của phần mềm	48
3.3.2	Lưu đồ tổng quát hoạt động của hệ thống	48
3.3.3	Lưu đồ thu thập dữ liệu cho mô hình YOLO	50
3.3.4	Lưu đồ huấn luyện mô hình YOLO	53
3.3.5	Lưu đồ thu thập dữ liệu cho mô hình MLP	55

3.3.6	Lưu đồ huấn luyện mô hình MLP	60
3.3.7	Lưu đồ nhận diện vật thể và dự đoán lực kẹp ban đầu	62
3.3.8	Lưu đồ đọc cảm biến xúc giác và điều khiển thích nghi	64
3.3.9	Lưu đồ giải thuật của giao diện	66
4	KẾT QUẢ	68
4.1	KẾT QUẢ MÔ HÌNH THI CÔNG	68
4.1.1	Mô hình tay kẹp robot hoàn chỉnh	68
4.1.2	Kết quả huấn luyện mô hình YOLOv8	71
4.1.3	Kết quả huấn luyện các mô hình học máy	73
4.2	HOẠT ĐỘNG CỦA HỆ THỐNG	75
4.2.1	Quá trình vận hành với vật thể thuộc lớp <i>ball</i>	75
4.2.2	Quá trình vận hành với vật thể thuộc lớp <i>bottle</i>	79
4.2.3	Hoạt động của hệ thống với vật thể lớp <i>can</i>	82
4.2.4	Nhận xét đánh giá	85
5	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	86
5.1	Kết luận	86
5.2	Hướng phát triển	87
	TÀI LIỆU THAM KHẢO	88
	PHỤ LỤC	90

DANH MỤC HÌNH ẢNH

Hình 2.1.	Cấu tạo và nguyên lý hoạt động của cảm biến điện trở áp: (a) cấu tạo cảm biến FSR; (b) sự thay đổi điện trở của cao su dẫn điện khi có lực tác dụng	7
Hình 2.2.	Cấu trúc cảm biến xúc giác điện dung gồm lớp tiếp xúc, lớp điện cực trên, lớp điện môi trung gian và lớp điện cực dưới	8
Hình 2.3.	Khi có áp lực tác động lên vật liệu áp điện, sự biến dạng của cấu trúc tinh thể làm xuất hiện điện tích trên bề mặt vật liệu, từ đó tạo ra điện áp đầu ra tỉ lệ với lực tác dụng	9
Hình 2.4.	Ảnh minh họa đầu ra của bài toán object detection	10
Hình 2.5.	Minh họa cách YOLO nhận diện đối tượng	11
Hình 2.6.	Minh họa Bounding box	12
Hình 2.7.	Minh họa Anchor box.....	12
Hình 2.8.	So sánh hiệu suất của YOLOv8 với các phiên bản tiền nhiệm..	14
Hình 2.9.	Kiến trúc mô hình YOLOv8.....	14
Hình 2.10.	Các thành phần của Machine Learning.....	15
Hình 2.11.	Mối quan hệ giữa AI, Machine Learning và Deep Learning....	18
Hình 2.12.	Cấu trúc nơ-ron và mô hình tính toán của nơ-ron.....	18
Hình 2.13.	Mạng nơ-ron nhân tạo	19
Hình 2.14.	Mô hình mạng Perceptron nhiều lớp.....	21
Hình 2.15.	Mô hình nơ-ron với hàm kích hoạt phi tuyến	22
Hình 3.1.	Mô hình tổng quát của hệ thống	27
Hình 3.2.	Sơ đồ khối của hệ thống	30
Hình 3.3.	Minh họa thiết kế lắp ráp cảm biến xúc giác.....	34

Hình 3.4.	Tấm Velostat.....	34
Hình 3.5.	Module ADC ADS1115	35
Hình 3.6.	Sơ đồ nguyên lý khối cảm biến và module ADS1115	38
Hình 3.7.	Động cơ servo TD8120MG	39
Hình 3.8.	Sơ đồ nguyên lý khối động cơ.....	41
Hình 3.9.	Raspberry Pi 4.....	42
Hình 3.10.	Sơ đồ nguyên lý kết nối Raspberry Pi 4	43
Hình 3.11.	Camera Huddly GO 1.0.....	45
Hình 3.12.	Adapter CTD-050300A.....	47
Hình 3.13.	Lưu đồ tổng quát hoạt động của hệ thống	49
Hình 3.14.	Lưu đồ thu thập dữ liệu cho mô hình YOLO.....	50
Hình 3.15.	Một số hình ảnh minh họa trong quá trình xây dựng tập dữ liệu YOLO	52
Hình 3.16.	Lưu đồ huấn luyện mô hình YOLO	53
Hình 3.17.	Minh họa môi trường huấn luyện và hiệu suất xuất mô hình YOLO	55
Hình 3.18.	Lưu đồ thu thập dữ liệu cho mô hình MLP.....	56
Hình 3.19.	Minh họa quá trình thu thập dữ liệu cho mô hình MLP	57
Hình 3.20.	Lưu đồ huấn luyện mô hình MLP.....	60
Hình 3.21.	Kết quả xuất mô hình MLP sau khi huấn luyện.....	62
Hình 3.22.	Lưu đồ nhận diện vật thể và dự đoán lực kẹp ban đầu.....	63
Hình 3.23.	Lưu đồ đọc cảm biến xúc giác và điều khiển thích nghi	65
Hình 3.24.	Lưu đồ giải thuật của giao diện	66
Hình 3.25.	Giao diện điều khiển và giám sát hoạt động của hệ thống	67
Hình 4.1.	Mô hình tay kẹp robot sau khi hoàn thiện	68
Hình 4.2.	Cụm tay kẹp và cơ cấu truyền động.....	69
Hình 4.3.	Kết cấu cơ khí của ngón tay kẹp	69
Hình 4.4.	Cảm biến xúc giác được gắn trên ngón tay	70

Hình 4.5.	Khởi xử lý trung tâm của hệ thống.....	70
Hình 4.6.	Kết quả huấn luyện mô hình YOLOv8	71
Hình 4.7.	Ma trận nhầm lẫn chuẩn hóa của mô hình YOLOv8	72
Hình 4.8.	Kết quả huấn luyện các mô hình học máy	73
Hình 4.9.	Giao diện nhận diện vật thể và dự đoán tỉ lệ kẹp ban đầu với lốp ball	76
Hình 4.10.	Giao diện hệ thống khi tay kẹp giữ vật thể ball thành công	76
Hình 4.11.	Hình ảnh thực tế tay kẹp giữ vật thể ball	77
Hình 4.12.	Giao diện hệ thống khi phát hiện trạng thái trượt của vật thể ball	78
Hình 4.13.	Giao diện hệ thống sau khi điều chỉnh và giữ lại vật thể ball thành công	78
Hình 4.14.	Giao diện nhận diện vật thể và dự đoán tỉ lệ kẹp ban đầu với lốp bottle	79
Hình 4.15.	Giao diện hệ thống khi tay kẹp giữ vật thể bottle thành công...	80
Hình 4.16.	Hình ảnh thực tế tay kẹp giữ vật thể bottle	80
Hình 4.17.	Giao diện hệ thống khi phát hiện trạng thái trượt của vật thể bottle	81
Hình 4.18.	Giao diện hệ thống sau khi điều chỉnh và giữ lại vật thể bottle thành công	82
Hình 4.19.	Quá trình nhận diện và phát hiện trạng thái quá lực với vật thể lốp can	83
Hình 4.20.	Kết quả kẹp vật thành công sau khi xử lý trạng thái quá lực	84

DANH MỤC BẢNG

Bảng 3.1.	Quy ước các lớp vật thể trong tập dữ liệu YOLO	51
Bảng 3.2.	Số lượng dữ liệu YOLO sau khi chia tập train và val	53
Bảng 3.3.	Các nhóm dữ liệu được thu thập cho mô hình MLP	57
Bảng 3.4.	Ví dụ một mẫu dữ liệu kẹp được ghi nhận	58
Bảng 3.5.	Các nhóm đặc trưng đầu vào dùng để huấn luyện mô hình MLP	61
Bảng 3.6.	Các thông số chính khi huấn luyện mô hình MLP	62
Bảng 4.1.	Kết quả đánh giá mô hình MLPRegressor	73
Bảng 4.2.	Kết quả đánh giá mô hình MLPClassifier	75
Bảng 4.3.	Thông số chính trong quá trình vận hành với vật thể ball	77
Bảng 4.4.	Thông số chính trong quá trình vận hành với vật thể bottle	81
Bảng 4.5.	Một số thông số chính trong quá trình vận hành với vật thể lớp can	84

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Nghĩa đầy đủ
AI	Artificial Intelligence
ML	Machine Learning
DL	Deep Learning
ANN	Artificial Neural Network
CNN	Convolutional Neural Network
MLP	Multilayer Perceptron
YOLO	You Only Look Once
NCNN	Neural Network Compression Framework
ADC	Analog-to-Digital Converter
FSR	Force Sensitive Resistor
GPIO	General Purpose Input/Output
PWM	Pulse Width Modulation
I2C	Inter-Integrated Circuit
USB	Universal Serial Bus
UI	User Interface
MSE	Mean Squared Error
MAE	Mean Absolute Error
RMSE	Root Mean Squared Error
IoU	Intersection over Union
NMS	Non-Maximum Suppression

CHƯƠNG 1

GIỚI THIỆU

1.1 Giới thiệu

Trong những năm gần đây, robot ngày càng được ứng dụng rộng rãi trong công nghiệp, y tế, dịch vụ và đời sống. Trong đó, thao tác cầm nắm vật thể là một trong những nhiệm vụ quan trọng, đóng vai trò nền tảng trong nhiều hệ thống robot. Tuy nhiên, so với bàn tay con người, khả năng cầm nắm của robot vẫn còn nhiều hạn chế, đặc biệt khi phải thao tác với các vật thể có hình dạng, khối lượng, độ cứng và bề mặt khác nhau.

Đối với con người, lực cầm nắm có thể được điều chỉnh linh hoạt nhờ khả năng cảm nhận xúc giác. Khi vật có xu hướng trượt, lực cầm sẽ được tăng lên; ngược lại, khi vật dễ vỡ, lực tác động sẽ được giảm để tránh gây hư hỏng. Trong khi đó, nhiều hệ thống tay kẹp robot hiện nay vẫn chủ yếu điều khiển theo vị trí hoặc theo lực đặt trước, dẫn đến nguy cơ kẹp không chắc, làm rơi vật hoặc tác dụng lực quá lớn lên vật thể.

Vì vậy, cảm biến xúc giác cùng với trí tuệ nhân tạo và các phương pháp học máy đang trở thành hướng nghiên cứu quan trọng trong lĩnh vực robot cầm nắm [1, 2]. Cảm biến xúc giác cho phép robot thu nhận thông tin về lực tiếp xúc, vùng tiếp xúc và sự thay đổi tín hiệu trong quá trình thao tác, từ đó hỗ trợ nhận biết trạng thái tiếp xúc, phát hiện hiện tượng trượt, đánh giá độ ổn định của quá trình kẹp và điều chỉnh lực tác động lên vật thể [1, 3]. Bên cạnh đó, học máy cho phép robot học từ dữ liệu thực nghiệm để đưa ra dự đoán hoặc quyết định phù hợp hơn, thay vì chỉ dựa trên các ngưỡng cố định hoặc mô hình toán học phức tạp.

Xuất phát từ động lực đó, đề tài **“Xây dựng hệ thống tay kẹp robot thích nghi**

tích hợp cảm biến xúc giác và trí tuệ nhân tạo” được thực hiện nhằm nghiên cứu và triển khai một mô hình tay kẹp robot có khả năng kết hợp giữa cảm biến xúc giác, thị giác máy tính và trí tuệ nhân tạo. Đề tài hướng đến việc xây dựng một hệ thống thực nghiệm có thể nhận dạng vật thể, thực hiện thao tác kẹp và điều chỉnh quá trình kẹp dựa trên phản hồi từ cảm biến xúc giác, qua đó góp phần nâng cao tính linh hoạt và độ ổn định của robot trong quá trình tương tác với vật thể.

1.2 Mục tiêu đề tài

Đề tài này được thực hiện nhằm thiết kế và xây dựng một hệ thống tay kẹp robot tự động có khả năng nhận dạng vật thể, cảm nhận lực tiếp xúc và tự điều chỉnh quá trình cầm nắm dựa trên trạng thái thực tế của vật thể. Hệ thống hướng đến việc nâng cao độ ổn định khi kẹp, hạn chế hiện tượng trượt vật và tránh tác dụng lực quá mức lên vật thể. Để đạt được mục tiêu này, đề tài tập trung vào việc thiết kế mô hình tay kẹp robot hai ngón có khả năng đóng mở, tích hợp cảm biến xúc giác để thu nhận dữ liệu lực tiếp xúc trong quá trình thao tác, ứng dụng thị giác máy tính để nhận dạng vật thể và trích xuất các đặc trưng phục vụ điều khiển. Bên cạnh đó, đề tài còn xây dựng và huấn luyện mô hình học máy nhằm dự đoán lực đóng và đánh giá trạng thái kẹp dựa trên dữ liệu xúc giác [2, 4], thiết kế thuật toán điều khiển thích nghi để tay kẹp có thể điều chỉnh theo trạng thái thực tế, đồng thời xây dựng chương trình điều khiển chính có giao diện tổng quan nhằm hỗ trợ theo dõi hoạt động của hệ thống.

1.3 Giới hạn đề tài

Trong khuôn khổ của đồ án, đề tài được thực hiện với một số giới hạn nhất định nhằm đảm bảo tính khả thi về thời gian, thiết bị, dữ liệu và điều kiện triển khai thực tế.

Về phần cứng, hệ thống sử dụng tay kẹp robot hai ngón với cơ cấu đóng mở đơn giản. Do hạn chế về cơ khí và điều kiện chế tạo, tay kẹp chỉ tập trung vào thao tác

kẹp giữ cơ bản, chưa thực hiện các thao tác phức tạp như xoay, dịch chuyển hoặc thay đổi tư thế vật thể.

Về cảm biến xúc giác, cảm biến được tự thi công nên tín hiệu giữa các điểm đo có thể không đồng đều và chịu ảnh hưởng bởi nhiễu, chất lượng tiếp xúc cũng như vật liệu chế tạo. Vì vậy, dữ liệu cảm biến được sử dụng chủ yếu để đánh giá tương đối trạng thái tiếp xúc, chưa nhằm mục đích đo lực chính xác theo đơn vị vật lý.

Về điều kiện vận hành, hệ thống được thử nghiệm trong môi trường tương đối ổn định, với ánh sáng phù hợp và vật thể nằm trong vùng quan sát của camera. Khi vật thể di chuyển quá nhanh, đặt quá gần nhau, bị che khuất hoặc điều kiện ánh sáng thay đổi mạnh, quá trình nhận dạng, xử lý và kẹp vật có thể bị ảnh hưởng.

Về mô hình AI, dữ liệu huấn luyện được giới hạn trong 3 nhóm vật thể với tổng cộng 9 vật. Do đó, khả năng dự đoán và điều chỉnh của hệ thống chủ yếu phản ánh trên các vật thể đã được thử nghiệm, chưa đánh giá toàn diện với nhiều loại vật thể khác nhau về hình dạng, khối lượng, vật liệu và bề mặt.

1.4 Phương pháp nghiên cứu

Phương pháp nghiên cứu của đề tài là kết hợp giữa nghiên cứu lý thuyết, phân tích thiết kế hệ thống, thi công phần cứng, xây dựng phần mềm điều khiển và đánh giá thực nghiệm. Các phương pháp chính được thực hiện gồm:

- Nghiên cứu cơ sở lý thuyết về tay kẹp robot, cảm biến xúc giác, xử lý tín hiệu cảm biến, thị giác máy tính và học máy.
- Phân tích và thiết kế hệ thống tổng thể, bao gồm tay kẹp, cảm biến xúc giác, bộ xử lý trung tâm, phần mềm điều khiển và giao diện giám sát.
- Thi công phần cứng, tích hợp cảm biến xúc giác vào tay kẹp và kết nối các khối chức năng với bộ xử lý.
- Thu thập dữ liệu thực nghiệm từ quá trình kẹp vật để phục vụ huấn luyện và

đánh giá mô hình AI.

- Huấn luyện và ứng dụng mô hình học máy nhằm dự đoán mức đóng ban đầu và phân loại trạng thái kẹp của vật thể.
- Thử nghiệm hệ thống với phần cứng thực tế, đánh giá khả năng nhận dạng, cầm nắm, phản hồi xúc giác, tính thích nghi và độ ổn định của hệ thống.

1.5 Đối tượng và phạm vi nghiên cứu

1.5.1 Đối tượng nghiên cứu

Đối tượng nghiên cứu của đề tài là hệ thống tay kẹp robot thích nghi có tích hợp cảm biến xúc giác và trí tuệ nhân tạo. Các đối tượng chính bao gồm cơ cấu tay kẹp robot, cảm biến xúc giác, hệ thống nhận dạng vật thể, mô hình AI hỗ trợ điều khiển và chương trình giao diện giám sát hệ thống.

1.5.2 Phạm vi nghiên cứu

Trong phạm vi đề tài, hệ thống được nghiên cứu và xây dựng ở mức mô hình thực nghiệm. Đề tài tập trung vào việc thiết kế tay kẹp hai ngón, tích hợp cảm biến xúc giác để thu nhận dữ liệu tiếp xúc, ứng dụng thị giác máy tính để nhận dạng vật thể và sử dụng mô hình AI để hỗ trợ quá trình kẹp thích nghi.

Hệ thống được thử nghiệm với một số vật thể đại diện, chưa tập trung vào các thao tác phức tạp như xoay, dịch chuyển vật trong lòng tay, đo lực tuyệt đối theo đơn vị chuẩn hoặc triển khai trong môi trường công nghiệp thực tế.

1.6 Bố cục quyển báo cáo

Quyển báo cáo được tổ chức thành 5 chương, trình bày lần lượt từ phần giới thiệu tổng quan, cơ sở lý thuyết, thiết kế và xây dựng hệ thống, kết quả thực nghiệm đến

kết luận và hướng phát triển. Nội dung chính của từng chương được trình bày như sau:

- **Chương 1 – Giới thiệu:** Chương này trình bày tổng quan về đề tài, lý do thực hiện, mục tiêu nghiên cứu, phương pháp nghiên cứu, đối tượng và phạm vi nghiên cứu.
- **Chương 2 – Cơ sở lý thuyết:** Chương này trình bày các kiến thức nền tảng phục vụ cho quá trình thiết kế và xây dựng hệ thống. Các nội dung chính bao gồm tổng quan về cảm biến xúc giác, thị giác máy tính, các mô hình nhận diện vật thể, học máy, học sâu, mạng nơ-ron nhân tạo và chủ đề liên quan.
- **Chương 3 – Thiết kế và xây dựng hệ thống:** Chương này trình bày quá trình phân tích, thiết kế và triển khai hệ thống. Nội dung bao gồm sơ đồ khối tổng thể, thiết kế phần cứng tay kẹp, tích hợp cảm biến xúc giác, lựa chọn bộ xử lý trung tâm, xây dựng chương trình đọc cảm biến, nhận diện vật thể, điều khiển tay kẹp, huấn luyện mô hình học máy và thiết kế giao diện tổng quan để giám sát hoạt động của hệ thống.
- **Chương 4 – Kết quả thực nghiệm và đánh giá:** Chương này trình bày quá trình thử nghiệm hệ thống sau khi hoàn thiện. Các kết quả được đánh giá dựa trên khả năng nhận dạng vật thể, khả năng xử lý dữ liệu và mức độ ổn định khi hệ thống vận hành trên phần cứng thực tế.
- **Chương 5 – Kết luận và hướng phát triển:** Chương này tổng kết các kết quả chính đã đạt được so với mục tiêu ban đầu của đề tài. Đồng thời, chương trình bày những hạn chế còn tồn tại và đề xuất các hướng phát triển trong tương lai.

CHƯƠNG 2

CƠ SỞ LÝ THUYẾT

2.1 Cảm biến xúc giác

2.1.1 Khái quát về xúc giác

Xúc giác là khả năng cảm nhận các đặc tính vật lý của vật thể thông qua sự tiếp xúc trực tiếp. Thông qua các thụ thể cảm giác trên da, con người có thể nhận biết các thông tin như lực tác động, nhiệt độ, hình dạng, kích thước và trạng thái bề mặt của vật thể. Các tín hiệu này được truyền đến hệ thần kinh trung ương để xử lý và đưa ra phản ứng phù hợp với môi trường xung quanh.

2.1.2 Công nghệ cảm biến xúc giác

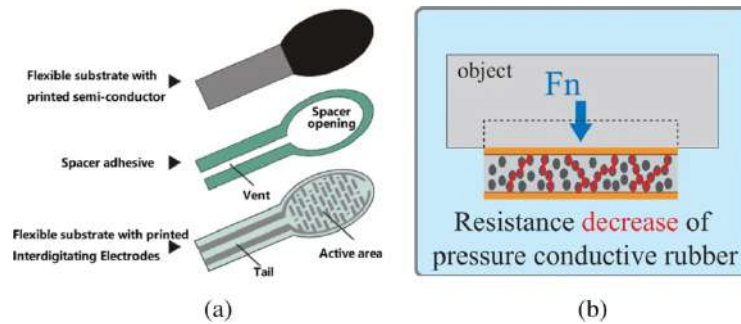
Cảm biến xúc giác là thiết bị dùng để thu nhận thông tin thông qua sự tiếp xúc vật lý với đối tượng, như lực tác động, áp suất, vùng tiếp xúc, độ biến dạng, đặc tính bề mặt hoặc sự thay đổi trạng thái tiếp xúc [1, 3]. Khi được thiết kế dưới dạng mảng, các phần tử cảm nhận tạo thành bản đồ xúc giác, cho phép biểu diễn phân bố tín hiệu trên bề mặt tiếp xúc [5].

Tùy theo yêu cầu ứng dụng, cảm biến xúc giác có thể được thiết kế theo các tiêu chí như độ nhạy, độ phân giải không gian, tốc độ đáp ứng, độ bền cơ học, tính linh hoạt và khả năng tích hợp vào hệ thống [1, 3].

Về nguyên lý hoạt động, cảm biến xúc giác thường dựa trên sự thay đổi điện trở, điện dung, điện tích hoặc đặc tính quang học khi có lực tác dụng; trong đó phổ biến là cảm biến điện trở áp, cảm biến điện dung và cảm biến áp điện.

Cảm biến điện trở áp (Piezoresistive)

Cảm biến điện trở áp hoạt động dựa trên hiệu ứng piezoresistive, trong đó điện trở của vật liệu thay đổi khi chịu tác động cơ học như lực nén hoặc biến dạng [6]. Khi có lực tác dụng lên vùng cảm biến, cấu trúc tiếp xúc bên trong vật liệu thay đổi, làm khả năng dẫn điện biến thiên và tạo ra sự thay đổi điện trở tương ứng với mức lực tác động.



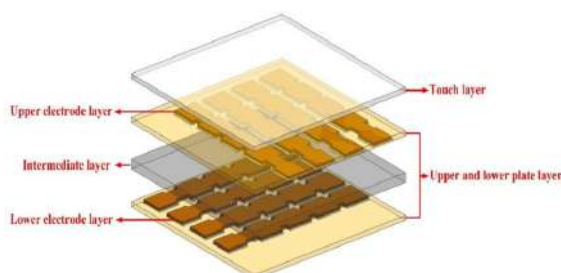
Hình 2.1: Cấu tạo và nguyên lý hoạt động của cảm biến điện trở áp: (a) cấu tạo cảm biến FSR; (b) sự thay đổi điện trở của cao su dẫn điện khi có lực tác dụng

Các cảm biến xúc giác dựa trên nguyên lý điện trở áp có thể được chế tạo dưới nhiều dạng như cảm biến lực dạng điện trở (Force Sensitive Resistor – FSR), cao su dẫn điện, bột dẫn điện hoặc vải dẫn điện nhạy áp. Trong đó, FSR có cấu tạo đơn giản, dễ tích hợp và được sử dụng phổ biến trong các ứng dụng đo lực tiếp xúc. Với vật liệu cao su dẫn điện, khi có áp lực tác dụng, khoảng cách giữa các hạt dẫn điện giảm, số đường dẫn điện tăng lên và điện trở của vật liệu giảm xuống.

Ưu điểm của cảm biến điện trở áp là cấu trúc đơn giản, chi phí thấp, dễ chế tạo và có thể thiết kế ở dạng mỏng, mềm hoặc linh hoạt. Loại cảm biến này phù hợp với các ứng dụng cần đo lực tiếp xúc trên nhiều bề mặt khác nhau và thường ít nhạy với nhiễu điện từ hơn so với cảm biến điện dung. Tuy nhiên, cảm biến điện trở áp cũng có một số hạn chế như quan hệ lực – điện trở không tuyến tính, khó hiệu chuẩn, có hiện tượng trễ và độ lặp lại chưa cao. Ngoài ra, cảm biến còn có thể chịu ảnh hưởng bởi nhiệt độ, độ ẩm và sự mài mòn vật liệu trong quá trình sử dụng.

Cảm biến điện dung

Cảm biến điện dung thường gồm hai bản cực dẫn điện được ngăn cách bởi một lớp điện môi có khả năng biến dạng [1]. Khi có lực tác động, khoảng cách giữa hai bản cực hoặc đặc tính của lớp điện môi thay đổi, dẫn đến sự thay đổi điện dung. Giá trị điện dung này được sử dụng để xác định mức độ tác động lên cảm biến.



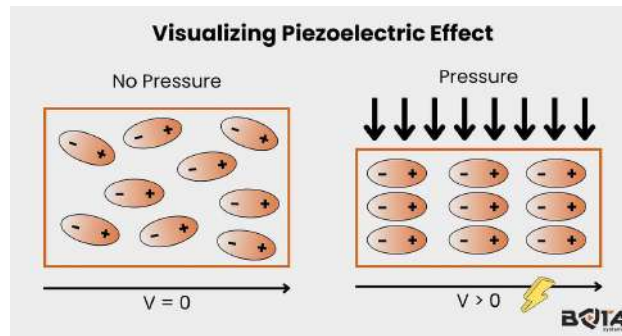
Hình 2.2: Cấu trúc cảm biến xúc giác điện dung gồm lớp tiếp xúc, lớp điện cực trên, lớp điện môi trung gian và lớp điện cực dưới

Cảm biến điện dung có thể được chế tạo dưới dạng mảng bằng cách bố trí các điện cực theo hàng và cột, giữa các điện cực là lớp điện môi đàn hồi. Loại cảm biến này có độ nhạy tương đối cao, đặc biệt đối với các lực tác động nhỏ. Ngoài lực pháp tuyến, một số cấu trúc cảm biến điện dung còn có thể được thiết kế để ước lượng lực tiếp tuyến hoặc lực cắt.

Ưu điểm của cảm biến điện dung là độ nhạy cao, khả năng đáp ứng nhanh và phù hợp với nhiều ứng dụng đo tiếp xúc. Tuy nhiên, cảm biến điện dung thường nhạy với nhiễu điện từ, nhiệt độ và các yếu tố môi trường. Bên cạnh đó, mạch đo và xử lý tín hiệu của loại cảm biến này thường phức tạp hơn so với cảm biến điện trở áp.

Cảm biến áp điện

Cảm biến áp điện hoạt động dựa trên hiệu ứng áp điện, trong đó một số vật liệu có khả năng sinh ra điện tích khi chịu biến dạng cơ học [1]. Các vật liệu áp điện phổ biến gồm thạch anh, gốm áp điện và một số polymer như polyvinylidene fluoride. Khi có lực hoặc áp suất tác dụng, vật liệu áp điện tạo ra tín hiệu điện tương ứng với sự biến dạng của nó.



Hình 2.3: Khi có áp lực tác động lên vật liệu áp điện, sự biến dạng của cấu trúc tinh thể làm xuất hiện điện tích trên bề mặt vật liệu, từ đó tạo ra điện áp đầu ra tỉ lệ với lực tác dụng

Trong các hệ thống cảm biến xúc giác, vật liệu áp điện có thể được chế tạo dưới dạng màng mỏng hoặc tích hợp trong các lớp vật liệu đàn hồi. Đặc điểm nổi bật của cảm biến áp điện là khả năng đáp ứng tốt với các tác động động, chẳng hạn như rung động, va chạm hoặc sự thay đổi nhanh của lực tác dụng.

Hạn chế của cảm biến áp điện là không phù hợp để đo lực tĩnh trong thời gian dài, do tín hiệu tạo ra chủ yếu phản ánh sự thay đổi của lực theo thời gian. Ngoài ra, loại cảm biến này có thể bị ảnh hưởng bởi nhiệt độ và yêu cầu kết nối điện ổn định để đảm bảo độ tin cậy trong quá trình đo.

2.2 Tổng quan về thị giác máy tính (Computer Vision)

2.2.1 Định nghĩa

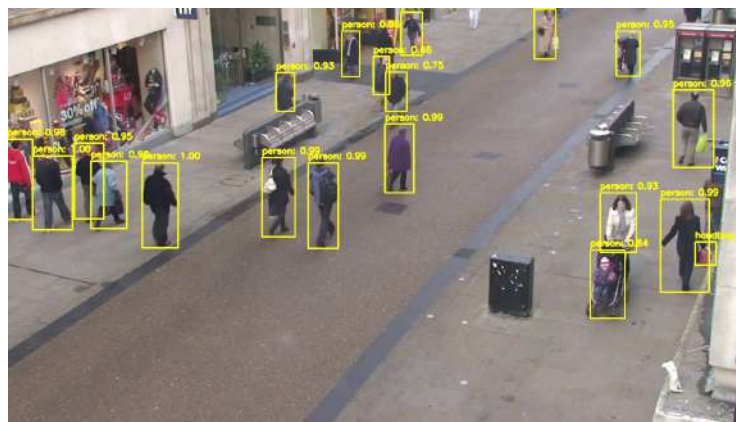
Thị giác máy tính là một lĩnh vực nghiên cứu các phương pháp thu nhận, xử lý, phân tích và nhận dạng dữ liệu hình ảnh nhằm trích xuất thông tin phục vụ cho quá trình ra quyết định. Dữ liệu đầu vào của thị giác máy tính có thể là ảnh tĩnh, chuỗi ảnh, video hoặc dữ liệu hình ảnh thu nhận từ các loại cảm biến khác nhau.

Mục tiêu của thị giác máy tính là xây dựng các mô hình có khả năng mô phỏng một phần quá trình nhận thức thị giác của con người. Thông qua các phương pháp toán học, thống kê, xử lý ảnh và học máy, hệ thống có thể nhận biết đối tượng, phân

tích đặc trưng hình ảnh và hỗ trợ thực hiện các tác vụ tự động trong nhiều lĩnh vực ứng dụng.

2.2.2 Bài toán phát hiện vật thể (Object detection)

Object detection là bài toán phát hiện vị trí và phân loại các đối tượng xuất hiện trong ảnh, có thể có một hoặc nhiều đối tượng trên cùng một ảnh. Với đầu vào là ảnh màu, đầu ra là tọa độ của các đối tượng được phát hiện trên ảnh và tỉ lệ phần trăm chắc chắn của mỗi đối tượng.



Hình 2.4: Ảnh minh họa đầu ra của bài toán object detection

2.3 Mô hình YOLO

2.3.1 Giới thiệu về YOLO

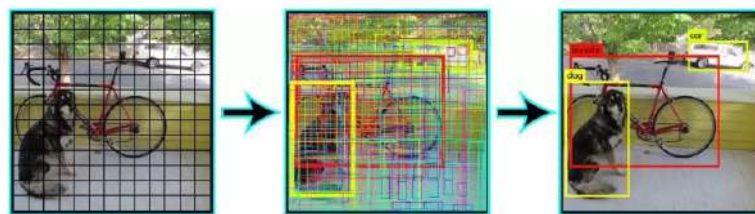
YOLO, viết tắt của You Only Look Once, là một dòng mô hình thị giác máy tính được sử dụng phổ biến trong bài toán phát hiện đối tượng theo thời gian thực [7]. Khác với một số phương pháp phát hiện đối tượng nhiều giai đoạn, YOLO thực hiện dự đoán trực tiếp trên ảnh đầu vào chỉ trong một lần suy luận qua mạng nơ-ron. Kết quả đầu ra của mô hình bao gồm vị trí hộp giới hạn, nhãn lớp và độ tin cậy tương ứng của các đối tượng được phát hiện.

Các mô hình YOLO hiện đại không chỉ hỗ trợ phát hiện đối tượng mà còn có thể

được mở rộng cho nhiều tác vụ khác như phân đoạn ảnh, phân loại ảnh, ước lượng tư thế và theo dõi đối tượng [7]. Nhờ ưu điểm về tốc độ xử lý, độ chính xác và khả năng triển khai linh hoạt trên nhiều nền tảng, YOLO được ứng dụng rộng rãi trong các hệ thống cần xử lý hình ảnh hoặc video theo thời gian thực.

2.3.2 Thuật toán phát hiện vật thể của YOLO

YOLO là thuật toán phát hiện đối tượng một giai đoạn, trong đó quá trình xác định vị trí và phân loại đối tượng được thực hiện trực tiếp trong một lần suy luận qua mạng nơ-ron [7]. Thay vì tách riêng bước đề xuất vùng và bước phân loại, YOLO dự đoán đồng thời hộp giới hạn, nhãn lớp và độ tin cậy của đối tượng từ ảnh đầu vào.



Hình 2.5: Minh họa cách YOLO nhận diện đối tượng

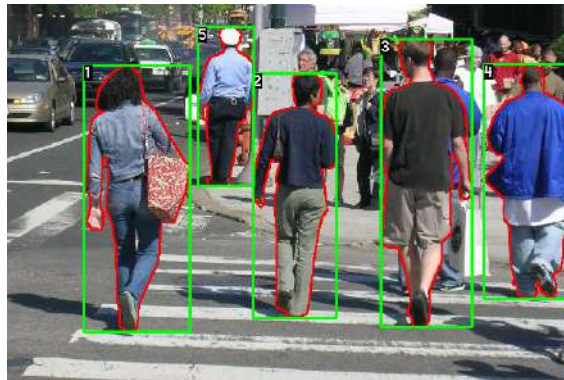
Về nguyên lý, ảnh đầu vào được đưa qua mạng nơ-ron tích chập để trích xuất đặc trưng. Từ các đặc trưng này, mô hình dự đoán các hộp giới hạn bao quanh đối tượng. Mỗi hộp giới hạn thường gồm thông tin về vị trí, kích thước, độ tin cậy và xác suất lớp của đối tượng.

Trong các phiên bản YOLO ban đầu, ảnh đầu vào được chia thành lưới kích thước $S \times S$. Mỗi ô lưới chịu trách nhiệm dự đoán đối tượng nếu tâm của đối tượng nằm trong ô đó. Cách tiếp cận này giúp mô hình phát hiện nhiều đối tượng trong cùng một ảnh với tốc độ xử lý cao.

Hộp giới hạn (Bounding box)

Hộp giới hạn là vùng hình chữ nhật dùng để xác định vị trí của đối tượng trong ảnh. Một hộp giới hạn có thể được biểu diễn bằng tọa độ hai điểm góc (x_{min}, y_{min})

và (x_{max}, y_{max}) , tương ứng với góc trên bên trái và góc dưới bên phải của hộp. Ngoài ra, hộp giới hạn cũng có thể được biểu diễn bằng tọa độ tâm (x, y) , chiều rộng w và chiều cao h .

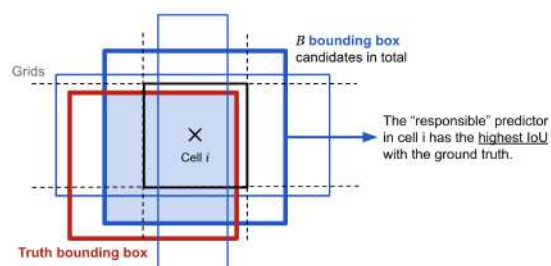


Hình 2.6: Minh họa Bounding box

Trong YOLO, mô hình không chỉ dự đoán lớp của đối tượng mà còn dự đoán vị trí và kích thước của hộp giới hạn bao quanh đối tượng đó. Các tham số của hộp giới hạn được hiệu chỉnh trong quá trình huấn luyện nhằm làm cho hộp dự đoán gần với hộp thực tế nhất có thể.

Hộp neo (Anchor box)

Trong một số phiên bản YOLO dựa trên cơ chế anchor-based, hộp neo là các hộp tham chiếu được định nghĩa trước với nhiều kích thước và tỉ lệ khác nhau. Các hộp này đóng vai trò như khung dự đoán ban đầu, giúp mô hình dễ dàng ước lượng hộp giới hạn phù hợp với đối tượng cần phát hiện.



Hình 2.7: Minh họa Anchor box

Trong quá trình huấn luyện, mỗi đối tượng trong ảnh có thể được so sánh với

nhiều hộp neo khác nhau. Hộp neo có mức độ tương đồng cao nhất với hộp giới hạn thực tế sẽ được chọn để chịu trách nhiệm dự đoán đối tượng đó. Sau đó, mô hình học cách điều chỉnh độ lệch về vị trí và kích thước của hộp neo để tạo ra hộp giới hạn chính xác hơn.

Việc sử dụng hộp neo giúp mô hình phát hiện các đối tượng có kích thước và tỉ lệ khác nhau trong ảnh. Tuy nhiên, hiệu quả của phương pháp này phụ thuộc vào việc lựa chọn số lượng, kích thước và tỉ lệ hộp neo. Các phiên bản YOLO mới như YOLOv8 đã chuyển sang hướng anchor-free, trong đó mô hình không phụ thuộc vào các hộp neo cố định mà dự đoán hộp giới hạn theo cách trực tiếp hơn.

Non-Maximum Suppression

Sau khi mô hình YOLO tạo ra các dự đoán ban đầu, có thể xuất hiện nhiều hộp giới hạn cùng bao quanh một đối tượng. Thuật toán Non-Maximum Suppression được sử dụng để loại bỏ các hộp trùng lặp và giữ lại hộp phù hợp nhất.

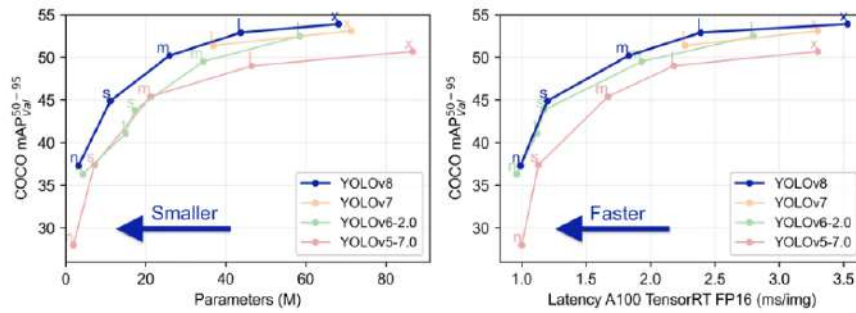
Thuật toán này dựa trên điểm tin cậy của hộp giới hạn và chỉ số Intersection over Union. Điểm tin cậy cho biết khả năng hộp dự đoán chứa đối tượng, trong khi IoU thể hiện mức độ chồng lấn giữa hai hộp giới hạn. Với hai hộp B_1 và B_2 , IoU được xác định theo công thức:

$$IoU = \frac{|B_1 \cap B_2|}{|B_1 \cup B_2|} \quad (2.1)$$

Nguyên tắc của Non-Maximum Suppression là chọn hộp có điểm tin cậy cao nhất, sau đó loại bỏ các hộp khác có IoU lớn hơn một ngưỡng cho trước so với hộp đã chọn. Quá trình này được lặp lại cho đến khi chỉ còn lại các hộp giới hạn đại diện tốt nhất cho các đối tượng trong ảnh.

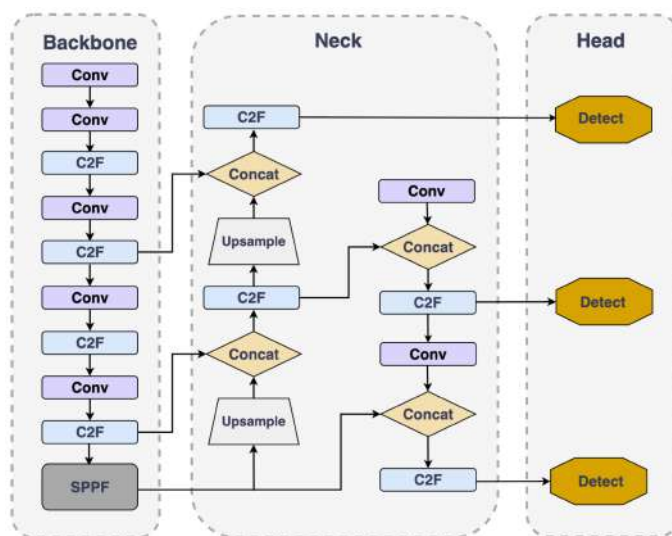
2.3.3 Tổng quan về mô hình YOLOv8

YOLOv8 là một phiên bản thuộc dòng mô hình YOLO do Ultralytics phát triển, được thiết kế cho các bài toán thị giác máy tính theo thời gian thực. Mô hình này hướng đến sự cân bằng giữa tốc độ xử lý và độ chính xác, phù hợp với các ứng dụng cần phát hiện đối tượng nhanh.



Hình 2.8: So sánh hiệu suất của YOLOv8 với các phiên bản tiền nhiệm

So với các phiên bản trước, YOLOv8 có nhiều cải tiến về kiến trúc và thuật toán [7]. Mô hình sử dụng backbone và neck được tối ưu để trích xuất, tổng hợp đặc trưng hiệu quả hơn, đồng thời áp dụng đầu phát hiện dạng anchor-free. Cách tiếp cận này giúp giảm sự phụ thuộc vào các hộp neo định nghĩa trước, đơn giản hóa quá trình dự đoán hộp giới hạn và cải thiện hiệu quả suy luận.



Hình 2.9: Kiến trúc mô hình YOLOv8

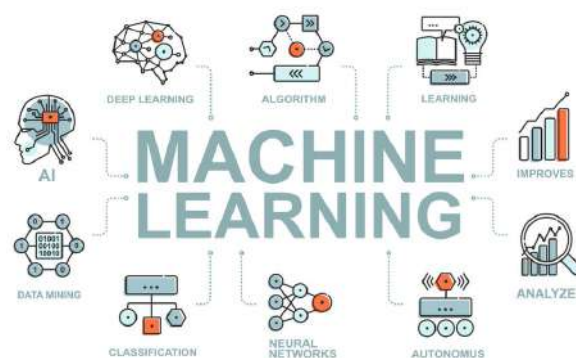
YOLOv8 hỗ trợ nhiều tác vụ như phát hiện đối tượng, phân đoạn ảnh, phân loại ảnh và ước lượng tư thế. Đối với bài toán phát hiện đối tượng, mô hình trả về các thông tin gồm hộp giới hạn, nhãn lớp và độ tin cậy. Dòng YOLOv8 gồm nhiều biến thể như YOLOv8n, YOLOv8s, YOLOv8m, YOLOv8l và YOLOv8x. Trong đó, YOLOv8n là phiên bản nhỏ nhất, có tốc độ xử lý nhanh và phù hợp với các hệ thống bị giới hạn về tài nguyên tính toán.

Về tổng thể, kiến trúc YOLOv8 gồm ba thành phần chính: backbone, neck và head. Backbone có nhiệm vụ trích xuất đặc trưng từ ảnh đầu vào; neck kết hợp các đặc trưng ở nhiều tầng khác nhau để hỗ trợ phát hiện đối tượng ở nhiều kích thước; còn head thực hiện dự đoán cuối cùng, bao gồm vị trí hộp giới hạn, nhãn lớp và độ tin cậy của đối tượng.

2.4 Tổng quan về học máy (Machine Learning)

2.4.1 Khái niệm

Học máy là một lĩnh vực quan trọng của trí tuệ nhân tạo, tập trung vào việc xây dựng các mô hình có khả năng học từ dữ liệu để thực hiện dự đoán, phân loại hoặc đưa ra quyết định [8]. Thay vì lập trình trực tiếp từng quy tắc xử lý, hệ thống học máy sử dụng dữ liệu huấn luyện để nhận biết các mẫu, xu hướng và mối quan hệ trong dữ liệu.



Hình 2.10: Các thành phần của Machine Learning

Trong quá trình huấn luyện, mô hình học máy điều chỉnh các tham số bên trong nhằm cải thiện khả năng dự đoán trên dữ liệu mới [8]. Hiệu quả của mô hình phụ thuộc vào chất lượng dữ liệu, thuật toán được lựa chọn và phương pháp huấn luyện. Vì vậy, dữ liệu đầu vào cần được xử lý phù hợp để hạn chế nhiễu, sai lệch và các giá trị không hợp lệ.

Ngày nay, học máy được ứng dụng rộng rãi trong nhiều lĩnh vực như y tế, giao thông, tài chính, sản xuất và tự động hóa. Nhờ khả năng học từ dữ liệu và thích nghi với nhiều tình huống khác nhau, học máy đã trở thành một nền tảng quan trọng trong các hệ thống thông minh hiện đại.

2.4.2 Phân loại học máy

Học có giám sát (Supervised Learning)

Học có giám sát là phương pháp học máy trong đó mô hình được huấn luyện trên tập dữ liệu đã được gán nhãn [8]. Mỗi mẫu dữ liệu gồm đầu vào và đầu ra tương ứng, từ đó mô hình học mối quan hệ giữa chúng để dự đoán kết quả cho dữ liệu mới.

Phương pháp này thường được dùng cho hai nhóm bài toán chính là phân loại và hồi quy. Với bài toán phân loại, đầu ra là một nhãn lớp; còn với bài toán hồi quy, đầu ra là một giá trị số liên tục. Ưu điểm của học có giám sát là kết quả dễ đánh giá khi có dữ liệu nhãn đầy đủ, tuy nhiên chất lượng mô hình phụ thuộc nhiều vào độ chính xác và tính đại diện của tập dữ liệu huấn luyện.

Học không giám sát (Unsupervised Learning)

Học không giám sát là phương pháp học máy sử dụng dữ liệu huấn luyện không có nhãn đầu ra. Thay vì học mối quan hệ giữa đầu vào và đầu ra, mô hình tập trung tìm kiếm cấu trúc, quy luật hoặc sự tương đồng tiềm ẩn trong dữ liệu.

Phương pháp này thường được dùng trong các bài toán như phân cụm dữ liệu và giảm chiều dữ liệu. Học không giám sát phù hợp với các trường hợp dữ liệu chưa

được gán nhãn hoặc khó xác định trước kết quả đầu ra. Tuy nhiên, do không có nhãn tham chiếu, việc đánh giá chất lượng mô hình thường khó hơn so với học có giám sát.

Học tăng cường (Reinforcement Learning)

Học tăng cường là phương pháp học máy trong đó một tác nhân học cách đưa ra hành động thông qua quá trình tương tác với môi trường. Sau mỗi hành động, tác nhân nhận được phần thưởng hoặc hình phạt, từ đó điều chỉnh chiến lược nhằm tối đa hóa tổng phần thưởng trong dài hạn.

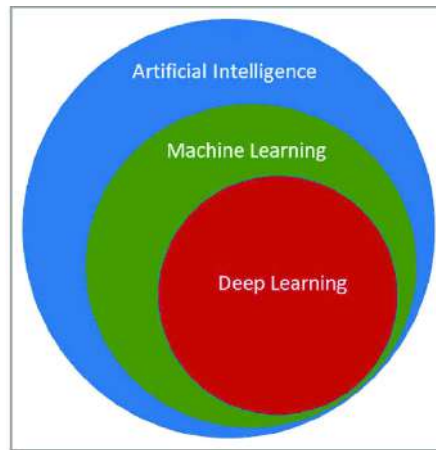
Khác với học có giám sát, học tăng cường không cần dữ liệu gồm các cặp đầu vào và đầu ra mong muốn, mà học thông qua thử nghiệm và phản hồi từ môi trường. Phương pháp này phù hợp với các bài toán ra quyết định tuần tự như trò chơi, điều khiển robot và tối ưu hóa hệ thống, nhưng thường cần nhiều lần tương tác để đạt được kết quả tốt.

2.5 Tổng quan về học sâu (Deep Learning)

Học sâu là một nhánh của học máy, sử dụng các mô hình có nhiều tầng xử lý để học và biểu diễn dữ liệu ở các mức độ trừu tượng khác nhau [9]. Khác với các phương pháp phụ thuộc nhiều vào đặc trưng được thiết kế thủ công, mô hình học sâu có khả năng tự động trích xuất đặc trưng từ dữ liệu thông qua quá trình huấn luyện.

Nền tảng của học sâu là các mạng nơ-ron nhân tạo nhiều tầng [8, 9]. Dữ liệu được truyền từ lớp đầu vào qua các lớp ẩn đến lớp đầu ra. Các lớp đầu thường học những đặc trưng đơn giản, trong khi các lớp sâu hơn kết hợp các đặc trưng này để biểu diễn những thông tin phức tạp hơn.

Học sâu có mối quan hệ chặt chẽ với trí tuệ nhân tạo và học máy. Trí tuệ nhân tạo (Artificial Intelligence – AI) là lĩnh vực rộng, hướng đến việc xây dựng các hệ



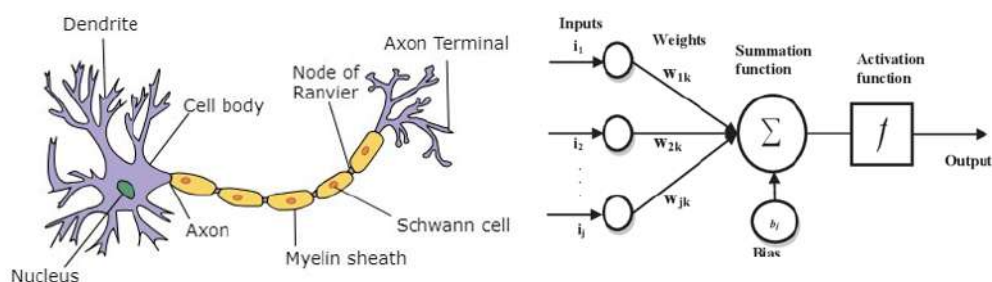
Hình 2.11: Mối quan hệ giữa AI, Machine Learning và Deep Learning

thông có khả năng thực hiện các tác vụ thông minh. Học máy (Machine Learning) là một nhánh của AI, tập trung vào việc xây dựng mô hình có khả năng học từ dữ liệu. Trong khi đó, học sâu là một hướng tiếp cận chuyên sâu hơn của học máy, sử dụng mạng nơ-ron nhiều tầng để tự động học và biểu diễn dữ liệu.

2.6 Mạng nơ-ron nhân tạo (Artificial Neural Network)

2.6.1 Nơ-ron nhân tạo

Nơ-ron nhân tạo là đơn vị tính toán cơ bản trong mạng nơ-ron nhân tạo [9]. Mô hình này được xây dựng dựa trên ý tưởng mô phỏng đơn giản hoạt động của nơ-ron sinh học, trong đó các tín hiệu đầu vào được tiếp nhận, xử lý và tạo ra tín hiệu đầu ra. Trong mô hình toán học, mỗi đầu vào được gán với một trọng số tương ứng nhằm biểu diễn mức độ ảnh hưởng của đầu vào đó đến kết quả của nơ-ron.



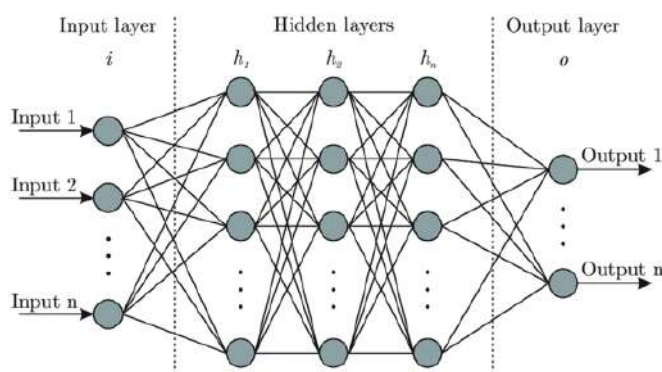
Hình 2.12: Cấu trúc nơ-ron và mô hình tính toán của nơ-ron

Cấu trúc tính toán của một nơ-ron nhân tạo gồm ba bước chính: nhân các tín hiệu đầu vào với trọng số, cộng tổng các giá trị thu được với hệ số lệch, sau đó đưa kết quả qua hàm kích hoạt để tạo đầu ra. Mô hình tổng quát của một nơ-ron nhân tạo được minh họa trong Hình 2.12.

2.6.2 Mạng nơ-ron nhân tạo (Artificial Neural Network - ANN)

Mạng nơ-ron nhân tạo (Artificial Neural Network - ANN) được hình thành bằng cách liên kết nhiều nơ-ron nhân tạo theo một cấu trúc xác định [8, 9]. Tùy theo cách tổ chức và hướng truyền tín hiệu giữa các nơ-ron, mạng có thể có nhiều dạng khác nhau, phù hợp với từng loại bài toán xử lý dữ liệu.

Một trong những cấu trúc cơ bản và phổ biến là Mạng lan truyền tiến (Feedforward neural network). Trong cấu trúc này, tín hiệu được truyền theo một chiều từ lớp đầu vào, qua các lớp ẩn và đến lớp đầu ra, không tồn tại vòng lặp hay đường truyền ngược trong quá trình suy luận. Cấu trúc tổng quát của mạng nơ-ron nhân tạo được minh họa trong Hình 2.13.



Hình 2.13: Mạng nơ-ron nhân tạo

Một mạng nơ-ron nhân tạo thường bao gồm ba thành phần chính. Lớp đầu vào có nhiệm vụ tiếp nhận dữ liệu ban đầu. Các lớp ẩn thực hiện quá trình xử lý, biến đổi và trích xuất đặc trưng từ dữ liệu. Lớp đầu ra tạo ra kết quả cuối cùng của mạng, chẳng hạn như giá trị dự đoán hoặc nhãn phân loại. Số lượng lớp ẩn và số nơ-ron trong mỗi lớp phụ thuộc vào độ phức tạp của dữ liệu cũng như yêu cầu cụ thể của

bài toán.

Trong mạng lan truyền tiến, mỗi nơ-ron ở một lớp thường được kết nối với các nơ-ron ở lớp kế tiếp thông qua các trọng số. Các trọng số này quyết định mức độ ảnh hưởng của tín hiệu đầu vào đến đầu ra của nơ-ron và được điều chỉnh trong quá trình huấn luyện. Quá trình tính toán được thực hiện tuần tự từ lớp này sang lớp khác cho đến khi thu được kết quả ở lớp đầu ra.

2.7 Mô hình Multilayer Perceptron (MLP)

2.7.1 Khái niệm

Mạng Perceptron nhiều lớp, hay Multilayer Perceptron (MLP), là một dạng mạng nơ-ron nhân tạo thuộc nhóm mạng lan truyền tiến [8, 9]. Trong mô hình này, tín hiệu được truyền theo một chiều từ đầu vào đến đầu ra, không tồn tại vòng lặp phản hồi trong quá trình suy luận. MLP được phát triển từ mô hình Perceptron cơ bản, nhưng có khả năng biểu diễn các quan hệ phi tuyến phức tạp hơn nhờ sử dụng các lớp xử lý trung gian và các hàm kích hoạt phi tuyến [9]. Một vài mô hình MLP có thể kể đến như MLP đơn lớp, MLPClassifier (Phân loại dữ liệu), MLPRegressor (Dự đoán giá trị liên tục),...

Đặc điểm quan trọng của MLP là khả năng học mối quan hệ giữa dữ liệu đầu vào và đầu ra thông qua quá trình huấn luyện. Trong quá trình này, các trọng số của mạng được điều chỉnh nhằm giảm sai lệch giữa kết quả dự đoán và giá trị mong muốn. Nhờ đó, MLP có thể được ứng dụng trong nhiều bài toán khác nhau như phân loại, hồi quy, nhận dạng mẫu và dự đoán dữ liệu.

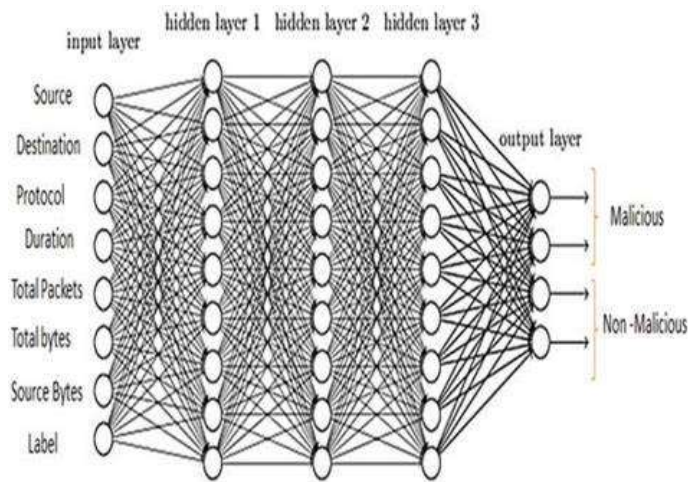
2.7.2 Cấu trúc

Một mô hình MLP cơ bản thường gồm ba thành phần chính: lớp đầu vào, các lớp ẩn và lớp đầu ra [8, 9]. Lớp đầu vào nhận dữ liệu ban đầu của bài toán. Các lớp ẩn

thực hiện quá trình biến đổi và trích xuất đặc trưng thông qua các trọng số, hệ số lệch và hàm kích hoạt. Lớp đầu ra tạo ra kết quả cuối cùng của mô hình, có thể là giá trị dự đoán hoặc nhãn phân loại tùy theo bài toán.

Trong MLP, các nơ-ron giữa hai lớp liên tiếp thường được kết nối đầy đủ với nhau. Điều này có nghĩa là mỗi nơ-ron ở một lớp sẽ nhận tín hiệu từ tất cả các nơ-ron của lớp trước đó và truyền kết quả đến các nơ-ron ở lớp tiếp theo. Số lượng lớp ẩn và số nơ-ron trong từng lớp là các tham số cấu trúc quan trọng, ảnh hưởng đến khả năng biểu diễn của mô hình.

Cấu trúc tổng quát của một mạng MLP được minh họa trong Hình 2.14.



Hình 2.14: Mô hình mạng Perceptron nhiều lớp

Giả sử một MLP gồm lớp đầu vào có n đặc trưng, một lớp ẩn có p nơ-ron và một lớp đầu ra có m nơ-ron. Đầu ra của nơ-ron thứ j trong lớp ẩn được xác định như sau:

$$h_j = f \left(\sum_{i=1}^n w_{ij} x_i + b_j \right), \quad j = 1, 2, \dots, p \quad (2.2)$$

Trong đó, x_i là tín hiệu đầu vào, w_{ij} là trọng số kết nối từ đầu vào thứ i đến nơ-ron ẩn thứ j , b_j là hệ số lệch và $f(\cdot)$ là hàm kích hoạt.

Dưới dạng ma trận, quá trình tính toán tại một lớp có thể được biểu diễn như sau:

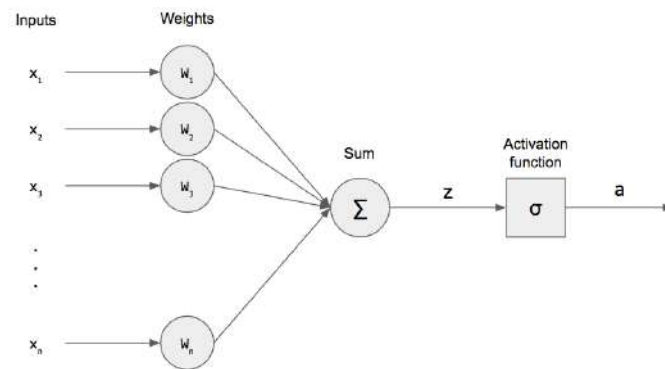
$$\mathbf{h} = f(W^T \mathbf{x} + \mathbf{b}) \quad (2.3)$$

Trong đó, $\mathbf{x}$ là vector đầu vào, W là ma trận trọng số, $\mathbf{b}$ là vector hệ số lệch và $\mathbf{h}$ là vector đầu ra của lớp. Quá trình này được lặp lại qua các lớp tiếp theo cho đến khi thu được kết quả tại lớp đầu ra.

2.7.3 Hàm kích hoạt

Trong mạng nơ-ron nhân tạo, nếu mỗi nơ-ron chỉ thực hiện tổ hợp tuyến tính của các đầu vào thì toàn bộ mạng, dù có nhiều lớp, vẫn tương đương với một phép biến đổi tuyến tính. Điều này làm hạn chế khả năng mô hình hóa các quan hệ phức tạp trong dữ liệu. Vì vậy, hàm kích hoạt được sử dụng sau phép tổng có trọng số để tạo ra tính phi tuyến cho nơ-ron.

Mô hình nơ-ron với hàm kích hoạt phi tuyến được minh họa trong Hình 2.15.



Hình 2.15: Mô hình nơ-ron với hàm kích hoạt phi tuyến

Hàm kích hoạt quyết định cách nơ-ron biến đổi tổng tín hiệu đầu vào thành đầu ra. Một số hàm kích hoạt phổ biến gồm sigmoid, tanh và ReLU. Trong đó, sigmoid thường được sử dụng trong các mô hình nơ-ron cơ bản, còn ReLU được sử dụng rộng rãi trong các mạng nơ-ron hiện đại do có ưu điểm về tốc độ tính toán và khả năng hỗ trợ quá trình huấn luyện.

Hàm sigmoid, hay còn gọi là hàm logistic, cho đầu ra nằm trong khoảng $(0, 1)$ và

được xác định bởi công thức:

$$f(x) = \frac{1}{1 + e^{-x}} \quad (2.4)$$

Hàm sigmoid có ưu điểm là đầu ra bị chặn trong một khoảng xác định, phù hợp với một số bài toán cần biểu diễn xác suất. Tuy nhiên, khi giá trị đầu vào quá lớn hoặc quá nhỏ, đạo hàm của sigmoid tiến gần về 0. Điều này có thể gây ra hiện tượng suy giảm gradient, làm cho quá trình cập nhật trọng số ở các lớp trước diễn ra chậm hoặc kém hiệu quả.

Hàm ReLU, viết tắt của Rectified Linear Unit, được định nghĩa như sau:

$$f(x) = \max(0, x) \quad (2.5)$$

Theo định nghĩa trên, nếu đầu vào x nhỏ hơn hoặc bằng 0 thì đầu ra của ReLU bằng 0; nếu x lớn hơn 0 thì đầu ra bằng chính giá trị x .

Một ưu điểm khác của ReLU là tạo ra biểu diễn thưa, vì các nơ-ron có đầu vào âm sẽ cho đầu ra bằng 0. Điều này giúp giảm số lượng nơ-ron hoạt động tại một thời điểm và có thể làm cho quá trình học trở nên hiệu quả hơn. Ngoài ra, ReLU không yêu cầu tính toán hàm mũ như sigmoid, do đó phù hợp với các mô hình cần tốc độ xử lý nhanh.

2.7.4 Thuật toán lan truyền ngược (Backpropagation)

Thuật toán lan truyền ngược (Backpropagation) là phương pháp huấn luyện phổ biến cho mạng nơ-ron nhiều lớp [10]. Thuật toán này giúp tính toán ảnh hưởng của các trọng số đến sai số đầu ra, từ đó cập nhật tham số nhằm giảm giá trị hàm mất mát.

Quá trình huấn luyện gồm hai bước chính: lan truyền tiến và lan truyền ngược. Ở bước lan truyền tiến, dữ liệu đầu vào được đưa qua các lớp để tạo ra kết quả dự đoán

và tính sai số. Ở bước lan truyền ngược, sai số được truyền từ lớp đầu ra về các lớp trước để tính gradient và cập nhật trọng số.

Sai số cục bộ tại nơ-ron k của lớp đầu ra được xác định bởi:

$$\delta_k = (d_k - y_k)f'(z_k) \quad (2.6)$$

Đối với nơ-ron j thuộc lớp ẩn, sai số cục bộ được tính theo:

$$\delta_j = f'(z_j) \sum_k w_{jk} \delta_k \quad (2.7)$$

Sau khi xác định được sai số cục bộ, trọng số và hệ số lệch được cập nhật theo:

$$w_{ij}^{new} = w_{ij}^{old} + \eta \delta_j x_i \quad (2.8)$$

$$b_j^{new} = b_j^{old} + \eta \delta_j \quad (2.9)$$

Trong đó, d_k là giá trị mong muốn, y_k là giá trị dự đoán, z_k là tổng có trọng số trước hàm kích hoạt, $f'(z)$ là đạo hàm của hàm kích hoạt và η là hệ số học.

2.7.5 Hàm mất mát MSE (Loss Function)

Hàm mất mát là hàm dùng để đánh giá mức độ sai lệch giữa đầu ra dự đoán của mô hình và giá trị mong muốn. Trong quá trình huấn luyện mạng nơ-ron, các trọng số được điều chỉnh nhằm tối thiểu hóa giá trị của hàm mất mát, từ đó cải thiện khả năng dự đoán của mô hình.

Sai số bình phương trung bình, hay Mean Squared Error (MSE), là một hàm mất mát phổ biến trong các bài toán hồi quy [8]. Với m mẫu dữ liệu huấn luyện, trong đó y_i là giá trị dự đoán của mô hình và t_i là giá trị mong muốn tương ứng, MSE

được xác định như sau:

$$\text{MSE} = \frac{1}{m} \sum_{i=1}^m (t_i - y_i)^2 \quad (2.10)$$

Giá trị MSE càng nhỏ cho thấy kết quả dự đoán của mô hình càng gần với giá trị thực tế. Do đó, trong quá trình huấn luyện, mô hình sẽ cố gắng giảm giá trị MSE xuống mức thấp nhất có thể.

2.7.6 Bộ tối ưu Adam (Adam Optimizer)

Adam (Adaptive Moment Estimation) là một thuật toán tối ưu hóa được sử dụng rộng rãi trong huấn luyện mạng nơ-ron sâu nhờ sự kết hợp giữa hai kỹ thuật: Momentum và Adaptive Learning Rate [11]. Adam tính toán các trung bình động (moving average) của gradient (momentum) và bình phương gradient (adaptive learning rate) để điều chỉnh tốc độ học theo từng tham số.

Các phương trình cập nhật của Adam được mô tả như sau:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (\text{momentum}) \quad (2.11)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (\text{adaptive term}) \quad (2.12)$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \quad (\text{bias correction for } m_t) \quad (2.13)$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (\text{bias correction for } v_t) \quad (2.14)$$

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \quad (2.15)$$

Trong đó:

- g_t là gradient tại bước t ,
- m_t và v_t lần lượt là trung bình động bậc nhất và bậc hai,
- β_1, β_2 là các hệ số giảm trung bình động thường được chọn lần lượt là 0.9 và 0.999,

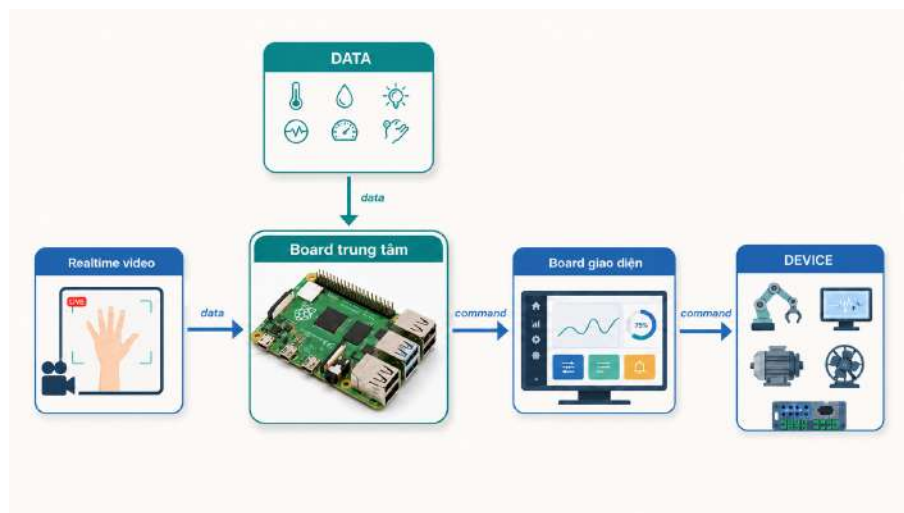
- η là tốc độ học (learning rate),
- ϵ là một hằng số nhỏ (ví dụ 10^{-8}) để tránh chia cho 0.

CHƯƠNG 3

THIẾT KẾ HỆ THỐNG

3.1 Mô hình hệ thống

Dựa trên mục tiêu xây dựng hệ thống tay kẹp robot thích nghi, mô hình tổng quát của hệ thống được đề xuất nhằm thể hiện các thành phần chính và luồng dữ liệu cơ bản giữa các thành phần. Mô hình này giúp hình dung sơ bộ cách hệ thống thu nhận thông tin từ môi trường, xử lý dữ liệu, đưa ra lệnh điều khiển và hỗ trợ người dùng theo dõi quá trình hoạt động.



Hình 3.1: Mô hình tổng quát của hệ thống

Mô hình tổng quát của hệ thống được trình bày như Hình 3.1. Trong đó:

- **Board realtime video:** thu thập hình ảnh đầu vào của vật thể theo thời gian thực. Dữ liệu hình ảnh sau khi thu nhận sẽ được cung cấp cho board trung tâm để phục vụ quá trình xử lý và nhận biết vật thể cần kẹp.
- **Board data:** thu thập dữ liệu từ các cảm biến được bố trí trên hệ thống tay kẹp. Các dữ liệu này phản ánh trạng thái tiếp xúc trong quá trình kẹp và được gửi về

board trung tâm để làm cơ sở cho việc đánh giá trạng thái kẹp vật.

- **Board trung tâm:** là nơi xử lý dữ liệu và thực hiện các tính toán chính của hệ thống. Board trung tâm nhận dữ liệu từ board realtime video và board data, sau đó xử lý thông tin, tính toán lực kẹp phù hợp và gửi lệnh điều khiển đến các thành phần liên quan.
- **Board giao diện:** cung cấp giao diện trực quan giúp người dùng theo dõi trạng thái hoạt động của hệ thống. Thông qua board giao diện, người dùng có thể quan sát thông tin trong quá trình kẹp, theo dõi dữ liệu phản hồi và thực hiện một số thao tác điều khiển cần thiết.
- **Board device:** là thành phần đầu cuối của hệ thống, có nhiệm vụ nhận lệnh từ board trung tâm và board giao diện. Từ các lệnh nhận được, board device điều khiển các thiết bị chấp hành như động cơ và cơ cấu tay kẹp để thực hiện thao tác đóng, mở hoặc giữ vật.

3.2 Thiết kế phần cứng

3.2.1 Chức năng của phần cứng

Dựa vào mục tiêu của đề tài, phần cứng của hệ thống được thiết kế nhằm phục vụ quá trình nhận diện vật thể, thu nhận tín hiệu tiếp xúc và điều khiển cơ cấu tay kẹp hai ngón trong quá trình thao tác. Các chức năng chính của phần cứng trong hệ thống gồm:

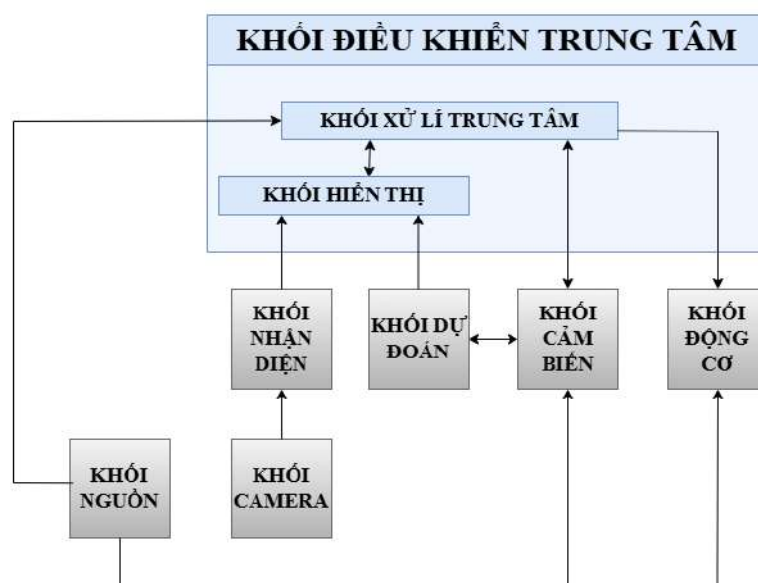
- **Thu nhận hình ảnh:** thu thập hình ảnh của vật thể trong vùng làm việc theo thời gian thực. Dữ liệu hình ảnh được cung cấp cho bộ phận xử lý trung tâm để phục vụ quá trình nhận biết vật thể cần kẹp.
- **Thu nhận tín hiệu xúc giác:** thu thập tín hiệu từ các cảm biến được bố trí trên tay kẹp. Dữ liệu thu được phản ánh trạng thái tiếp xúc giữa tay kẹp và vật thể, giúp hệ thống có cơ sở đánh giá quá trình kẹp vật.

- **Chức năng xử lý trung tâm:** tiếp nhận dữ liệu từ phần thu nhận hình ảnh và phần thu nhận tín hiệu xúc giác, đồng thời phát tín hiệu điều khiển đến các bộ phận chấp hành. Đây là thành phần phần cứng đóng vai trò kết nối và điều phối hoạt động giữa các phần trong hệ thống.
- **Điều khiển và chấp hành của tay kẹp:** nhận tín hiệu điều khiển từ bộ phận xử lý trung tâm để điều khiển cơ cấu truyền động của tay kẹp. Chức năng này giúp tay kẹp hai ngón thực hiện các thao tác đóng, mở, tiếp xúc và giữ vật trong quá trình làm việc.
- **Chức năng cấp nguồn:** cung cấp năng lượng cho các thành phần phần cứng trong hệ thống, bao gồm bộ phận xử lý, phần thu nhận dữ liệu, phần điều khiển và cơ cấu chấp hành, đảm bảo hệ thống hoạt động ổn định trong quá trình vận hành.

Với các chức năng trên, phần cứng của hệ thống được định hướng thiết kế nhằm đáp ứng yêu cầu xây dựng tay kẹp robot hai ngón có khả năng thu nhận thông tin vật thể, cảm nhận trạng thái tiếp xúc và thực hiện thao tác kẹp vật ổn định.

3.2.2 Sơ đồ khối phần cứng

Dựa trên các chức năng đã xác định, sơ đồ khối của hệ thống được xây dựng nhằm thể hiện mối liên hệ giữa các thành phần chính trong hệ thống tay kẹp robot thích nghi.



Hình 3.2: Sơ đồ khối của hệ thống

Nhiệm vụ các khối trong khối điều khiển trung tâm:

- **Khối xử lý trung tâm:** là thành phần xử lý chính của hệ thống. Khối này tiếp nhận dữ liệu từ khối nhận diện, khối dự đoán và khối cảm biến, sau đó thực hiện xử lý, tính toán lực kẹp và đưa ra tín hiệu điều khiển phù hợp cho khối động cơ. Ngoài ra, khối xử lý trung tâm còn truyền dữ liệu và trao đổi thông tin với khối hiển thị.
- **Khối hiển thị:** có nhiệm vụ truyền nhận dữ liệu giữa người dùng với hệ thống, đồng thời hiển thị các thông tin phản hồi từ khối xử lý trung tâm. Khối này đóng vai trò trung gian giúp người dùng theo dõi trạng thái hoạt động, quan sát dữ liệu cảm biến, kết quả nhận diện, trạng thái tay kẹp và gửi các thao tác điều khiển cần thiết trong quá trình vận hành.

Chức năng các khối khác trong hệ thống:

- **Khối camera:** có nhiệm vụ thu thập dữ liệu hình ảnh của vật thể trong vùng làm việc và cung cấp dữ liệu đầu vào cho khối nhận diện.
- **Khối nhận diện:** tiếp nhận dữ liệu hình ảnh từ khối camera để nhận biết vật

thể mà tay kẹp cần thao tác. Kết quả nhận diện được truyền đến các thành phần liên quan để phục vụ cho quá trình xử lý tiếp theo.

- **Khối dự đoán:** có nhiệm vụ dự đoán giá trị lực kẹp phù hợp dựa trên các dữ liệu đầu vào của hệ thống. Kết quả dự đoán được sử dụng để hỗ trợ khối xử lý trung tâm trong quá trình điều khiển tay kẹp.
- **Khối cảm biến:** thu nhận tín hiệu tiếp xúc giữa ma trận cảm biến và vật thể trong quá trình thao tác. Dữ liệu từ khối cảm biến được gửi về khối xử lý trung tâm.
- **Khối động cơ:** nhận tín hiệu điều khiển từ khối xử lý trung tâm để điều khiển cơ cấu tay kẹp. Thông qua khối động cơ, tay kẹp có thể thực hiện các thao tác đóng, mở và giữ vật trong quá trình làm việc.
- **Khối nguồn:** có chức năng cấp nguồn cho khối điều khiển trung tâm, khối cảm biến và khối động cơ, giúp hệ thống hoạt động ổn định trong quá trình vận hành.

Nguyên lý hoạt động của hệ thống

Nguyên lý hoạt động của hệ thống được xây dựng dựa trên sự phối hợp giữa dữ liệu hình ảnh, dữ liệu cảm biến xúc giác và cơ cấu chấp hành của tay kẹp. Quá trình hoạt động tổng quát của hệ thống được mô tả qua các bước sau:

Bước 1: Thu nhận hình ảnh vật thể

Camera tiến hành thu nhận hình ảnh vật thể trong vùng làm việc theo thời gian thực. Dữ liệu hình ảnh sau đó được đưa đến khối nhận diện để xác định vật thể cần kẹp và trích xuất các thông tin cần thiết phục vụ cho quá trình xử lý tiếp theo.

Bước 2: Nhận diện vật thể

Khối nhận diện tiếp nhận dữ liệu hình ảnh từ camera và sử dụng mô hình YOLO để nhận dạng vật thể xuất hiện trong vùng thao tác. Kết quả nhận diện được truyền

đến khối xử lý trung tâm và khối hiển thị, giúp hệ thống xác định đối tượng cần kẹp và hỗ trợ người dùng theo dõi trạng thái nhận diện.

Bước 3: Dự đoán lực kẹp ban đầu

Dựa trên thông tin vật thể đã được nhận diện và các đặc trưng thu được từ quá trình xử lý hình ảnh, mô hình MLP thực hiện ước lượng lực kẹp ban đầu phù hợp. Giá trị này được gửi đến khối xử lý trung tâm để làm cơ sở điều khiển tay kẹp trong giai đoạn bắt đầu tiếp xúc với vật thể.

Bước 4: Điều khiển tay kẹp tiếp xúc với vật thể

Khối xử lý trung tâm gửi tín hiệu điều khiển đến khối động cơ để điều khiển cơ cấu tay kẹp thực hiện thao tác đóng vào. Trong quá trình này, tay kẹp tiến dần đến vật thể và tạo lực kẹp theo giá trị lực kẹp ban đầu đã được dự đoán.

Bước 5: Thu nhận dữ liệu cảm biến xúc giác

Khi tay kẹp tiếp xúc với vật thể, khối cảm biến tiến hành thu thập dữ liệu lực tiếp xúc tại các ma trận cảm biến được bố trí trên tay kẹp. Dữ liệu cảm biến được gửi về khối xử lý trung tâm để đánh giá trạng thái tiếp xúc và mức độ ổn định của quá trình kẹp.

Bước 6: Đánh giá và điều chỉnh quá trình kẹp

Khối xử lý trung tâm sử dụng dữ liệu cảm biến xúc giác để đánh giá trạng thái kẹp vật. Nếu lực kẹp chưa phù hợp hoặc vật thể có xu hướng trượt, hệ thống sẽ điều chỉnh tín hiệu điều khiển gửi đến khối động cơ để thay đổi lực kẹp. Việc điều chỉnh này giúp hạn chế hiện tượng trượt vật và tránh tác dụng lực quá mức lên vật thể.

Bước 7: Hiển thị và theo dõi trạng thái hệ thống

Trong quá trình hoạt động, các thông tin quan trọng như hình ảnh nhận diện, trạng thái tay kẹp, dữ liệu cảm biến và kết quả điều khiển được đưa đến khối hiển thị. Thông qua khối này, người dùng có thể quan sát quá trình vận hành của hệ thống và thực hiện các thao tác điều khiển cần thiết.

3.2.3 Thiết kế từng khối

Tiếp theo, đề tài trình bày chức năng, phương án lựa chọn, thông số kỹ thuật, tính toán thiết kế và nguyên lý hoạt động của các khối phần cứng. Các nội dung liên quan đến phần mềm và thuật toán xử lý sẽ được trình bày ở các mục sau.

Khối cảm biến

◆ Chức năng

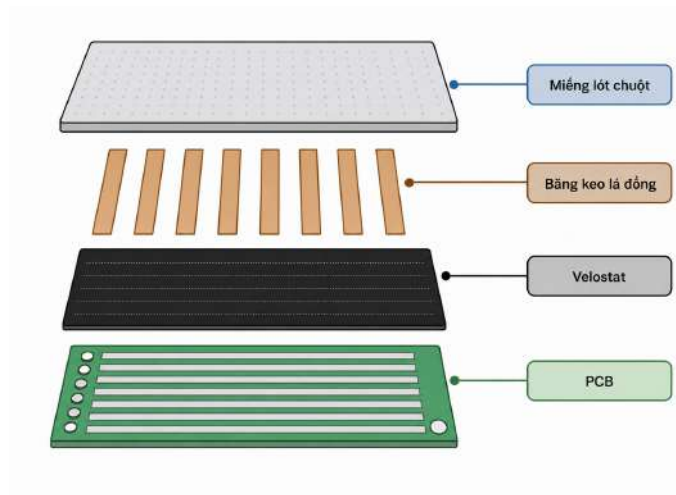
Khối cảm biến có nhiệm vụ thu nhận tín hiệu lực tiếp xúc trên bề mặt tay kẹp thông qua ma trận cảm biến xúc giác. Dữ liệu từ khối cảm biến được gửi về khối xử lý trung tâm để phục vụ quá trình đánh giá trạng thái tiếp xúc giữa tay kẹp và vật thể.

◆ Phương án và lựa chọn

- *Lựa chọn loại cảm biến và phương án thực hiện*

Velostat là vật liệu dạng tấm polymer, thường được làm từ polyolefin có pha muội than để tạo khả năng dẫn điện [12]. Khi vật liệu bị uốn, nén hoặc chịu áp lực, sự tiếp xúc giữa các hạt dẫn điện bên trong thay đổi, làm thay đổi điện trở thể tích (bulk resistance) của vật liệu [6, 12]. Nhờ đặc tính điện trở thay đổi theo tác động cơ học, Velostat có thể được ứng dụng để chế tạo cảm biến áp suất hoặc cảm biến lực đơn giản. Phương án này được lựa chọn do Velostat có cấu tạo đơn giản, dễ thi công, chi phí thấp, độ linh hoạt cao và phù hợp với yêu cầu chế tạo cảm biến dạng ma trận trên bề mặt ngón tay kẹp [5, 12].

Để tạo các đường dẫn điện cho cảm biến, đề tài sử dụng lá đồng làm điện cực hàng và cột. So với lá nhôm, lá đồng có độ dẫn điện tốt hơn, điện trở suất thấp hơn và thuận tiện hơn trong quá trình hàn nối. Các dải lá đồng được bố trí theo dạng ma trận với khoảng cách giữa các đường dẫn là 2 mm, giúp tạo các điểm cảm biến tại vị trí giao nhau giữa hàng và cột.



Hình 3.3: Minh họa thiết kế lắp ráp cảm biến xúc giác

Các vật liệu chính được sử dụng để thiết kế cảm biến gồm:

- **Lamination Sheet:** sử dụng vật liệu cao su non từ miếng lót chuột, có tác dụng tăng ma sát và phân bố lực tốt hơn trên bề mặt cảm biến.
- **Velostat Sheet:** là lớp vật liệu cảm biến chính, có điện trở thay đổi khi chịu tác động lực.
- *Lựa chọn linh kiện và tính toán*

Tấm Velostat



Hình 3.4: Tấm Velostat

- Kích thước: $11'' \times 11''$ ($280\text{ mm} \times 280\text{ mm}$).
- Độ dày: 8 mil (0.2 mm).

- Trọng lượng: 18.66 g.
- Giới hạn nhiệt độ: $-45^{\circ}C$ đến $65^{\circ}C$.
- Khả năng hàn kín nhiệt: có.
- Điện trở suất khối: $< 500 \Omega.cm$.
- Điện trở suất bề mặt: $< 31,000 \Omega/sq.cm$.

Diode chỉnh lưu 1N5819

- Điện áp ngược cực đại: 40 V.
- Dòng chỉnh lưu trung bình: 1 A.
- Điện áp rơi thuận: khoảng 510 mV tại 1 A.
- Tốc độ phục hồi: $\leq 500 ns$.
- Dòng rò ngược: 500 μA tại 40 V.
- Điện dung tiếp giáp: 30 pF tại 10 V, 1 MHz.
- Kiểu đóng gói: SOD-123F.

ADC ADS1115



Hình 3.5: Module ADC ADS1115

- Độ phân giải: 16 bit.
- Số kênh ngõ vào: 4 kênh single-ended hoặc 2 kênh differential.

- Chuẩn giao tiếp: I2C.
- Điện áp hoạt động: 2.0 V đến 5.5 V.
- Tốc độ lấy mẫu: từ 8 đến 860 mẫu/giây.
- Có bộ khuếch đại lập trình được PGA.
- Dải đo có thể cấu hình: $\pm 6.144\text{ V}$ đến $\pm 0.256\text{ V}$.

Điện trở SMD 0805

- Kiểu linh kiện: điện trở dán SMD.
- Kích thước: 0805.
- Giá trị điện trở: $470\ \Omega$.

Do bộ xử lý trung tâm không có sẵn ngõ vào analog, đề tài sử dụng ADC ADS1115 để đọc tín hiệu điện áp từ các hàng của ma trận cảm biến. Các cột của ma trận được điều khiển lần lượt bằng các chân GPIO, trong khi các hàng được đưa về ADC để thu nhận tín hiệu tương ứng với lực tác động lên cảm biến.

Hệ thống sử dụng bốn tấm cảm biến xúc giác, gồm hai tấm ma trận 3×3 ở đốt trong và hai tấm ma trận 2×3 ở đốt ngoài. Cách bố trí cảm biến dạng ma trận giúp hệ thống thu nhận phân bố tín hiệu tiếp xúc trên bề mặt tay kẹp [3, 5]. Tổng số điểm xúc giác là:

$$2 \times (3 \times 3) + 2 \times (2 \times 3) = 30 \text{ điểm xúc giác}$$

Theo thiết kế mạch, các hàng cảm biến được dùng chung thành ba đường ROW1, ROW2 và ROW3; các cột được tách thành mười đường COL1 đến COL10. Vì vậy, số đường tín hiệu chính dùng cho quá trình quét cảm biến là:

$$3 \text{ hàng} + 10 \text{ cột} = 13 \text{ đường tín hiệu}$$

Trên các đường cột, đề tài sử dụng diode 1N5819 để cách ly tín hiệu giữa các nhánh, hạn chế dòng điện chạy ngược và giảm ảnh hưởng chéo giữa các điểm cảm biến khi quét ma trận.

Các điện trở R1, R2 và R3 được mắc từ các đường ROW1, ROW2 và ROW3 xuống GND. Trong mạch thực tế, điện trở được sử dụng là loại SMD 0805 giá trị 470 Ω . Điện áp đưa về ADC được mô tả gần đúng theo công thức:

$$V_{ROW} = V_{CC} \times \frac{R}{R + R_{Velo}} \quad (3.1)$$

Trong đó, V_{ROW} là điện áp đưa về ADC, V_{CC} là điện áp điều khiển cột, R là điện trở kéo xuống và R_{Velo} là điện trở tương đương của Velostat tại điểm cảm biến.

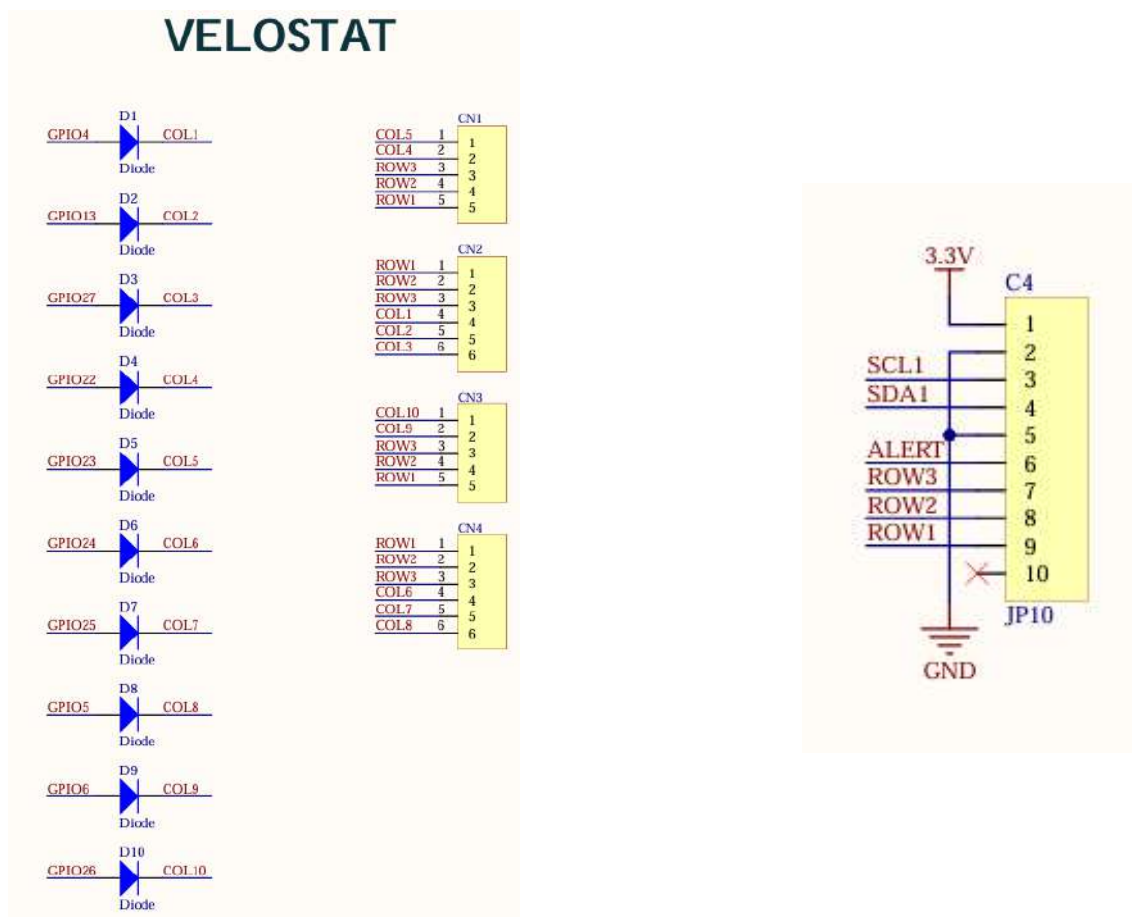
Khi có lực tác động, điện trở của Velostat giảm làm điện áp tại ngõ vào ADC tăng lên [6, 12]. Sự thay đổi điện áp này được ADS1115 chuyển đổi thành dữ liệu số để khối xử lý trung tâm đánh giá trạng thái tiếp xúc. Ví dụ:

$$V_{ROW} = 3.3 \times \frac{470}{470 + 470} = 1.65 \text{ V} \quad (3.2)$$

Giá trị điện áp này nằm trong dải đo phù hợp của ADC, đồng thời phản ánh được sự thay đổi lực tác động lên cảm biến. Như vậy, việc sử dụng điện trở 470 Ω giúp mạch cảm biến tạo được tín hiệu điện áp thay đổi theo lực tác động và phù hợp với quá trình đọc dữ liệu bằng ADS1115.

- *Sơ đồ nguyên lý*

Sơ đồ nguyên lý được trình bày trong Hình 3.6. Khối này bao gồm các đường điều khiển cột, các đường đọc tín hiệu hàng, diode cách ly và module ADC ADS1115 dùng để chuyển đổi tín hiệu analog từ cảm biến sang dữ liệu số.



Hình 3.6: Sơ đồ nguyên lý khối cảm biến và module ADS1115

Dựa vào sơ đồ nguyên lý, các cột của ma trận cảm biến được điều khiển thông qua các đường COL1 đến COL10. Tại mỗi thời điểm, khối xử lý trung tâm kích hoạt một cột và đọc tín hiệu phản hồi trên ba đường ROW1, ROW2 và ROW3 thông qua ADC ADS1115. Quá trình này được lặp lại cho đến khi quét hết toàn bộ các cột của ma trận.

Các diode 1N5819 được mắc nối tiếp trên các đường cột nhằm cách ly tín hiệu giữa các nhánh, hạn chế dòng điện chạy ngược và giảm ảnh hưởng chéo trong quá trình quét. Các điện trở R1, R2 và R3 được mắc từ ROW1, ROW2 và ROW3 xuống GND, sử dụng loại SMD 0805 giá trị 470 Ω . Các điện trở này có nhiệm vụ kéo tín hiệu hàng về mức thấp khi chưa có tác động lực, đồng thời tạo mạch chia áp với điện trở thay đổi của lớp Velostat.

ADS1115 giao tiếp với khối xử lý trung tâm thông qua chuẩn I2C, gồm hai đường

SDA1 và SCL1. Các tín hiệu analog từ ba đường hàng được đưa vào các kênh ADC để chuyển đổi thành dữ liệu số, phục vụ cho việc đánh giá trạng thái tiếp xúc giữa tay kẹp và vật thể.

Khởi động cơ

◆ Chức năng

Khởi động cơ có nhiệm vụ nhận tín hiệu điều khiển từ khối xử lý trung tâm và tạo chuyển động cho cơ cấu tay kẹp. Trong đề tài này, hệ thống sử dụng hai động cơ servo để điều khiển hai phía của tay kẹp, giúp tay kẹp thực hiện thao tác đóng, mở và giữ vật.

◆ Phương án và lựa chọn

• Lựa chọn linh kiện

Đối với cơ cấu tay kẹp, động cơ cần có khả năng điều khiển vị trí tương đối chính xác, mô-men xoắn đủ lớn và dễ tích hợp với chương trình điều khiển. Vì vậy, đề tài lựa chọn động cơ servo thay vì động cơ DC thông thường. Servo có ưu điểm là dễ điều khiển bằng tín hiệu PWM, có khả năng giữ vị trí và phù hợp với cơ cấu tay kẹp kích thước nhỏ. Trong mô hình thực tế, đề tài sử dụng hai servo TD8120MG để điều khiển cơ cấu tay kẹp hai ngón.

Động cơ servo TD8120MG



Hình 3.7: Động cơ servo TD8120MG

- Điện áp hoạt động: 4.8 V và 7.2 V.
- Tốc độ không tải: 0.18 s/60° tại 4.8 V; 0.14 s/60° tại 7.2 V.
- Dòng không tải: 210 mA tại 4.8 V; 260 mA tại 7.2 V.
- Mô-men xoắn cực đại: 20.5 kg.cm tại 4.8 V; 22.8 kg.cm tại 7.2 V.
- Dòng điện khi kẹt trục: 2100 mA ± 10% tại 4.8 V; 2700 mA ± 10% tại 7.2 V.
- Dải xung điều khiển: 500–2500 μs.
- Góc quay: 270° ± 3°.
- Kích thước: 40.0 × 20.5 × 40.5 mm.
- Khối lượng: 56 g ± 5%.

Servo RC đã tích hợp sẵn mạch điều khiển bên trong nên hệ thống không cần sử dụng mạch cầu H hay IC điều khiển động cơ rời. Khối xử lý trung tâm chỉ cần xuất hai tín hiệu PWM riêng biệt để điều khiển hai servo.

• *Tính toán nguồn cấp*

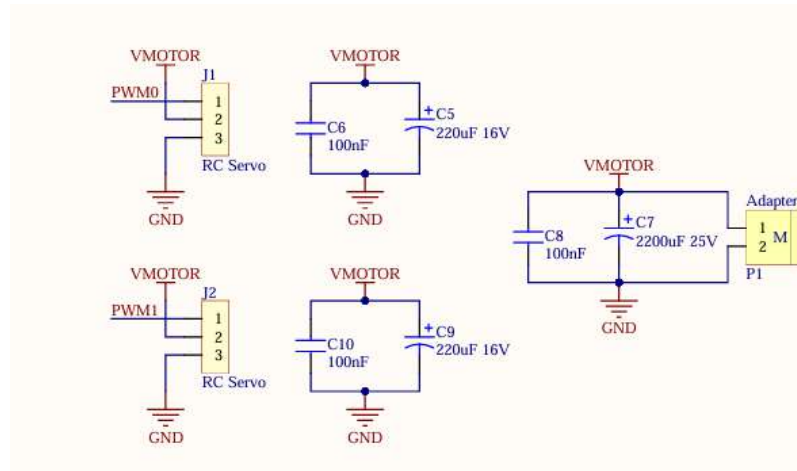
Trong hệ thống thực tế, hai servo TD8120MG được cấp chung bằng nguồn 6 V–5 A. Dòng cấp trung bình tối đa cho mỗi servo có thể ước lượng như sau:

$$I_{servo} = \frac{I_{nguồn}}{2} = \frac{5}{2} = 2.5 \text{ A} \quad (3.3)$$

Như vậy, trong điều kiện lý tưởng, mỗi servo có thể được cấp dòng xấp xỉ 2.5 A. Giá trị này phù hợp với yêu cầu hoạt động của cơ cấu tay kẹp và giúp hệ thống đáp ứng tốt hơn khi cần tạo lực kẹp trong quá trình thao tác với vật thể.

• *Sơ đồ nguyên lý*

Sơ đồ nguyên lý của khối động cơ được trình bày như Hình 3.8. Khối này gồm hai ngõ kết nối servo J1 và J2, hai đường tín hiệu điều khiển PWM0 và PWM1, đường cấp nguồn VMOTOR, GND và các tụ lọc nguồn cho servo.



Hình 3.8: Sơ đồ nguyên lý khối động cơ

Dựa vào sơ đồ nguyên lý, hai servo được kết nối với mạch thông qua hai đầu nối RC Servo J1 và J2. Mỗi servo gồm ba chân: tín hiệu điều khiển, nguồn VMOTOR và GND. Hai tín hiệu PWM0 và PWM1 được xuất từ khối xử lý trung tâm để điều khiển riêng từng servo của tay kẹp.

Nguồn cấp cho servo sử dụng đường VMOTOR và GND. Trong thực tế, hệ thống sử dụng nguồn riêng 6 V–5 A để cấp chung cho hai servo. GND của nguồn servo được nối chung với GND của khối xử lý trung tâm để đảm bảo tín hiệu PWM có cùng mức tham chiếu.

Các tụ C6, C8 và C10 có giá trị 100 nF, được dùng để lọc nhiễu cao tần trên đường nguồn. Các tụ C5 và C9 có giá trị 220 μ F/16 V, được bố trí gần từng ngõ servo để hỗ trợ ổn định điện áp khi servo thay đổi tải. Ngoài ra, tụ C7 có giá trị 2200 μ F/25 V được đặt tại ngõ vào nguồn VMOTOR nhằm giảm sụt áp tức thời khi servo bắt đầu chuyển động hoặc chịu tải lớn.

Khối điều khiển trung tâm

Khối xử lý trung tâm

◆ Chức năng

Khối xử lý trung tâm có nhiệm vụ tiếp nhận và xử lý dữ liệu từ các thành phần

trong hệ thống. Khối này nhận dữ liệu hình ảnh từ camera, đọc dữ liệu cảm biến xúc giác thông qua ADC, tính toán lực kẹp và xuất tín hiệu điều khiển đến khối động cơ.

◆ Phương án và lựa chọn

- *Lựa chọn linh kiện*

Trong đề tài này, Raspberry Pi 4 được lựa chọn làm khối xử lý trung tâm. So với các vi điều khiển thông thường, Raspberry Pi 4 có khả năng xử lý cao hơn, hỗ trợ hệ điều hành, camera, các giao tiếp ngoại vi và có thể chạy trực tiếp các chương trình xử lý ảnh, học máy và giao diện người dùng.

Raspberry Pi 4 phù hợp với yêu cầu của hệ thống vì hỗ trợ các giao tiếp cần thiết như USB, I2C, GPIO và PWM. Nhờ đó, thiết bị có thể kết nối với camera, ADC ADS1115, khối cảm biến xúc giác và hai servo điều khiển tay kẹp.

Raspberry Pi 4

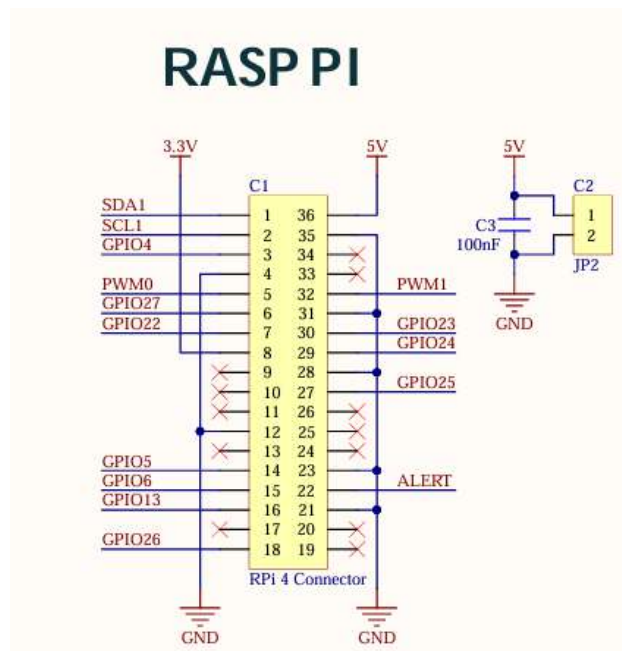


Hình 3.9: Raspberry Pi 4

Một số thông số chính của Raspberry Pi 4:

- Bộ xử lý: Broadcom BCM2711, Quad-core Cortex-A72 64-bit.
- Tốc độ xử lý: 1.5 GHz.
- Bộ nhớ RAM: tùy phiên bản, từ 2 GB đến 8 GB.

- Sơ đồ nguyên lý kết nối Raspberry Pi 4 được trình bày như Hình 3.10. Khối này gồm đầu nối Raspberry Pi 4, các chân cấp nguồn, các chân giao tiếp I2C, các chân GPIO điều khiển cột cảm biến và các chân PWM điều khiển servo.



Trong sơ đồ, hai chân SDA1 và SCL1 được dùng để giao tiếp I2C với module ADC ADS1115. Các chân GPIO được sử dụng để điều khiển các cột của ma trận cảm biến xúc giác, trong khi hai tín hiệu PWM0 và PWM1 được dùng để điều khiển hai servo của tay kẹp. Ngoài ra, chân ALERT được kết nối với ADS1115 để hỗ trợ nhận biết trạng thái chuyển đổi khi cần thiết.

43

tham chiếu chung cho các mạch ngoại vi. Tụ C3 có giá trị 100 nF được bố trí gần đường nguồn nhằm lọc nhiễu, giúp tín hiệu cấp cho mạch ổn định hơn trong quá trình hoạt động.

Khởi hiển thị

◆ Chức năng

Khởi hiển thị có nhiệm vụ hiển thị các thông tin quan trọng của hệ thống và hỗ trợ người dùng thao tác trong quá trình vận hành. Thông qua giao diện, người dùng có thể quan sát hình ảnh nhận diện, trạng thái tay kẹp, dữ liệu cảm biến xúc giác và gửi các lệnh điều khiển cần thiết cho chương trình.

◆ Phương án và lựa chọn

Trong đề tài này, giao diện hiển thị và điều khiển được xây dựng trực tiếp trên Raspberry Pi 4. Do đó, hệ thống không cần thiết kế thêm mạch giao tiếp UART, USB hay module hiển thị rời cho khối này. Raspberry Pi 4 vừa đảm nhiệm vai trò xử lý trung tâm, vừa xuất giao diện để người dùng theo dõi và tương tác với hệ thống.

Khối camera

◆ Chức năng

Khối camera có nhiệm vụ thu nhận hình ảnh vật thể trong vùng làm việc và truyền dữ liệu hình ảnh về khối xử lý trung tâm. Dữ liệu này được sử dụng cho quá trình nhận dạng vật thể và hỗ trợ dự đoán lực kẹp ban đầu.

◆ Phương án và lựa chọn

Trong đề tài này, camera Huddly GO 1.0 được sử dụng và kết nối trực tiếp với Raspberry Pi 4 thông qua cổng USB. Việc sử dụng webcam USB giúp hệ thống đơn giản hơn, không cần thiết kế thêm mạch giao tiếp riêng và thuận tiện cho quá trình thu nhận hình ảnh bằng chương trình xử lý trên Raspberry Pi 4.

Camera Huddly GO 1.0



Hình 3.11: Camera Huddly GO 1.0

- Loại camera: webcam USB.
- Kết nối: USB-C.
- Độ phân giải ngõ ra: HD 720p.
- Tốc độ khung hình: 30 *fps*.
- Góc nhìn rộng, phù hợp quan sát vùng thao tác của tay kẹp.

Khối nhận diện

◆ Chức năng

Khối nhận diện có nhiệm vụ tiếp nhận dữ liệu hình ảnh từ khối camera và xác định vật thể xuất hiện trong vùng làm việc của tay kẹp. Kết quả nhận diện được dùng để xác định đối tượng cần thao tác, đồng thời trích xuất một số đặc trưng hình ảnh phục vụ cho quá trình dự đoán lực kẹp ban đầu.

◆ Phương án thực hiện

Trong đề tài này, khối nhận diện được xây dựng dựa trên mô hình YOLOv8n [7]. Đây là phiên bản nhỏ của họ YOLOv8, có ưu điểm là tốc độ xử lý nhanh, phù hợp với yêu cầu nhận dạng vật thể theo thời gian thực trên hệ thống nhúng [7]. Dữ liệu đầu ra của khối nhận diện bao gồm nhãn vật thể, độ tin cậy và thông tin vùng bao quanh vật thể.

Khối nhận diện chủ yếu thuộc phần xử lý phần mềm. Vì vậy, cách huấn luyện, triển khai và xử lý kết quả của mô hình YOLOv8n sẽ được trình bày chi tiết hơn ở phần thiết kế phần mềm.

Khối dự đoán

◆ Chức năng

Khối dự đoán có nhiệm vụ hỗ trợ hệ thống lựa chọn mức kẹp ban đầu và đánh giá trạng thái kẹp trong quá trình thao tác với vật thể. Kết quả từ khối dự đoán được gửi về khối xử lý trung tâm để điều khiển tay kẹp phù hợp hơn với từng vật thể và trạng thái tiếp xúc thực tế.

◆ Phương án thực hiện

Trong đề tài này, khối dự đoán sử dụng hai mô hình MLP chính. Mô hình thứ nhất là MLPRegressor, có nhiệm vụ dự đoán lực ban đầu dựa trên các đặc trưng thu được từ khối nhận diện.

Mô hình thứ hai là MLPClassifier, có nhiệm vụ phân loại trạng thái kẹp dựa trên dữ liệu thị giác, giá trị kẹp hiện tại và các đặc trưng cảm biến xúc giác [2, 13]. Các trạng thái đầu ra gồm kẹp ổn định, trượt vật, lực kẹp quá lớn hoặc kẹp chưa đạt yêu cầu.

Khối dự đoán là một khối xử lý phần mềm, không phải một mạch phần cứng riêng biệt. Vì vậy quá trình triển khai hai mô hình MLP sẽ được trình bày chi tiết hơn ở phần thiết kế phần mềm.

Khối nguồn

◆ Chức năng

Khối nguồn có nhiệm vụ cung cấp điện áp hoạt động cho các thành phần trong hệ thống, bao gồm Raspberry Pi 4, mạch cảm biến và hai động cơ servo. Nguồn cấp cần đảm bảo ổn định để hệ thống xử lý dữ liệu, đọc cảm biến và điều khiển tay kẹp

hoạt động bình thường.

◆ Phương án và lựa chọn

Trong hệ thống thực tế, Raspberry Pi 4 và hai servo được cấp nguồn bằng hai adapter riêng biệt. Raspberry Pi 4 sử dụng adapter model CTD-050300A, có điện áp ngõ ra 5 V và dòng tối đa 3000 mA. Hai servo sử dụng adapter riêng model ST-909, có điện áp ngõ ra 6 V và dòng tối đa 5 A.

Việc sử dụng nguồn riêng cho Raspberry Pi 4 và servo giúp hạn chế ảnh hưởng của sụt áp, nhiễu nguồn khi servo hoạt động, từ đó giúp khối xử lý trung tâm hoạt động ổn định hơn.



Hình 3.12: Adapter CTD-050300A

- Nguồn cấp Raspberry Pi 4: adapter CTD-050300A.
 - Điện áp ngõ vào: 100–240 V, 50/60 Hz.
 - Điện áp ngõ ra: 5 V.
 - Dòng ngõ ra tối đa: 3000 mA.
- Nguồn cấp servo: adapter ST-909.
 - Điện áp ngõ vào: 100–240 V, 50/60 Hz.
 - Điện áp ngõ ra: 6 V.
 - Dòng ngõ ra tối đa: 5 A.

3.3 Thiết kế phần mềm

3.3.1 Chức năng hoạt động của phần mềm

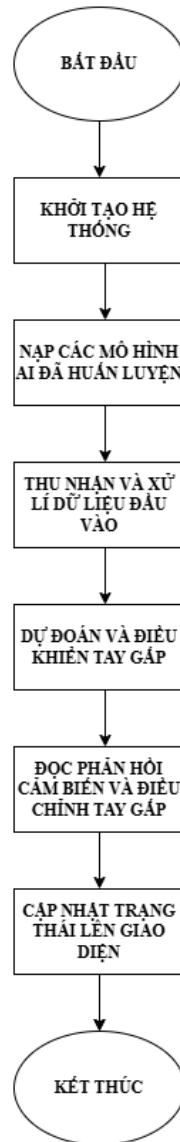
Phần mềm của hệ thống được xây dựng nhằm thu nhận dữ liệu, xử lý thông tin và điều khiển tay kẹp hoạt động theo mục tiêu của đề tài. Các chức năng chính của phần mềm gồm:

- **Thu nhận và xử lý hình ảnh:** nhận dữ liệu từ camera, nhận dạng vật thể trong vùng làm việc và trích xuất các đặc trưng cần thiết cho quá trình dự đoán lực kẹp.
- **Đọc và xử lý dữ liệu xúc giác:** thu nhận dữ liệu từ các cảm biến trên tay kẹp, xử lý tín hiệu để đánh giá mức độ tiếp xúc và trạng thái kẹp vật.
- **Dự đoán lực kẹp ban đầu:** sử dụng các đặc trưng của vật thể để ước lượng mức kẹp ban đầu phù hợp, tránh việc dùng một giá trị cố định cho mọi vật thể [8, 13].
- **Điều khiển và điều chỉnh tay kẹp:** gửi lệnh điều khiển servo để tay kẹp mở, đóng và giữ vật. Trong quá trình kẹp, chương trình theo dõi dữ liệu xúc giác để điều chỉnh lực kẹp khi cần thiết.
- **Giao diện và theo dõi hệ thống:** hiển thị hình ảnh nhận dạng, trạng thái tay kẹp và dữ liệu cảm biến, giúp người dùng quan sát và kiểm tra quá trình vận hành.
- **Lưu trữ dữ liệu:** ghi lại dữ liệu vật thể, lực kẹp, tín hiệu cảm biến và trạng thái kẹp để phục vụ đánh giá kết quả và huấn luyện mô hình.

3.3.2 Lưu đồ tổng quát hoạt động của hệ thống

Lưu đồ tổng quát hoạt động của hệ thống phần mềm được trình bày như Hình 3.13. Khi bắt đầu, chương trình khởi tạo các thành phần cần thiết như camera, cảm biến

xúc giác, bộ điều khiển tay kẹp và giao diện hiển thị. Sau đó, hệ thống nạp các mô hình AI đã được huấn luyện trước, gồm mô hình YOLO dùng để nhận diện vật thể và các mô hình MLP dùng để dự đoán, đánh giá trạng thái kẹp.



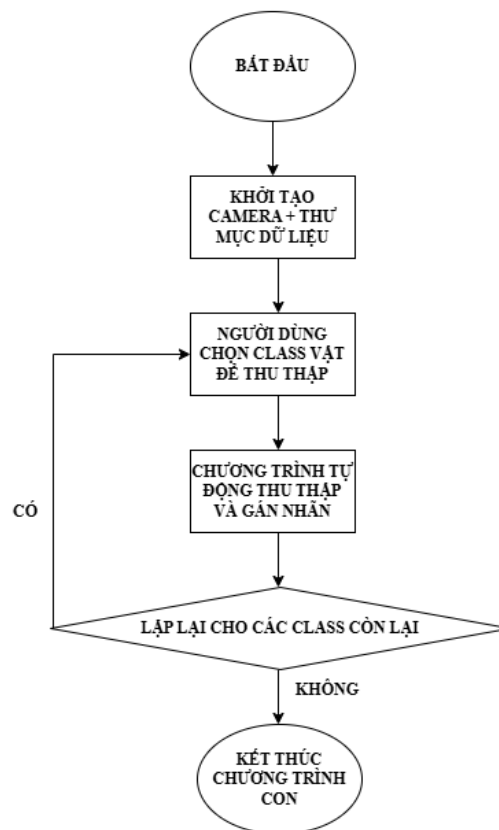
Hình 3.13: Lưu đồ tổng quát hoạt động của hệ thống

Trong quá trình hoạt động, hệ thống thu nhận và xử lý dữ liệu đầu vào từ camera và cảm biến xúc giác. Dữ liệu hình ảnh được dùng để nhận diện vật thể, trong khi dữ liệu xúc giác được dùng để đánh giá trạng thái tiếp xúc. Từ các dữ liệu này, chương trình dự đoán tỉ lệ kẹp ban đầu, điều khiển tay kẹp và tiếp tục điều chỉnh dựa trên phản hồi xúc giác. Các thông tin về hình ảnh, trạng thái kẹp, dữ liệu cảm biến và

lệnh điều khiển được cập nhật lên giao diện để người dùng theo dõi.

3.3.3 Lưu đồ thu thập dữ liệu cho mô hình YOLO

Quá trình thu thập dữ liệu ảnh cho mô hình YOLO được thực hiện nhằm xây dựng tập dữ liệu nhận diện các vật thể trong vùng làm việc của tay kẹp được trình bày như Hình 3.14.



Hình 3.14: Lưu đồ thu thập dữ liệu cho mô hình YOLO

Khi bắt đầu chương trình, hệ thống khởi tạo camera và các thư mục dùng để lưu ảnh, nhãn dữ liệu. Sau đó, người dùng chọn lớp vật thể cần thu thập. Chương trình tiến hành thu ảnh và tạo nhãn tương ứng cho lớp đó. Nếu còn lớp vật thể khác, quá trình được lặp lại cho đến khi hoàn thành toàn bộ tập dữ liệu.

Trong đề tài, tập dữ liệu YOLO được xây dựng gồm 3 lớp vật thể: *ball*, *can* và

bottle. Mỗi lớp gồm 3 vật thể khác nhau, tổng cộng 9 vật thể được sử dụng để thu thập dữ liệu.

Bảng 3.1: Quy ước các lớp vật thể trong tập dữ liệu YOLO

Class ID	Tên lớp	Số vật thể
0	Ball	3
1	Can	3
2	Bottle	3

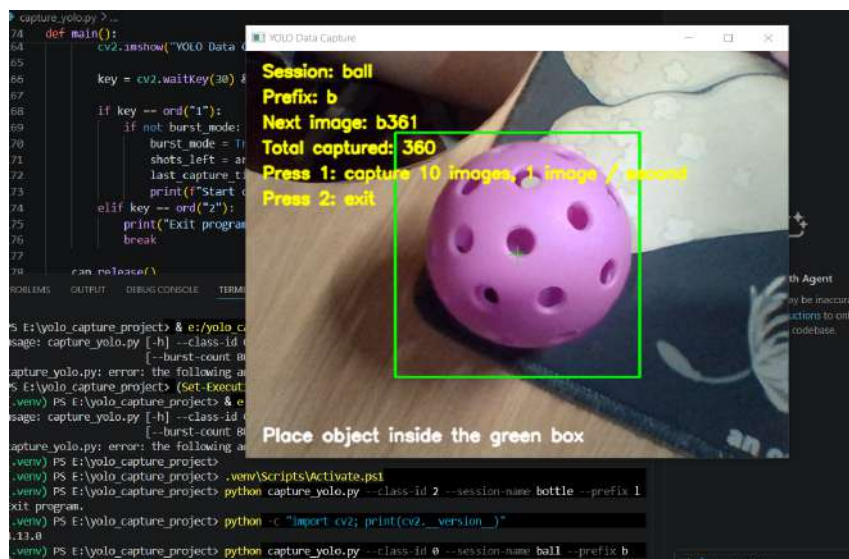
Để quá trình thu thập dữ liệu được nhanh và đồng nhất hơn, đề tài đã xây dựng một chương trình riêng bằng Python và OpenCV. Chương trình cho phép người dùng nhập mã lớp, tên phiên thu thập và tiền tố tên ảnh. Camera được thiết lập ở độ phân giải 640×480 pixel. Trên giao diện thu thập, chương trình hiển thị một khung màu xanh ở giữa ảnh để người dùng đặt vật thể vào đúng vùng cần chụp. Khi người dùng nhấn phím chụp, chương trình tự động lưu ảnh theo từng đợt và đồng thời tạo nhãn YOLO tương ứng.

Nhãn của mỗi ảnh được lưu theo định dạng YOLO với cấu trúc:

class_id x_center y_center width height

Trong đó, *class_id* là mã lớp của vật thể; *x_center* và *y_center* là tọa độ tâm của khung bao; *width* và *height* lần lượt là chiều rộng và chiều cao của khung bao. Các giá trị tọa độ và kích thước đều được chuẩn hóa theo kích thước ảnh, nằm trong khoảng từ 0 đến 1. Cách lưu này giúp dữ liệu có thể được sử dụng trực tiếp cho quá trình huấn luyện YOLO.

Sau khi hoàn tất thu thập, tập dữ liệu được kiểm tra lại để bảo đảm mỗi ảnh đều có tệp nhãn tương ứng. Tiếp theo, chương trình chia dữ liệu được sử dụng để tách dữ liệu thành tập train và val theo tỉ lệ 80% và 20%. Chương trình dựa trên tiền tố tên ảnh để phân nhóm dữ liệu, trong đó tiền tố *b* tương ứng với *ball*, tiền tố *c* tương ứng với *can* và tiền tố *l* tương ứng với *bottle*. Các nội dung chính trong quá trình xây dựng tập dữ liệu YOLO được minh họa trong Hình 3.15.



(a) Giao diện chương trình thu thập dữ liệu YOLO.

0 0.500000 0.500000 0.450000 0.600000

(b) Minh họa ảnh và nhãn dữ liệu theo định dạng YOLO



(c) Cấu trúc thư mục dữ liệu sau khi chia tập train và val

Hình 3.15: Một số hình ảnh minh họa trong quá trình xây dựng tập dữ liệu YOLO

Tổng số ảnh thu thập được là 1000 ảnh. Số lượng ảnh theo từng lớp và kết quả chia dữ liệu được trình bày trong Bảng 3.2.

Bảng 3.2: Số lượng dữ liệu YOLO sau khi chia tập train và val

Lớp vật thể	Số vật thể	Tổng ảnh	Train	Val
Ball	3	380	304	76
Can	3	300	240	60
Bottle	3	320	256	64
Tổng	9	1000	800	200

Sau khi chia tập dữ liệu, các ảnh và nhãn tương ứng được sắp xếp vào các thư mục train và val. Cấu trúc này được sử dụng trong tệp cấu hình *data.yaml* để phục vụ cho quá trình huấn luyện mô hình YOLO ở bước tiếp theo.

3.3.4 Lưu đồ huấn luyện mô hình YOLO

Lưu đồ huấn luyện mô hình YOLO được trình bày trong Hình 3.16.



Hình 3.16: Lưu đồ huấn luyện mô hình YOLO

Sau khi hoàn tất quá trình thu thập và chia dữ liệu, tập dữ liệu được đưa lên Google Colab để tiến hành huấn luyện. Google Colab được lựa chọn vì hỗ trợ môi trường làm việc trực tuyến, dễ dàng kết nối với Google Drive để lưu dữ liệu, kết quả huấn luyện và mô hình sau khi xuất. Trong phiên huấn luyện, môi trường được thiết lập với GPU, bộ nhớ hệ thống và dung lượng lưu trữ phù hợp, giúp tăng tốc quá trình huấn luyện so với khi chạy trực tiếp trên Raspberry Pi, do Raspberry Pi có tài nguyên tính toán hạn chế và còn phải đảm nhiệm các tác vụ khác như đọc cảm biến, điều khiển tay kẹp và hiển thị giao diện.

Sau khi chuẩn bị môi trường, chương trình kết nối Google Drive và thiết lập tệp cấu hình *data.yaml*. Tệp này khai báo đường dẫn đến tập train, tập val và tên các lớp vật thể cần nhận diện. Lệnh huấn luyện được sử dụng trong Google Colab có dạng:

```
yolo detect train model=yolov8n.pt epochs=100 imgsz=640 batch=8 device=0
```

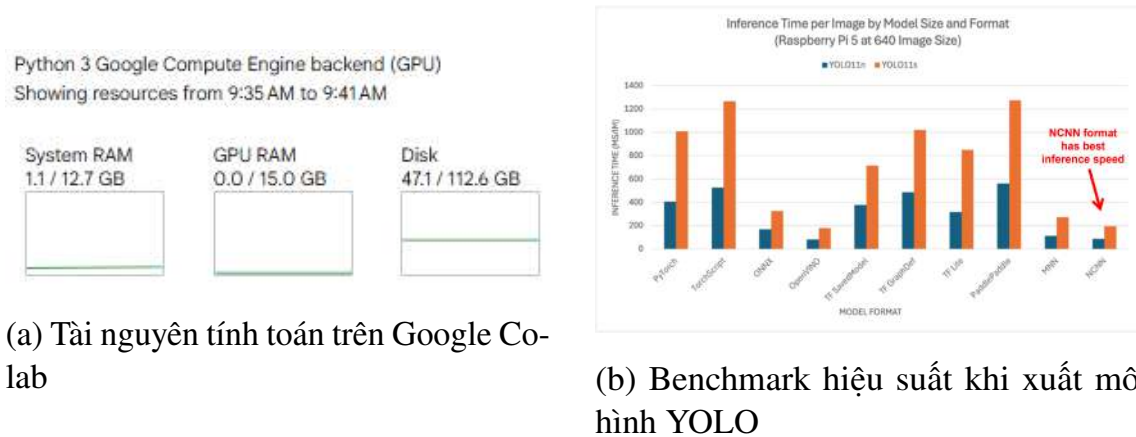
Trong đó, *model=yolov8n.pt* được chọn vì YOLOv8n là phiên bản nhỏ của YOLOv8, có tốc độ xử lý nhanh và phù hợp để triển khai trên Raspberry Pi [7]. Thông số *epochs=100* cho biết mô hình được huấn luyện qua 100 vòng lặp trên tập dữ liệu, giúp mô hình có đủ thời gian học đặc trưng của các vật thể. Thông số *imgsz=640* quy định kích thước ảnh đầu vào khi huấn luyện, phù hợp với kích thước phổ biến của YOLO và vẫn giữ được đủ thông tin của vật thể. Thông số *batch=8* được chọn để cân bằng giữa tốc độ huấn luyện và giới hạn bộ nhớ GPU. Cuối cùng, *device=0* cho phép quá trình huấn luyện sử dụng GPU đầu tiên trong môi trường Colab.

Sau khi quá trình huấn luyện hoàn tất, mô hình có kết quả tốt nhất được lưu trong thư mục kết quả với tên *best.pt*. Đây là mô hình được chọn để xuất sang định dạng phù hợp cho quá trình chạy thực tế trên Raspberry Pi.

Mô hình *best.pt* sau đó được xuất sang định dạng NCNN bằng lệnh:

```
yolo export model="/content/drive/weights/best.pt"format=ncnn
```

Định dạng NCNN được lựa chọn vì phù hợp với các thiết bị nhúng và nền tảng ARM như Raspberry Pi. So với việc chạy trực tiếp mô hình PyTorch, mô hình sau khi xuất sang NCNN thường có khả năng suy luận nhanh hơn và nhẹ hơn khi triển khai trên thiết bị có tài nguyên hạn chế. Tài nguyên tính toán trên Google Colab và benchmark hiệu suất khi xuất mô hình YOLO sang các định dạng khác nhau được minh họa trong Hình 3.17.

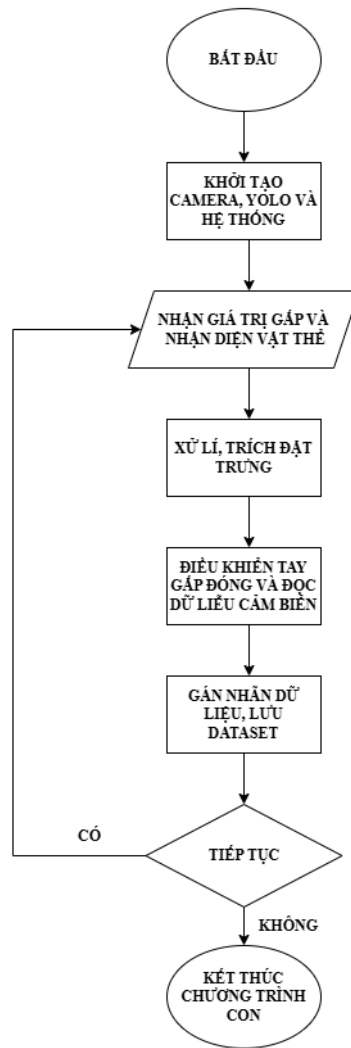


Hình 3.17: Minh họa môi trường huấn luyện và hiệu suất xuất mô hình YOLO

3.3.5 Lưu đồ thu thập dữ liệu cho mô hình MLP

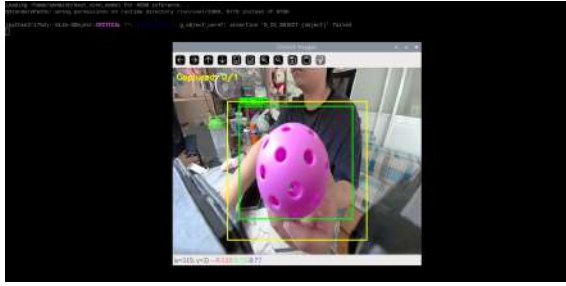
Quá trình này được thực hiện nhằm tạo tập dữ liệu phục vụ cho hai bài toán huấn luyện mô hình MLP. Lưu đồ thu thập dữ liệu cho mô hình MLP được trình bày trong Hình 3.18.

Hệ thống trước hết khởi tạo camera, mô hình YOLO, tay kẹp và cảm biến xúc giác. Khi vật thể được đặt vào vùng làm việc, YOLO nhận diện vật thể và trích xuất các đặc trưng hình ảnh như lớp vật thể, vị trí khung bao, kích thước khung bao và độ tin cậy nhận diện. Sau đó, người dùng nhập giá trị kẹp thử nghiệm, tay kẹp đóng lại theo giá trị này và cảm biến xúc giác tiến hành thu thập dữ liệu trong quá trình tiếp xúc với vật thể [2, 4].



Hình 3.18: Lưu đồ thu thập dữ liệu cho mô hình MLP

Sau mỗi lần kẹp, người dùng gán nhãn trạng thái cho kết quả kẹp. Các nhãn được sử dụng gồm *success*, *slip*, *overforce* và *fail*. Trong đó, *success* thể hiện vật được giữ ổn định, *slip* thể hiện vật bị trượt, *overforce* thể hiện tay kẹp tác động lực quá lớn và *fail* thể hiện lần kẹp không đạt yêu cầu. Các dữ liệu này được lưu lại cùng với đặc trưng hình ảnh, thông tin điều khiển và dữ liệu cảm biến xúc giác để tạo thành tập dữ liệu huấn luyện MLP. Quá trình thu thập dữ liệu được minh họa trong Hình 3.19.



(a) YOLO nhận diện vật thể

```
Enter grip percentage (0-100) or 'q' to skip this object: 72
Closing gripper to 72.0%...
Sampling tactile sensors... Press one of the following keys when ready:
y = success, s = slip, o = overforce, r = retry, f = fail
[INFO] Success recorded.
```

(b) Nhập tỉ lệ kẹp và gán nhãn



(c) Tay kẹp thực hiện thao tác kẹp

Hình 3.19: Minh họa quá trình thu thập dữ liệu cho mô hình MLP

Các nhóm dữ liệu chính được lưu trong tập dữ liệu MLP được trình bày trong Bảng 3.3.

Bảng 3.3: Các nhóm dữ liệu được thu thập cho mô hình MLP

Nhóm dữ liệu	Thông tin lưu trữ
Thông tin vật thể	Tên lớp vật thể, mã vật thể, tên ảnh
Đặc trưng hình ảnh	Kích thước khung bao, diện tích khung bao, vị trí tâm, độ tin cậy nhận diện
Thông tin điều khiển	Tỉ lệ kẹp, góc servo, xung điều khiển servo
Dữ liệu xúc giác	Lực trung bình, lực cực đại, vùng tiếp xúc và tổng lực trên các ma trận cảm biến
Nhãn trạng thái	success, slip, overforce, fail

Ví dụ trong Hình 3.19, YOLO nhận diện vật thuộc lớp *ball*. Người dùng nhập tỉ lệ kẹp thử nghiệm là 72%. Sau khi tay kẹp đóng lại và giữ được vật ổn định, người

dùng nhập nhãn y , tương ứng với trạng thái *success*. Một dòng dữ liệu minh họa được ghi nhận theo dạng rút gọn như Bảng 3.4.

Bảng 3.4: Ví dụ một mẫu dữ liệu kẹp được ghi nhận

Trường dữ liệu	Giá trị minh họa	Ý nghĩa
object_class	ball	Lớp vật thể được YOLO nhận diện
grip_ratio	0.72	Tỉ lệ kẹp do người dùng nhập
label	success	Trạng thái kẹp được gán nhãn
success	1	Kết quả kẹp thành công
bbox features	Tính từ YOLO	Đặc trưng khung bao vật thể
sensor features	Tính từ cảm biến	Đặc trưng lực và vùng tiếp xúc

Tập dữ liệu được thu thập trên 3 lớp vật thể giống với quá trình huấn luyện YOLO, gồm *ball*, *can* và *bottle*. Mỗi lớp gồm 3 vật thể khác nhau, tổng cộng 9 vật thể. Với mỗi vật thể, hệ thống thực hiện nhiều lần kẹp ở các tỉ lệ khác nhau để thu được dữ liệu đa dạng về trạng thái tiếp xúc. Kết quả sau quá trình thu thập tạo ra tập dữ liệu gồm 1090 mẫu kẹp, được sử dụng cho bước huấn luyện hai mô hình MLP ở phần tiếp theo.

Tiền xử lý dữ liệu cảm biến xúc giác

Do cảm biến xúc giác sử dụng vật liệu Velostat được chế tạo thủ công nên tín hiệu thu được có thể chịu ảnh hưởng bởi nhiễu, độ không đồng đều giữa các điểm đo, tính phi tuyến và hiện tượng trễ của vật liệu [6, 12]. Vì vậy, trước khi đưa dữ liệu cảm biến vào quá trình huấn luyện mô hình MLP, đề tài thực hiện một số bước tiền xử lý nhằm làm giảm ảnh hưởng của nhiễu và giúp dữ liệu đầu vào ổn định hơn.

Trước hết, hệ thống tiến hành hiệu chuẩn giá trị nền của cảm biến khi tay kẹp chưa tiếp xúc với vật thể. Giá trị nền tại mỗi điểm cảm biến được ghi nhận và dùng làm mốc tham chiếu. Khi hệ thống hoạt động, tín hiệu đọc được sẽ được trừ với giá trị nền tương ứng để giảm sai lệch ban đầu giữa các điểm đo. Giá trị sau hiệu chỉnh được xác định gần đúng theo công thức:

$$S'_i = \max(0, S_i - S_{i0}) \quad (3.4)$$

Trong đó, S_i là giá trị cảm biến tại điểm đo thứ i , S_{i0} là giá trị nền của điểm đo đó khi chưa có lực tác động và S'_i là giá trị sau khi hiệu chỉnh. Việc giới hạn giá trị nhỏ hơn 0 về 0 giúp loại bỏ các sai lệch âm do nhiễu hoặc dao động nhỏ của tín hiệu.

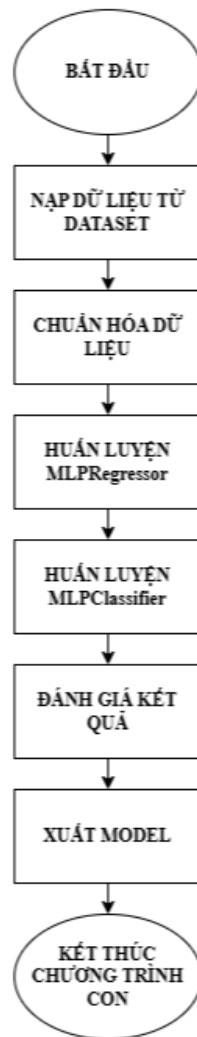
Sau bước hiệu chỉnh nền, dữ liệu cảm biến được thu trong một khoảng thời gian ngắn thay vì chỉ sử dụng một mẫu đơn lẻ. Các giá trị trong khoảng lấy mẫu được dùng để tính các đặc trưng như giá trị trung bình, giá trị cực đại, tổng tín hiệu và vùng tiếp xúc. Cách xử lý này giúp giảm ảnh hưởng của nhiễu tức thời và hạn chế tác động của hiện tượng trễ khi vật liệu Velostat chưa phản hồi ổn định ngay tại một thời điểm đo.

Các đặc trưng được trích xuất từ cảm biến bao gồm tổng tín hiệu bên trái, tổng tín hiệu bên phải, vùng tiếp xúc trung bình, giá trị trung bình và giá trị cực đại trên từng nhóm cảm biến. Việc sử dụng đặc trưng tổng hợp từ mảng cảm biến xúc giác phù hợp với hướng khai thác dữ liệu tiếp xúc trong các hệ thống nhận dạng và thao tác vật thể [2, 5]. Thay vì đưa trực tiếp toàn bộ giá trị thô của từng điểm đo vào mô hình, việc sử dụng các đặc trưng tổng hợp giúp mô hình MLP học được xu hướng thay đổi của trạng thái tiếp xúc ổn định hơn. Điều này cũng phù hợp với mục tiêu của đề tài là phân loại trạng thái kẹp như *success*, *slip* và *overforce*, thay vì đo lực tuyệt đối theo đơn vị vật lý.

Cuối cùng, trong quá trình huấn luyện, các đặc trưng dạng số được chuẩn hóa trước khi đưa vào mô hình MLP. Việc chuẩn hóa giúp các đặc trưng có thang đo phù hợp hơn, tránh trường hợp những đặc trưng có giá trị lớn chi phối quá trình huấn luyện. Nhờ các bước hiệu chỉnh nền, lọc theo cửa sổ thời gian, trích đặc trưng và chuẩn hóa dữ liệu, ảnh hưởng của nhiễu, tính phi tuyến và hiện tượng trễ của cảm biến Velostat được giảm bớt trước khi dữ liệu được sử dụng cho mô hình học máy.

3.3.6 Lưu đồ huấn luyện mô hình MLP

Lưu đồ huấn luyện mô hình MLP được trình bày như Hình 3.20. Quá trình huấn luyện được thực hiện trên Google Colab nhằm tận dụng môi trường tính toán trực tuyến và thuận tiện cho việc xử lý dữ liệu, huấn luyện mô hình và xuất file mô hình sau cùng.



Hình 3.20: Lưu đồ huấn luyện mô hình MLP

Theo lưu đồ, chương trình bắt đầu bằng việc nạp các tập dữ liệu đã thu thập, gồm dữ liệu đặc trưng hình ảnh, dữ liệu các lần kẹp và dữ liệu cảm biến xúc giác. Sau đó, dữ liệu được chuẩn hóa để phù hợp với quá trình huấn luyện, bao gồm xử lý giá trị thiếu, chuyển đổi kiểu dữ liệu, mã hóa các đặc trưng phân loại và chuẩn hóa các

đặc trưng dạng số [8, 13].

Hai mô hình MLP được huấn luyện gồm MLPRegressor và MLPClassifier. MLPRegressor dùng để dự đoán tỉ lệ kẹp ban đầu dựa trên thông tin vật thể và đặc trưng hình ảnh từ YOLO [8, 13]. MLPClassifier dùng để phân loại trạng thái kẹp dựa trên tỉ lệ kẹp hiện tại và dữ liệu cảm biến xúc giác [2, 13]. Các nhóm đặc trưng đầu vào được trình bày trong Bảng 3.5.

Đối với MLPRegressor, dữ liệu đầu vào chỉ sử dụng các thông tin có trước khi tay kẹp tiếp xúc vật thể, gồm lớp vật thể, vật thể cụ thể và đặc trưng khung bao từ YOLO. Đầu ra của mô hình là tỉ lệ kẹp ban đầu phù hợp với vật thể. Đối với MLPClassifier, dữ liệu đầu vào gồm thêm tỉ lệ kẹp hiện tại và các đặc trưng cảm biến xúc giác. Đầu ra của mô hình là trạng thái kẹp gồm *success*, *slip* hoặc *overforce*. Trong quá trình huấn luyện, nhãn *fail* được gộp vào nhóm *slip* để phù hợp với logic điều khiển thực tế.

Bảng 3.5: Các nhóm đặc trưng đầu vào dùng để huấn luyện mô hình MLP

Nhóm đặc trưng	Các thông tin sử dụng
Thông tin vật thể	object_class, object_instance
Đặc trưng hình ảnh từ YOLO	bbox_w_norm, bbox_h_norm, bbox_area_norm, aspect_ratio, cx_norm, cy_norm, conf
Thông tin điều khiển	grip_ratio
Đặc trưng cảm biến xúc giác	left_sum_avg, right_sum_avg, contact_area_avg, L_OUT_avg, L_IN_avg, R_IN_avg, R_OUT_avg, L_OUT_max, L_IN_max, R_IN_max, R_OUT_max, total_avg, total_max

Các thông số chính được sử dụng trong quá trình huấn luyện được trình bày trong Bảng 3.6.

Việc lựa chọn hàm kích hoạt ReLU giúp mô hình học được quan hệ phi tuyến giữa các đặc trưng đầu vào và kết quả đầu ra [9]. Bộ tối ưu Adam được sử dụng vì có khả năng hội tụ ổn định trong các bài toán học máy có tập dữ liệu vừa và nhỏ

[11]. Số vòng lặp tối đa được đặt là 2000 nhằm đảm bảo mô hình có đủ thời gian học dữ liệu, đồng thời vẫn giới hạn thời gian huấn luyện.

Bảng 3.6: Các thông số chính khi huấn luyện mô hình MLP

Mô hình	Thông số	Giá trị
MLPRegressor	Hidden layers	(64, 32)
MLPRegressor	Activation	ReLU
MLPRegressor	Solver	Adam
MLPRegressor	Max iterations	2000
MLPRegressor	Early stopping	True
MLPClassifier	Hidden layers	(128, 64, 32)
MLPClassifier	Activation	ReLU
MLPClassifier	Solver	Adam
MLPClassifier	Max iterations	2000

Sau khi huấn luyện, hai mô hình được đánh giá trên tập kiểm tra. MLPRegressor được đánh giá thông qua sai số dự đoán, trong khi MLPClassifier được đánh giá thông qua khả năng phân loại trạng thái kẹp. Cuối cùng, toàn bộ mô hình và thông tin đặc trưng được xuất ra file *grasp_mlp_models.joblib*. File này được đưa vào Raspberry Pi để chương trình chính nạp và sử dụng trong quá trình vận hành.

```
print("Saved model:", model_path)
print("Saved zip:", zip_path)

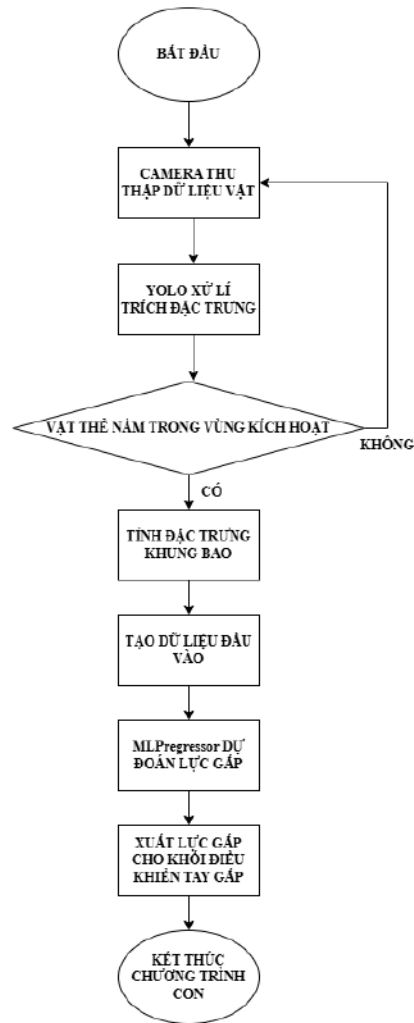
files.download(zip_path)
```

```
· Saved model: /content/grasp_mlp_models/grasp_mlp_models.joblib
  Saved zip: /content/grasp_mlp_models.zip
```

Hình 3.21: Kết quả xuất mô hình MLP sau khi huấn luyện

3.3.7 Lưu đồ nhận diện vật thể và dự đoán lực kẹp ban đầu

Lưu đồ này mô tả quá trình hệ thống sử dụng camera và mô hình YOLO để nhận diện vật thể, sau đó trích xuất các đặc trưng cần thiết làm dữ liệu đầu vào cho mô hình MLPRegressor nhằm dự đoán giá trị lực kẹp ban đầu [7, 13].



Hình 3.22: Lưu đồ nhận diện vật thể và dự đoán lực kẹp ban đầu

Khi bắt đầu chương trình con, camera tiến hành thu nhận hình ảnh vật thể trong vùng làm việc. Hình ảnh sau đó được đưa vào mô hình YOLO để xử lý và trích xuất thông tin vật thể. Nếu vật thể chưa nằm trong vùng kích hoạt, chương trình sẽ quay lại bước thu nhận hình ảnh và tiếp tục kiểm tra. Khi vật thể nằm đúng vùng kích hoạt, hệ thống sẽ tính toán các đặc trưng của khung bao như vị trí tâm, chiều rộng, chiều cao, diện tích khung bao, tỉ lệ khung bao và độ tin cậy nhận diện [7].

Các đặc trưng này được kết hợp với thông tin lớp vật thể để tạo thành vector đầu vào cho mô hình MLPRegressor. Mô hình MLPRegressor có nhiệm vụ dự đoán giá trị kẹp ban đầu phù hợp với vật thể đang được nhận diện. Giá trị dự đoán này được

gửi đến khối điều khiển tay kẹp để tay kẹp đóng lại ở mức ban đầu trước khi chuyển sang giai đoạn đọc cảm biến xúc giác và điều chỉnh thích nghi.

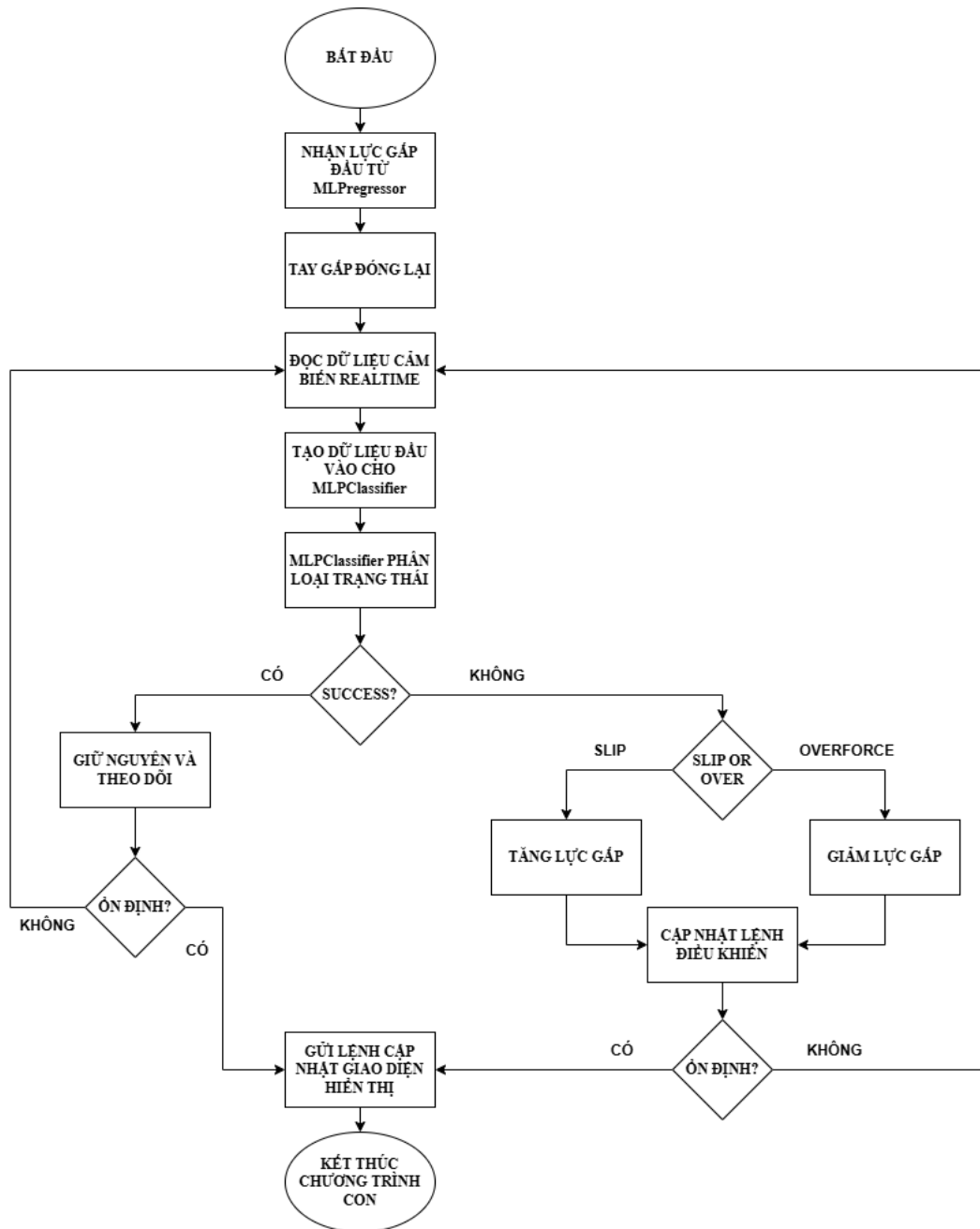
Ví dụ, khi camera phát hiện một vật thuộc lớp *can* nằm trong vùng kích hoạt, YOLO sẽ xác định khung bao của vật thể và trích xuất các đặc trưng hình ảnh tương ứng. Các đặc trưng này được đưa vào MLPRegressor để dự đoán giá trị kẹp ban đầu, chẳng hạn 0.62.

3.3.8 Lưu đồ đọc cảm biến xúc giác và điều khiển thích nghi

Lưu đồ đọc cảm biến xúc giác và điều khiển thích nghi được trình bày trong Hình 3.23. Lưu đồ mô tả quá trình hệ thống sử dụng giá trị kẹp ban đầu từ mô hình MLPRegressor, sau đó kết hợp dữ liệu cảm biến xúc giác và mô hình MLPClassifier để đánh giá trạng thái kẹp và điều chỉnh tay kẹp phù hợp [2, 4].

Khi bắt đầu chương trình, hệ thống nhận giá trị kẹp ban đầu do mô hình MLPRegressor dự đoán, sau đó điều khiển tay kẹp đóng lại theo giá trị này. Trong quá trình kẹp, cảm biến xúc giác được đọc liên tục theo thời gian thực. Dữ liệu cảm biến sau đó được xử lý để tạo thành vector đặc trưng đầu vào cho mô hình MLPClassifier. Mô hình này có nhiệm vụ phân loại trạng thái kẹp hiện tại của vật thể.

Nếu trạng thái được phân loại là *success*, hệ thống giữ nguyên lực kẹp và tiếp tục theo dõi. Khi trạng thái giữ ổn định, chương trình gửi thông tin cập nhật lên giao diện hiển thị. Nếu trạng thái chưa ổn định, hệ thống quay lại bước đọc dữ liệu cảm biến để tiếp tục kiểm tra.



Hình 3.23: Lưu đồ đọc cảm biến xúc giác và điều khiển thích nghi

Nếu trạng thái không phải *success*, hệ thống kiểm tra vật thể đang bị *slip* hay *overforce*. Với trạng thái *slip*, tay kẹp sẽ tăng lực kẹp theo một bước điều chỉnh nhỏ. Ngược lại, với trạng thái *overforce*, tay kẹp sẽ giảm lực kẹp để hạn chế tác động quá lớn lên vật thể [4]. Sau khi điều chỉnh, lệnh điều khiển mới được cập nhật và hệ thống tiếp tục đọc cảm biến để đánh giá lại trạng thái kẹp.

Ví dụ, nếu mô hình MLPRegressor dự đoán tỉ lệ kẹp ban đầu là 0.65, tay kẹp sẽ đóng đến giá trị này. Sau đó, nếu MLPClassifier phân loại trạng thái là *slip*, chương trình sẽ tăng tỉ lệ kẹp lên một lượng nhỏ, ví dụ từ 0.65 lên 0.67, rồi tiếp tục đọc cảm biến. Nếu lần đánh giá tiếp theo chuyển sang *success* và ổn định, tay kẹp sẽ giữ nguyên trạng thái. Nếu hệ thống phát hiện *overforce*, tỉ lệ kẹp sẽ được giảm xuống để tránh làm biến dạng hoặc tác động quá mạnh lên vật thể.

3.3.9 Lưu đồ giải thuật của giao diện

Lưu đồ giải thuật của giao diện được trình bày như Hình 3.24.

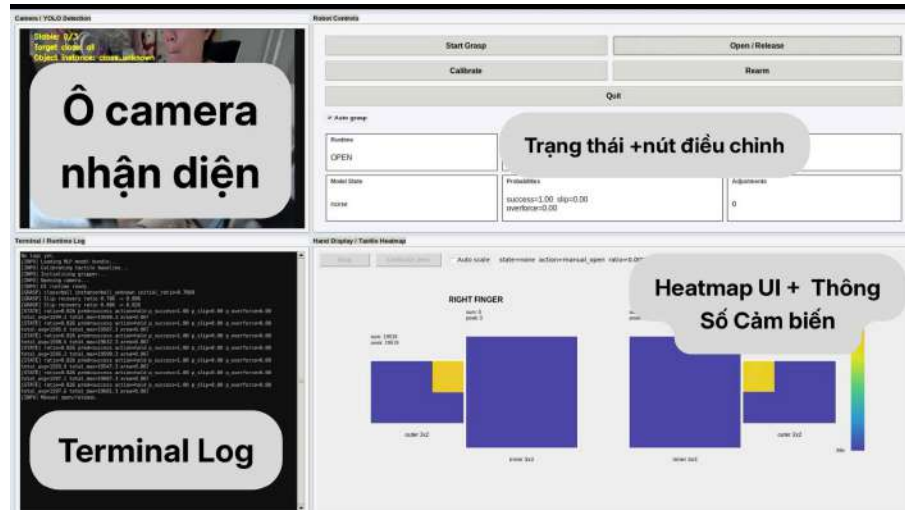


Hình 3.24: Lưu đồ giải thuật của giao diện

Khi bắt đầu chương trình, các thư viện cần thiết cho giao diện, xử lý hình ảnh, đọc dữ liệu và trao đổi trạng thái với hệ thống được nạp vào chương trình. Sau đó, giao diện được khởi tạo với các vùng hiển thị và các nút điều khiển cần thiết. Trong quá trình hoạt động, giao diện liên tục nhận thông tin từ chương trình chính, cập nhật trạng thái thiết bị và hiển thị các dữ liệu liên quan cho người dùng quan sát. Khi người dùng thoát chương trình, giao diện dừng cập nhật và kết thúc chương

trình con.

Giao diện thực tế của hệ thống được trình bày như Hình 3.25. Các khu vực chính trên giao diện gồm:



Hình 3.25: Giao diện điều khiển và giám sát hoạt động của hệ thống

- **Khu vực hiển thị camera:** hiển thị hình ảnh thời gian thực từ camera, khung nhận diện vật thể, lớp vật thể mục tiêu và trạng thái phát hiện ổn định.
- **Khu vực điều khiển tay kẹp:** gồm các nút *Start Grasp* để bắt đầu quá trình kẹp, *Open/Release* để mở hoặc nhả tay kẹp, *Calibrate* để hiệu chuẩn cảm biến, *Rearm* để khởi động lại trạng thái điều khiển và *Quit* để thoát chương trình. Ngoài ra, tùy chọn *Auto grasp* cho phép hệ thống tự động thực hiện quá trình kẹp khi điều kiện nhận diện vật thể phù hợp.
- **Khu vực trạng thái hệ thống:** hiển thị trạng thái vận hành hiện tại, hành động đang thực hiện, tỉ lệ kẹp, trạng thái dự đoán của mô hình, xác suất phân loại và số lần điều chỉnh tay kẹp.
- **Khu vực nhật ký chương trình:** hiển thị các thông báo trong quá trình chạy.
- **Khu vực hiển thị cảm biến xúc giác:** biểu diễn dữ liệu từ các ma trận cảm biến dưới dạng bản đồ nhiệt. Nhờ đó, người dùng có thể quan sát trực quan vùng tiếp xúc và mức lực tác động lên từng ngón tay của tay kẹp.

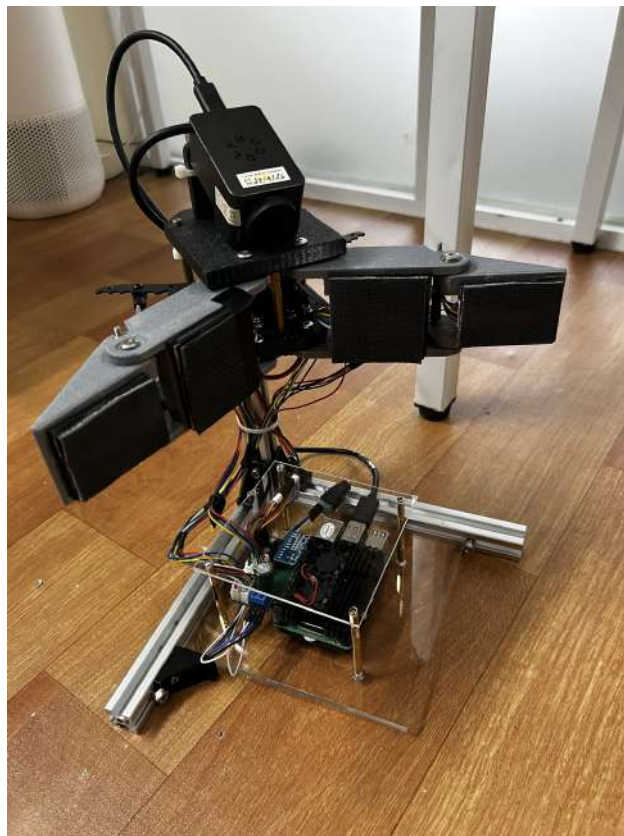
CHƯƠNG 4

KẾT QUẢ

4.1 KẾT QUẢ MÔ HÌNH THI CÔNG

4.1.1 Mô hình tay kẹp robot hoàn chỉnh

Sau quá trình thiết kế và lắp ráp, mô hình tay kẹp robot đã được hoàn thiện như Hình 4.1. Mô hình gồm tay kẹp hai ngón, camera nhận diện vật thể, cảm biến xúc giác, cơ cấu truyền động bằng servo và khối xử lý trung tâm đặt ở phần đế.



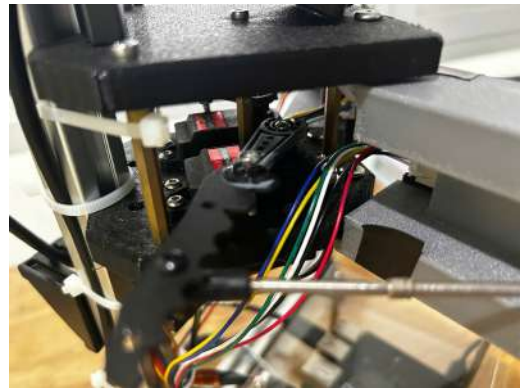
Hình 4.1: Mô hình tay kẹp robot sau khi hoàn thiện

Hình 4.2 trình bày cụm tay kẹp và cơ cấu truyền động. Hai ngón tay được điều khiển thông qua động cơ servo và các thanh liên kết, giúp tạo chuyển động đóng mở

để tiếp xúc và giữ vật thể.



(a) Cụm tay kẹp và camera



(b) Cụm động cơ servo

Hình 4.2: Cụm tay kẹp và cơ cấu truyền động

Hình 4.3 cho thấy kết cấu cơ khí của ngón tay. Các chi tiết được chế tạo bằng phương pháp in 3D, có khớp liên kết và lò xo hỗ trợ hồi vị, giúp ngón tay đóng mở ổn định hơn trong quá trình kẹp.



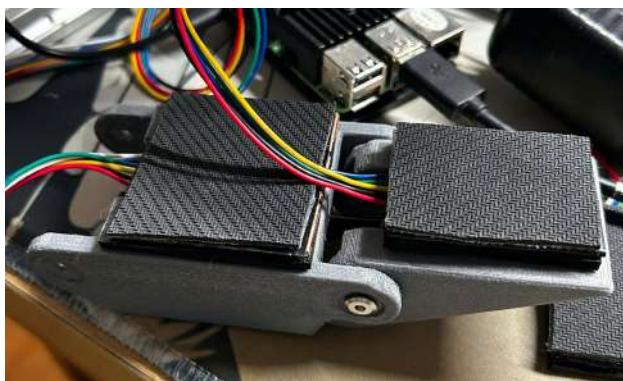
(a) Kết cấu khớp của ngón tay



(b) Cơ chế lò xo hồi vị

Hình 4.3: Kết cấu cơ khí của ngón tay kẹp

Hình 4.4 thể hiện cảm biến xúc giác sau khi được gắn lên bề mặt tiếp xúc của ngón tay. Các vùng cảm biến này dùng để ghi nhận tín hiệu khi tay kẹp tiếp xúc với vật thể, từ đó phục vụ cho quá trình đánh giá trạng thái kẹp.



Hình 4.4: Cảm biến xúc giác được gắn trên ngón tay

Hình 4.5 trình bày khối xử lý trung tâm của hệ thống. Khối này gồm Raspberry Pi, mạch đọc cảm biến và các cổng kết nối tín hiệu. Raspberry Pi đảm nhiệm vai trò xử lý hình ảnh, chạy mô hình AI, đọc dữ liệu cảm biến, điều khiển servo và cập nhật giao diện giám sát.

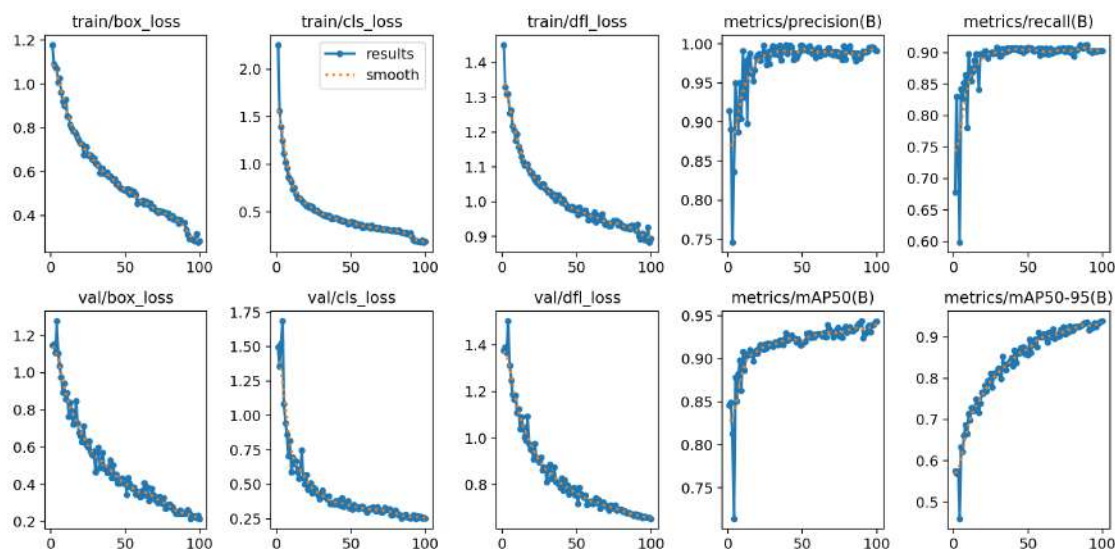


Hình 4.5: Khối xử lý trung tâm của hệ thống

Nhìn chung, mô hình thi công đã đáp ứng được các yêu cầu cơ bản của đề tài, gồm nhận diện vật thể, điều khiển tay kẹp, thu nhận dữ liệu xúc giác và xử lý trung tâm.

4.1.2 Kết quả huấn luyện mô hình YOLOv8

Mô hình YOLOv8 được huấn luyện trên tập dữ liệu gồm 3 lớp vật thể: *ball*, *can* và *bottle*. Quá trình huấn luyện được thực hiện trong 100 epoch.



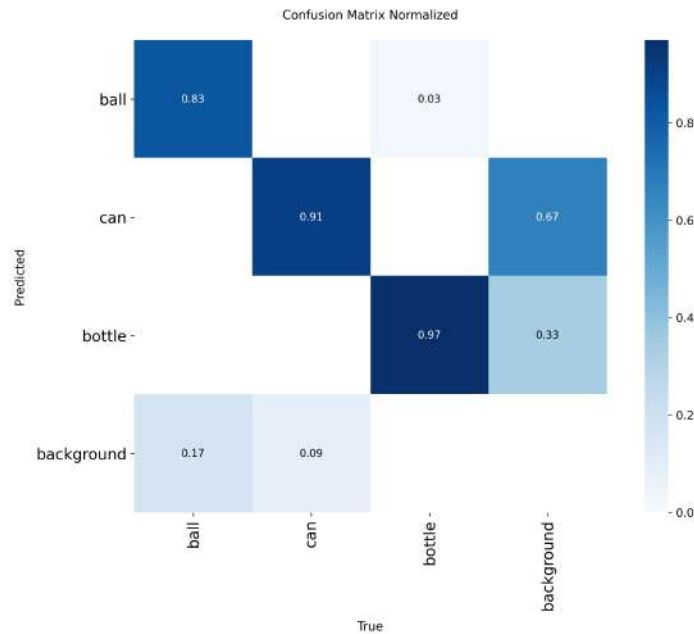
Hình 4.6: Kết quả huấn luyện mô hình YOLOv8

Hình 4.6 thể hiện sự thay đổi của các hàm mất mát và các chỉ số đánh giá qua từng epoch. Các giá trị *train/box_loss*, *train/cls_loss* và *train/dfl_loss* đều giảm dần trong quá trình huấn luyện. Điều này cho thấy mô hình đã học được cách xác định vị trí khung bao, phân loại vật thể và cải thiện độ chính xác của khung bao theo thời gian.

Ở tập kiểm tra, các giá trị *val/box_loss*, *val/cls_loss* và *val/dfl_loss* cũng có xu hướng giảm tương tự. Việc cả loss trên tập huấn luyện và tập kiểm tra đều giảm cho thấy mô hình học ổn định, không xuất hiện hiện tượng sai lệch quá lớn giữa hai tập dữ liệu.

Các chỉ số đánh giá như Precision, Recall, mAP50 và mAP50-95 tăng nhanh trong những epoch đầu và dần ổn định ở các epoch cuối. Precision đạt xấp xỉ 0.99, cho thấy phần lớn các vật thể được mô hình dự đoán là đúng lớp. Recall đạt khoảng 0.90, thể hiện mô hình nhận diện được phần lớn vật thể trong tập kiểm tra, tuy nhiên

vẫn còn một số trường hợp bị bỏ sót. Chỉ số mAP50 đạt khoảng 0.94 và mAP50-95 đạt trên 0.90, cho thấy mô hình có khả năng nhận diện và định vị vật thể tương đối tốt trong phạm vi dữ liệu của đề tài.



Hình 4.7: Ma trận nhầm lẫn chuẩn hóa của mô hình YOLOv8

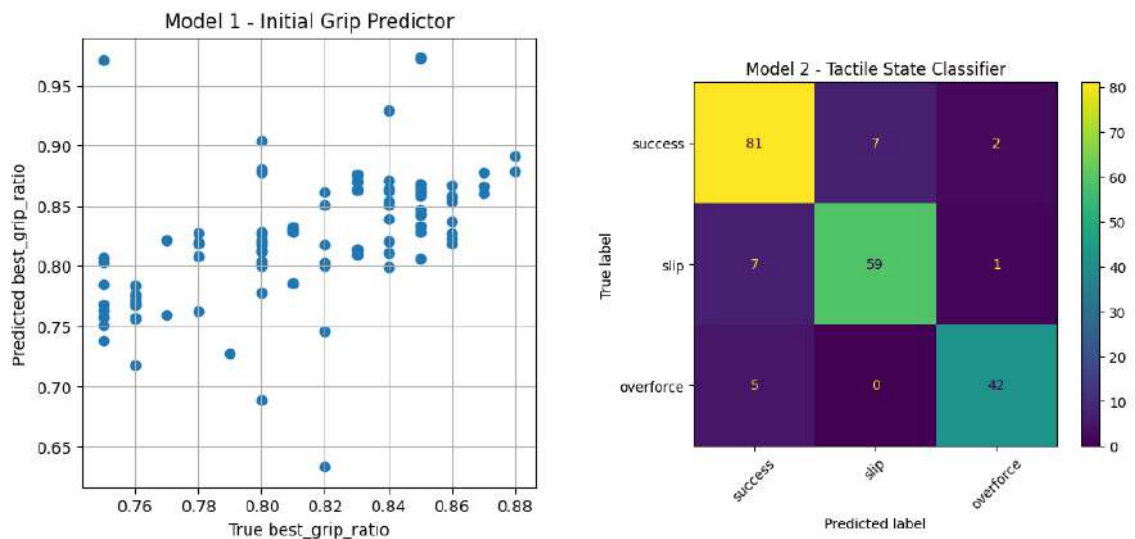
Hình 4.7 thể hiện khả năng phân loại của mô hình đối với từng lớp vật thể. Mô hình nhận diện đúng 77 mẫu thuộc lớp *ball*, 60 mẫu thuộc lớp *can* và 66 mẫu thuộc lớp *bottle*. Kết quả này cho thấy mô hình đã phân biệt được ba nhóm vật thể chính trong đề tài.

Ở ma trận chuẩn hóa, độ nhận diện đúng của lớp *ball* đạt khoảng 0.83, lớp *can* đạt khoảng 0.91 và lớp *bottle* đạt khoảng 0.97. Trong đó, lớp *bottle* có kết quả tốt nhất do đặc trưng hình dạng tương đối rõ ràng và dễ phân biệt. Một số mẫu của lớp *ball* và *can* bị bỏ sót hoặc nhầm với nền, có thể do ảnh hưởng của góc chụp, ánh sáng, bề mặt vật thể hoặc hình dạng chưa đủ khác biệt trong một số trường hợp. Hàng và cột *background* thể hiện các trường hợp mô hình bỏ sót vật thể hoặc phát hiện nhầm vùng nền thành vật thể. Tuy vẫn còn nhầm lẫn, số lượng này không lớn so với số mẫu được nhận diện đúng.

4.1.3 Kết quả huấn luyện các mô hình học máy

Kết quả của mô hình MLPRegressor và MLPClassifier được trình bày trong Hình 4.8. Trong đó, Hình 4.8(a) thể hiện mối quan hệ giữa giá trị tỉ lệ kẹp thực tế tốt nhất và giá trị tỉ lệ kẹp do mô hình MLPRegressor dự đoán. Hình 4.8(b) trình bày ma trận nhầm lẫn của mô hình MLPClassifier khi phân loại trạng thái kẹp thành ba lớp gồm *success*, *slip* và *overforce*.

Từ Hình 4.8(a), có thể thấy các điểm dự đoán phân bố quanh khoảng tỉ lệ kẹp từ 0.75 đến 0.90. Mô hình đã học được xu hướng chung của dữ liệu, tuy nhiên vẫn còn sai lệch giữa giá trị dự đoán và giá trị thực tế ở một số mẫu.



(a) Kết quả dự đoán tỉ lệ kẹp ban đầu của MLPRegressor (b) Ma trận nhầm lẫn của MLPClassifier

Hình 4.8: Kết quả huấn luyện các mô hình học máy

Kết quả đánh giá của mô hình được trình bày trong Bảng 4.1.

Bảng 4.1: Kết quả đánh giá mô hình MLPRegressor

Chỉ số đánh giá	Giá trị
MAE	0.0303
RMSE	0.0471
R^2	-0.5725

Chỉ số MAE đạt 0.0303 cho thấy sai số trung bình tuyệt đối của mô hình khoảng 3% tỉ lệ kẹp. Chỉ số RMSE đạt 0.0471, thể hiện sai số lớn hơn ở một số mẫu dự đoán. Tuy nhiên, giá trị R^2 âm cho thấy mô hình MLPRegressor chưa mô tả tốt toàn bộ quan hệ giữa đặc trưng đầu vào và tỉ lệ kẹp tối ưu.

Nguyên nhân chính là do bộ đặc trưng đầu vào của MLPRegressor hiện tại chủ yếu dựa trên thông tin nhận diện và bounding-box từ YOLO, gồm lớp vật thể, mã vật thể, chiều rộng, chiều cao, diện tích khung bao, tỉ lệ khung bao, vị trí tâm và độ tin cậy nhận diện. Các đặc trưng này chỉ phản ánh một phần thông tin hình học của vật thể trong ảnh, chưa biểu diễn đầy đủ những yếu tố ảnh hưởng trực tiếp đến quá trình kẹp như khối lượng, vật liệu, độ cứng, độ trơn bề mặt, hệ số ma sát, phân bố khối lượng và trạng thái tiếp xúc thực tế giữa tay kẹp với vật thể.

Do đó, có thể nhận xét rằng cấu trúc bộ đặc trưng đầu vào của MLPRegressor còn hạn chế. Việc chỉ sử dụng các đặc trưng thị giác từ YOLO chưa đủ để nội suy chính xác tỉ lệ kẹp cần thiết cho từng vật thể. Điều này cho thấy mô hình chưa đủ tin cậy để sử dụng như một bộ dự đoán cuối cùng cho tỉ lệ kẹp, mà chỉ phù hợp để tạo giá trị khởi tạo ban đầu cho quá trình điều khiển. Trong hệ thống hiện tại, MLPRegressor được sử dụng như một mô hình ước lượng ban đầu, giúp tay kẹp có giá trị khởi tạo phù hợp hơn so với việc dùng một mức kẹp cố định. Sau khi tay kẹp tiếp xúc với vật thể, hệ thống tiếp tục sử dụng dữ liệu cảm biến xúc giác và mô hình MLPClassifier để đánh giá trạng thái kẹp thực tế. Cơ chế điều chỉnh dựa trên các trạng thái *success*, *slip* và *overforce* giúp bù lại hạn chế của mô hình dự đoán ban đầu.

Từ Hình 4.8(b), mô hình phân loại đúng 81 mẫu *success*, 59 mẫu *slip* và 42 mẫu *overforce*. Một số mẫu bị nhầm giữa *success* và *slip*, do hai trạng thái này có thể có đặc trưng cảm biến gần giống nhau ở giai đoạn vật vừa được giữ ổn định hoặc bắt đầu trượt. Đối với lớp *overforce*, mô hình nhận diện tương đối tốt, chỉ có 5 mẫu bị nhầm sang *success*.

Bảng 4.2: Kết quả đánh giá mô hình MLPClassifier

Lớp	Precision	Recall	F1-score	Số mẫu
overforce	0.9333	0.8936	0.9130	47
slip	0.8939	0.8806	0.8872	67
success	0.8710	0.9000	0.8852	90
Accuracy	0.8922			204

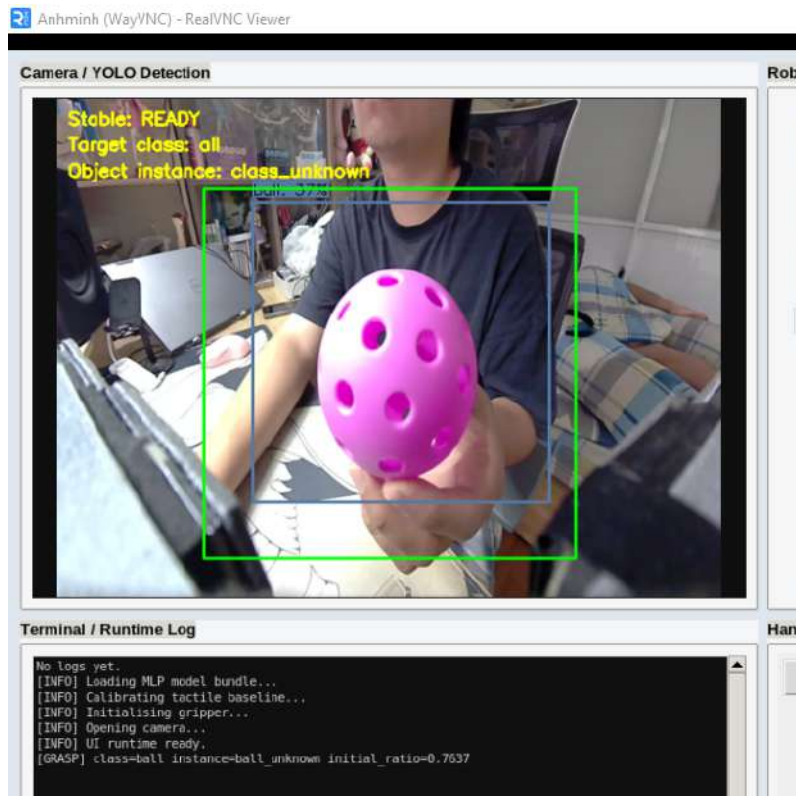
Bảng 4.2 cho thấy mô hình MLPClassifier đạt độ chính xác tổng thể 0.8922. Các chỉ số Precision, Recall và F1-score của ba lớp đều đạt trên 0.87, cho thấy mô hình có khả năng phân biệt khá tốt các trạng thái kẹp. Trong đó, lớp *overforce* có F1-score cao nhất, đạt 0.9130; lớp *success* đạt F1-score 0.8852; và lớp *slip* đạt F1-score 0.8872.

4.2 HOẠT ĐỘNG CỦA HỆ THỐNG

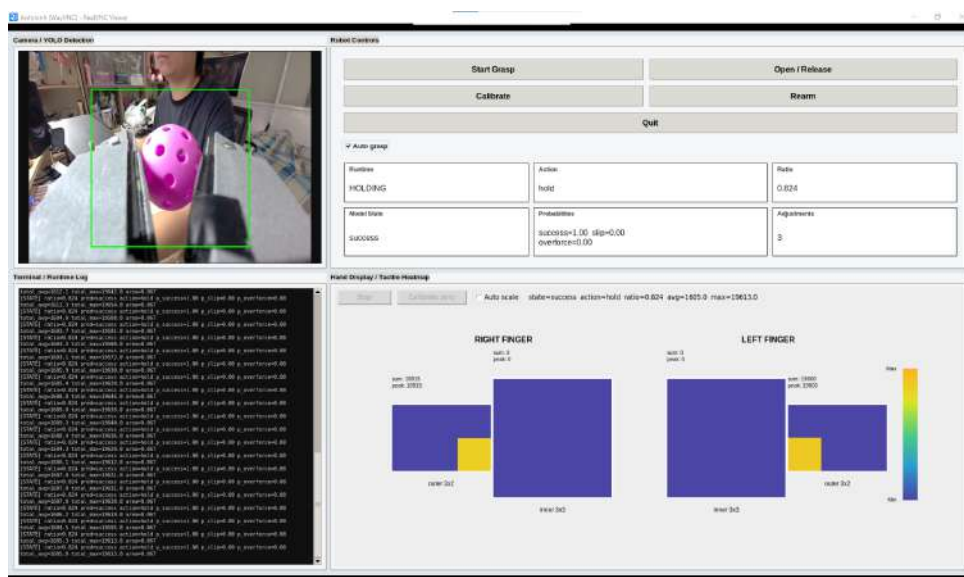
4.2.1 Quá trình vận hành với vật thể thuộc lớp *ball*

Quá trình vận hành đầu tiên được thực hiện với vật thể thuộc lớp *ball*. Nội dung đánh giá gồm khả năng nhận diện vật thể, dự đoán tỉ lệ kẹp ban đầu, đọc dữ liệu cảm biến xúc giác và tự điều chỉnh khi vật có dấu hiệu trượt.

Hình 4.9 cho thấy hệ thống nhận diện được vật thể thuộc lớp *ball* trong vùng quan sát của camera. Trạng thái nhận diện đạt mức ổn định và hệ thống dự đoán tỉ lệ kẹp ban đầu là 0.7637. Giá trị này được dùng làm lệnh điều khiển để tay kẹp bắt đầu đóng lại.



Hình 4.9: Giao diện nhận diện vật thể và dự đoán tỉ lệ kẹp ban đầu với lớp ball



Hình 4.10: Giao diện hệ thống khi tay kẹp giữ vật thể ball thành công

Sau khi tay kẹp đóng lại, hệ thống đọc dữ liệu cảm biến xúc giác và phân loại trạng thái kẹp bằng MLPClassifier. Theo nhật ký chương trình, tỉ lệ kẹp được điều chỉnh từ 0.764 lên 0.824 trước khi đạt trạng thái *success*. Trên giao diện Hình 4.10,

trạng thái mô hình là *success*, hành động là *hold*, tỉ lệ kẹp đạt 0.824 và số lần điều chỉnh là 3.



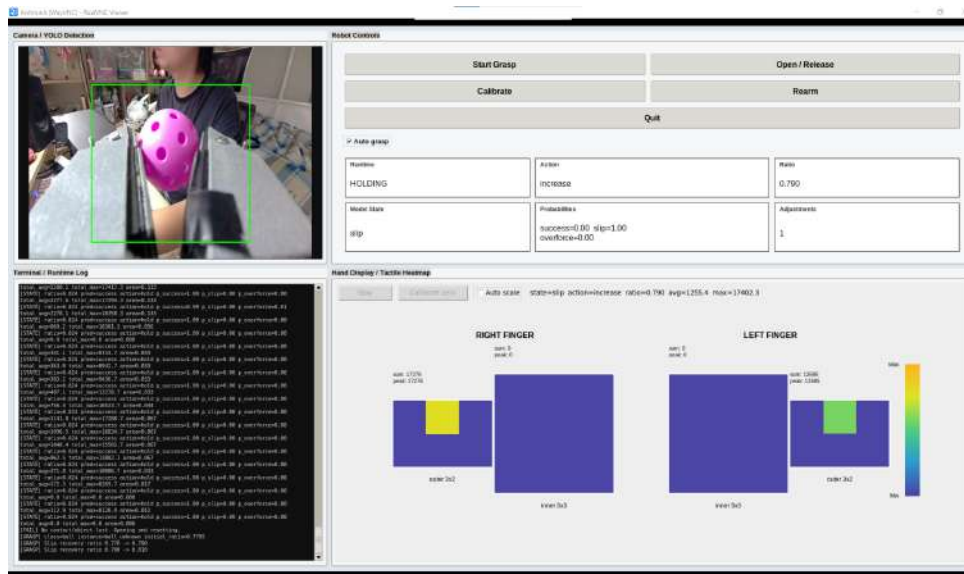
Hình 4.11: Hình ảnh thực tế tay kẹp giữ vật thể ball

Hình 4.11 minh họa kết quả thực tế khi tay kẹp giữ vật thể *ball*. Vật thể được giữ giữa hai ngón tay, các vùng cảm biến xúc giác tiếp xúc trực tiếp với bề mặt vật.

Bảng 4.3: Thông số chính trong quá trình vận hành với vật thể ball

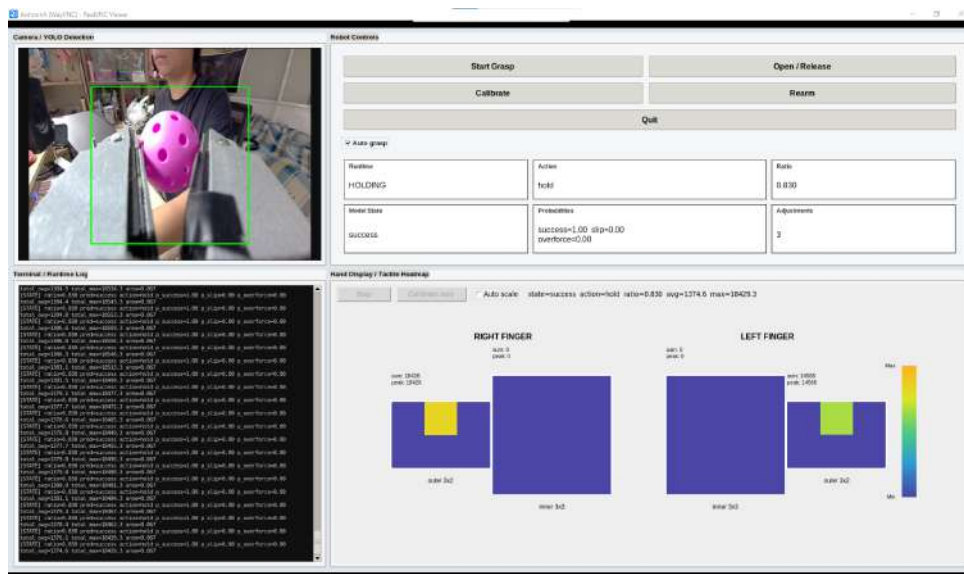
Giai đoạn	Tỉ lệ kẹp	Trạng thái	Hành động	Xác suất chính
Dự đoán ban đầu	0.7637	–	close	–
Kẹp ban đầu	0.764 → 0.824	success	hold	P(success)=1.00
Tạo trượt	0.790	slip	increase	P(slip)=1.00
Sau điều chỉnh	0.770 → 0.830	success	hold	P(success)=1.00

Bảng 4.3 tổng hợp các thông số chính từ nhật ký chương trình. Ban đầu hệ thống dự đoán tỉ lệ kẹp là 0.7637, sau đó tăng lên 0.824 để giữ vật ổn định. Khi vật được kéo nhẹ để tạo hiện tượng trượt, mô hình phát hiện trạng thái *slip* với xác suất 1.00 và tự tăng tỉ lệ kẹp. Sau quá trình điều chỉnh, tỉ lệ kẹp đạt 0.830 và trạng thái trở lại *success*.



Hình 4.12: Giao diện hệ thống khi phát hiện trạng thái trượt của vật thể ball

Hình 4.12 cho thấy hệ thống phát hiện trạng thái *slip* sau khi vật bị kéo nhẹ. Trên giao diện heatmap, các vùng màu sáng thể hiện vị trí cảm biến đang chịu tác động lớn hơn. Với vật thể dạng tròn như *ball*, tín hiệu tập trung chủ yếu ở các vùng cảm biến ngoài của hai ngón tay.



Hình 4.13: Giao diện hệ thống sau khi điều chỉnh và giữ lại vật thể ball thành công

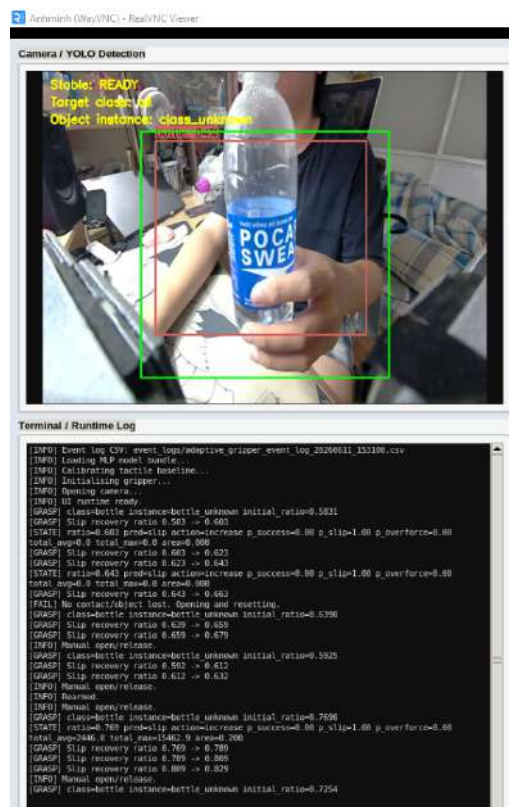
Hình 4.13 cho thấy sau khi tăng tỉ lệ kẹp, hệ thống chuyển lại trạng thái *success*, hành động là *hold*. Nhìn chung, quá trình vận hành với vật thể *ball* chứng minh hệ

thống có thể nhận diện vật thể, dự đoán tỉ lệ kẹp ban đầu, phát hiện trượt và tự điều chỉnh để giữ vật ổn định.

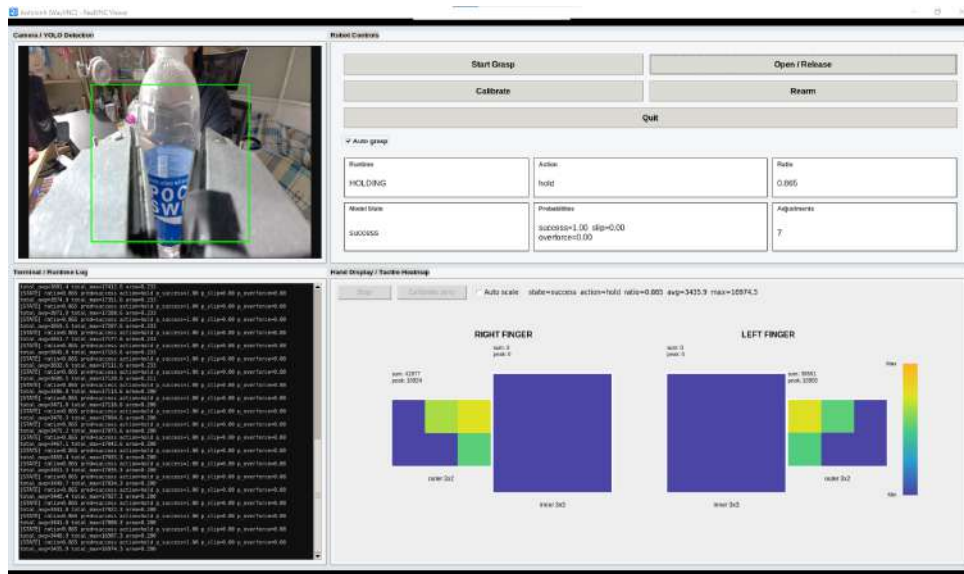
4.2.2 Quá trình vận hành với vật thể thuộc lớp *bottle*

Quá trình vận hành tiếp theo được thực hiện với vật thể thuộc lớp *bottle*. Ở lượt này, hệ thống nhận diện chai nước, dự đoán tỉ lệ kẹp ban đầu, giữ vật ổn định và tiếp tục tự điều chỉnh khi vật có dấu hiệu trượt.

Hình 4.14 cho thấy hệ thống nhận diện được vật thể thuộc lớp *bottle* trong vùng làm việc của camera. Nhật ký chương trình ghi nhận ở lượt minh họa này hệ thống dự đoán tỉ lệ kẹp ban đầu là 0.5831. Giá trị này được dùng làm lệnh điều khiển để tay kẹp bắt đầu đóng lại.



Hình 4.14: Giao diện nhận diện vật thể và dự đoán tỉ lệ kẹp ban đầu với lớp *bottle*



Hình 4.15: Giao diện hệ thống khi tay kẹp giữ vật thể bottle thành công

Sau khi tay kẹp đóng lại và tự điều chỉnh, hệ thống đạt trạng thái *success* như Hình 4.15. Khi đó tỉ lệ kẹp đạt 0.865, số lần điều chỉnh là 7 và xác suất *success* đạt 1.00. Dữ liệu cảm biến cho giá trị trung bình khoảng 3435.9, cực đại khoảng 16974.3 và vùng tiếp xúc khoảng 0.200.

Hình 4.16 minh họa kết quả thực tế khi tay kẹp giữ vật thể *bottle*. Trên giao diện heatmap, tín hiệu cảm biến tập trung chủ yếu ở các ô cảm biến ngoài của hai ngón tay, phù hợp với đặc điểm tiếp xúc của vật thể dạng trụ.

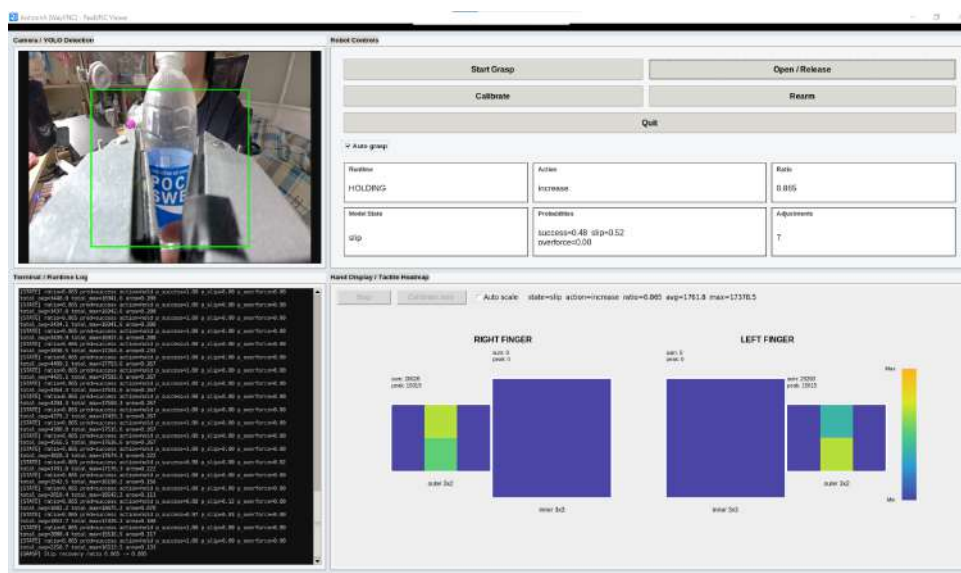


Hình 4.16: Hình ảnh thực tế tay kẹp giữ vật thể bottle

Bảng 4.4: Thông số chính trong quá trình vận hành với vật thể bottle

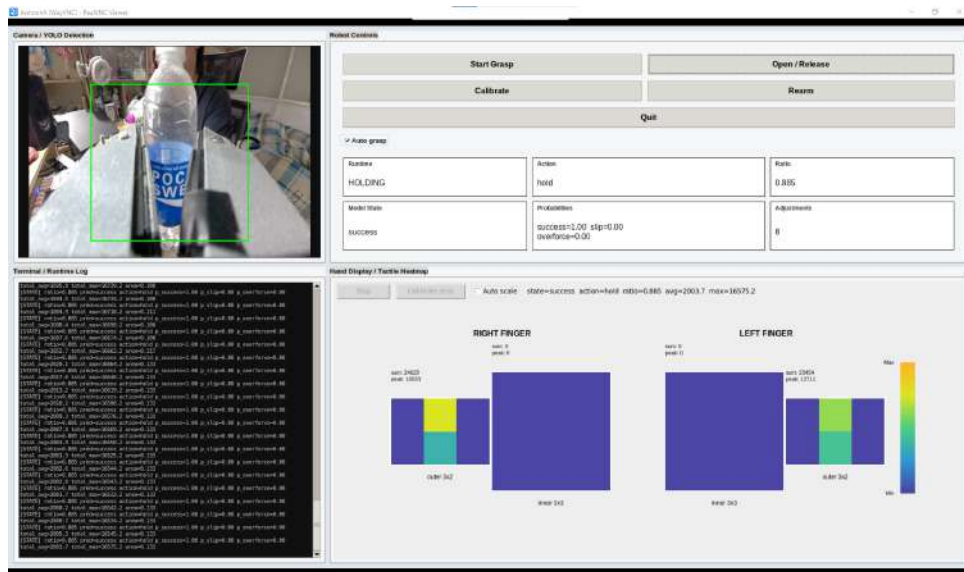
Giai đoạn	Tỉ lệ kẹp	Trạng thái	Hành động	Xác suất chính
Dự đoán ban đầu	0.5831	—	close	—
Kẹp ban đầu	0.865	success	hold	P(success)=1.00
Tạo trượt	0.865	slip	increase	P(slip)=0.52
Sau điều chỉnh	0.865 → 0.885	success	hold	P(success)=1.00

Bảng 4.4 tổng hợp các thông số chính từ nhật ký chương trình. Từ tỉ lệ dự đoán ban đầu 0.5831, hệ thống đã tăng dần lực kẹp để giữ được vật. Khi vật bị kéo nhẹ, mô hình phát hiện trạng thái *slip* và tiếp tục tăng tỉ lệ kẹp từ 0.865 lên 0.885 để đưa trạng thái trở lại *success*.



Hình 4.17: Giao diện hệ thống khi phát hiện trạng thái trượt của vật thể bottle

Hình 4.17 thể hiện thời điểm hệ thống phát hiện vật thể *bottle* có dấu hiệu trượt. Trên giao diện, trạng thái là *slip*, hành động là *increase*, tỉ lệ kẹp là 0.865, với xác suất *slip* là 0.52 và xác suất *success* là 0.48.



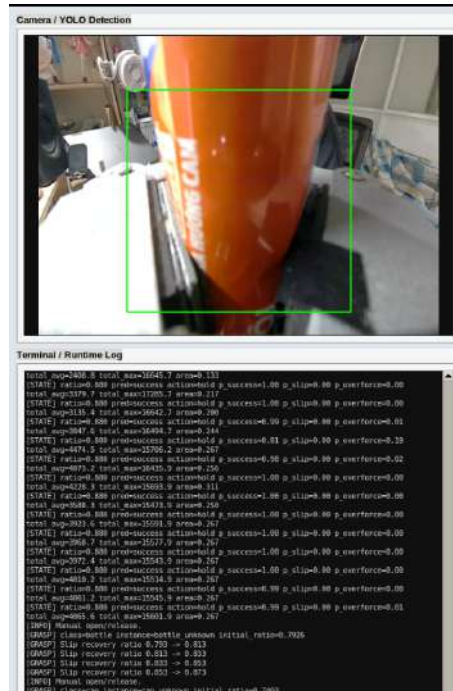
Hình 4.18: Giao diện hệ thống sau khi điều chỉnh và giữ lại vật thể bottle thành công

Hình 4.18 cho thấy sau khi tăng tỉ lệ kẹp lên 0.885, hệ thống quay lại trạng thái *success* với xác suất 1.00. Nhìn chung, kết quả này cho thấy hệ thống có thể nhận diện, kẹp và tự điều chỉnh để giữ ổn định vật thể thuộc lớp *bottle*.

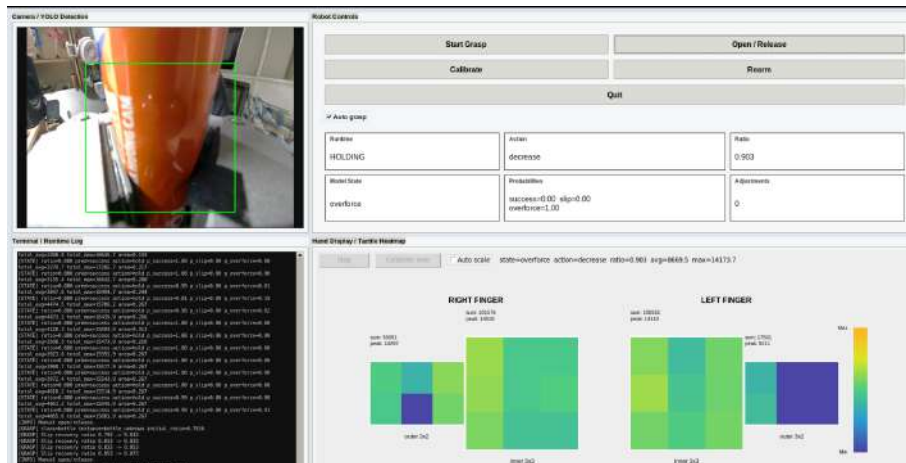
4.2.3 Hoạt động của hệ thống với vật thể lớp *can*

Hệ thống được kiểm tra với vật thể thuộc lớp *can*. Trường hợp này cho thấy hệ thống không chỉ nhận diện và kẹp được vật, mà còn có khả năng phát hiện trạng thái *overforce* và tự điều chỉnh lại tỉ lệ kẹp.

Hình 4.19(a) cho thấy hệ thống nhận diện đúng vật thể thuộc lớp *can*. Theo nhật ký chương trình, tỉ lệ kẹp ban đầu được dự đoán là 0.9028. Hình 4.19(b) thể hiện trạng thái hệ thống khi phát hiện quá lực với vật thể lớp *can*.



(a) Giao diện nhận diện vật thể lớp can



(b) Trạng thái hệ thống khi phát hiện quá lực

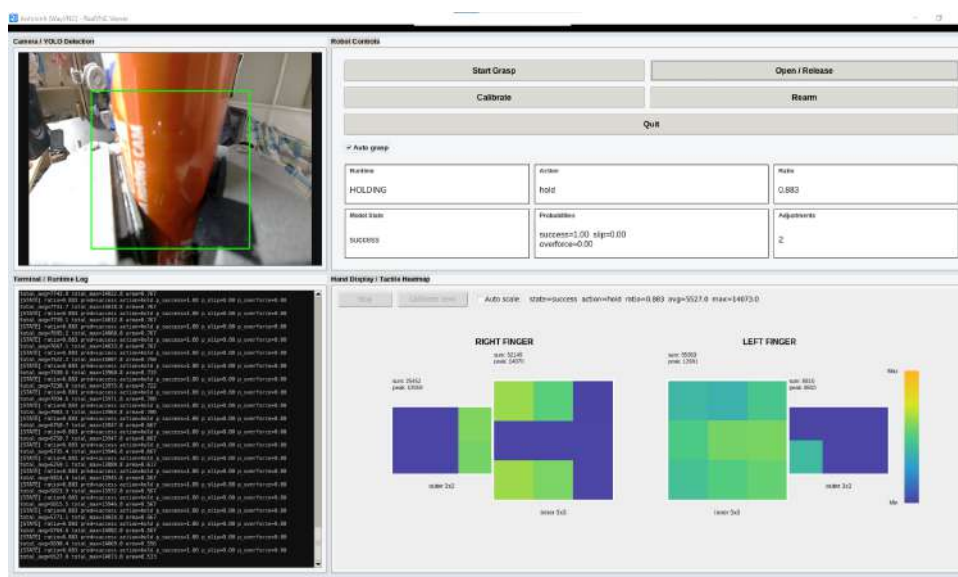
Hình 4.19: Quá trình nhận diện và phát hiện trạng thái quá lực với vật thể lớp can

Ở thời điểm này, mô hình phân loại trạng thái xác định vật đang ở trạng thái *overforce*, với hành động điều khiển là *decrease*. Tỷ lệ kẹp ghi nhận là 0.903, xác suất *overforce* đạt 1.00. Đồng thời, các chỉ số cảm biến tăng cao với *total_avg* khoảng 8669.5, *total_max* khoảng 14173.7 và vùng tiếp xúc khoảng 0.822. Heatmap xúc giác cho thấy nhiều ô cảm biến ở cả vùng trong và vùng ngoài được kích hoạt, phản ánh lực tiếp xúc lớn trên bề mặt lon.

Bảng 4.5: Một số thông số chính trong quá trình vận hành với vật thể lớp can

Giai đoạn	Tỉ lệ kẹp	Trạng thái	Hành động	Thông số chính
Dự đoán ban đầu	0.9028	–	close	–
Phát hiện quá lực	0.903	overforce	decrease	total_avg = 8669.5
Sau điều chỉnh	0.883	success	hold	total_avg = 5527.0

Sau khi phát hiện quá lực, hệ thống tự giảm tỉ lệ kẹp từ 0.903 xuống 0.893 và tiếp tục giảm còn 0.883. Kết quả sau điều chỉnh được thể hiện ở Hình 4.20.



(a) Giao diện sau khi điều chỉnh về trạng thái thành công



(b) Hình ảnh thực tế tay kẹp giữ vật

Hình 4.20: Kết quả kẹp vật thành công sau khi xử lý trạng thái quá lực

Sau điều chỉnh, hệ thống chuyển sang trạng thái *success* với xác suất 1.00, hành

động trở lại *hold* và số lần điều chỉnh là 2. Chỉ số *total_avg* giảm còn khoảng 5527.0, vùng tiếp xúc giảm còn khoảng 0.533. Kết quả này cho thấy hệ thống có thể tự động giảm lực kẹp khi phát hiện quá lực, từ đó giữ vật ổn định hơn và hạn chế nguy cơ làm biến dạng vật thể.

4.2.4 Nhận xét đánh giá

Qua quá trình vận hành thực tế, hệ thống đã đạt được một số nội dung so với mục tiêu ban đầu:

- Hệ thống đã nhận diện được các vật thể thuộc các lớp đã huấn luyện như *ball*, *can* và *bottle* thông qua mô hình YOLOv8.
- Hai mô hình MLP đã thực hiện được các chức năng chính: MLPRegressor dự đoán tỉ lệ kẹp ban đầu và MLPClassifier phân loại trạng thái kẹp gồm *success*, *slip* và *overforce*.
- Hệ thống cảm biến xúc giác đã ghi nhận được tín hiệu tiếp xúc trong quá trình kẹp và hiển thị trực quan thông qua giao diện heatmap.
- Khi phát hiện trạng thái bất thường, hệ thống có khả năng tự điều chỉnh tỉ lệ kẹp: tăng tỉ lệ kẹp khi phát hiện *slip* và giảm tỉ lệ kẹp khi phát hiện *overforce*.
- Giao diện hệ thống hiển thị được hình ảnh camera, trạng thái mô hình, tỉ lệ kẹp, xác suất phân loại, nhật ký vận hành và dữ liệu cảm biến theo thời gian thực.

CHƯƠNG 5

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết luận

Sau quá trình nghiên cứu và thực hiện đề tài “Xây dựng hệ thống tay kẹp robot thích nghi tích hợp cảm biến xúc giác và trí tuệ nhân tạo”, hệ thống đã cơ bản hoàn thành các mục tiêu đã đề ra. Đề tài đã thiết kế được mô hình tay kẹp hai ngón có tích hợp cảm biến xúc giác, camera nhận diện vật thể, mô hình học máy và giao diện giám sát.

Hệ thống có khả năng nhận diện vật thể bằng YOLO, dự đoán tỉ lệ kẹp ban đầu bằng MLPRegressor, đọc dữ liệu cảm biến xúc giác và sử dụng MLPClassifier để phân loại trạng thái kẹp. Từ đó, tay kẹp có thể điều chỉnh theo các trạng thái như kẹp thành công, trượt hoặc quá lực. Giao diện hiển thị cũng giúp người dùng theo dõi hình ảnh camera, dữ liệu cảm biến, trạng thái mô hình và quá trình điều khiển.

Ưu điểm của hệ thống là kết hợp được thị giác máy tính, cảm biến xúc giác và học máy trong cùng một mô hình tay kẹp thích nghi. Hệ thống không chỉ kẹp theo giá trị cố định mà có thể điều chỉnh dựa trên phản hồi thực tế từ cảm biến. Tuy nhiên, hệ thống vẫn còn một số hạn chế như tập dữ liệu huấn luyện còn ít, số lượng vật thể thử nghiệm chưa nhiều, cảm biến xúc giác tự chế chưa thật sự ổn định và kết quả nhận diện còn bị ảnh hưởng bởi ánh sáng, góc đặt vật và môi trường xung quanh.

Nhìn chung, đề tài đã xây dựng được mô hình tay kẹp thích nghi ở mức thử nghiệm, đáp ứng được các chức năng chính đã đặt ra và tạo cơ sở để tiếp tục cải tiến trong các nghiên cứu tiếp theo.

5.2 Hướng phát triển

Trong tương lai, đề tài có thể được phát triển theo các hướng sau:

- Mở rộng tập dữ liệu với nhiều loại vật thể hơn, đa dạng hơn về kích thước, khối lượng, chất liệu và điều kiện ánh sáng.
- Cải thiện cơ khí tay kẹp, sử dụng động cơ có độ chính xác và độ bền cao hơn để tăng độ ổn định khi kẹp vật.
- Nâng cấp cảm biến xúc giác, nghiên cứu sử dụng cảm biến có độ phân giải cao hơn hoặc mạch cảm biến dạng dẻo để tăng độ chính xác và độ bền.
- Tối ưu chương trình chạy trên Raspberry Pi nhằm giảm tải xử lý, tăng tốc độ phản hồi và cải thiện khả năng hoạt động thời gian thực.
- Cải thiện mô hình học máy bằng cách thu thập thêm dữ liệu, thử nghiệm thêm các kiến trúc mô hình khác và tối ưu quá trình phân loại trạng thái kẹp.
- Cải thiện thuật toán điều khiển thích nghi, tối ưu bước tăng giảm tỉ lệ kẹp và bổ sung cơ chế điều khiển lực chính xác hơn dựa trên phản hồi xúc giác.
- Thử nghiệm hệ thống trong nhiều điều kiện thực tế hơn, chẳng hạn như thay đổi vị trí đặt vật, điều kiện ánh sáng, bề mặt tiếp xúc và mức độ trơn trượt của vật thể.

TÀI LIỆU THAM KHẢO

- [1] Z. Kappassov, J. A. Corrales, and V. Perdereau, “Tactile sensing in dexterous robot hands – Review,” *Robotics and Autonomous Systems*, vol. 74, pp. 195–220, 2015.
- [2] R. Calandra, A. Owens, D. Jayaraman, J. Lin, W. Yuan, J. Malik, E. H. Adelson, and S. Levine, “More than a feeling: Learning to grasp and regrasp using vision and touch,” *IEEE Robotics and Automation Letters*, vol. 3, no. 4, pp. 3300–3307, 2018.
- [3] M. Costanzo, G. De Maria, C. Natale, and S. Pirozzi, “Design and calibration of a force/tactile sensor for dexterous manipulation,” *Sensors*, vol. 19, no. 4, Art. no. 966, 2019.
- [4] F. R. Hogan, M. Bauza, O. Canal, E. Donlon, and A. Rodriguez, “Tactile regrasp: Grasp adjustments via simulated tactile transformations,” in *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018, pp. 2963–2970.
- [5] L. Yuan, H. Qu, and J. Li, “Velostat sensor array for object recognition,” *IEEE Sensors Journal*, vol. 22, no. 2, pp. 1692–1701, 2022.
- [6] S. Stassi, V. Cauda, G. Canavese, and C. F. Pirri, “Flexible tactile sensing based on piezoresistive composites: A review,” *Sensors*, vol. 14, no. 3, pp. 5296–5332, 2014.
- [7] M. Yaseen, “What is YOLOv8: An in-depth exploration of the internal features of the next-generation object detector,” arXiv preprint arXiv:2408.15857, 2024.
- [8] C. M. Bishop, *Pattern Recognition and Machine Learning*. New York, NY, USA: Springer, 2006.

- [9] S. Haykin, *Neural Networks and Learning Machines*, 3rd ed. Upper Saddle River, NJ, USA: Pearson, 2009.
- [10] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [11] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” arXiv preprint arXiv:1412.6980, 2014.
- [12] A. Dzedzickis, E. Sutinyš, V. Bucinskas, U. Samukaite-Bubniene, B. Jakstys, A. Ramanavicius, and I. Morkvenaite-Vilkonciene, “Polyethylene-carbon composite (Velostat®) based tactile sensor,” *Polymers*, vol. 12, no. 12, Art. no. 2905, 2020.
- [13] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

PHỤ LỤC

Thư mục Google Drive chứa hướng dẫn sử dụng hệ thống và source code